# План развития: от 0 до Senior

# 🗺️ План развития: с абсолютного нуля до Senior (.NET / C#)

> Это дорожная карта, а не кусок теории. Ты идёшь по ней шаг за шагом:
> читаешь «как для ребёнка» → сразу делаешь задачи → разбираешь ошибки.
> Каждый уровень заканчивается **чек-поинтом**: если прошёл — переходишь дальше.

---

## Как пользоваться планом

- **Уровень** = этап твоего роста. Не перепрыгивай — пропущенный уровень даёт трещину в базе.
- **Аналогия** = объяснение «как для ребёнка». Если непонятно — перечитай её первой.
- **Задачи** = практика строго по пройденной теме. Сначала реши сам, потом смотри подсказку.
- **Разбор** = на что смотреть в своём коде, чтобы учиться на ошибках.
- **Чек-поинт** = мини-экзамен. Дробить «можешь объяснить и сделать» — двигайся дальше.

Формула каждого дня: **20 мин теории → 40 мин практики → 10 мин разбора ошибок**.

---

## Уровень 0 — Абсолютный ноль (недели 1–2)

**Цель:** понять, что такое программа, и написать первый код, не пугаясь терминов.

### Теория «как для ребёнка»

- **Компьютер — очень послушный, но глупый помощник.** Он делает ровно то, что ему сказали. Твоя работа — объяснить ему точно.
- **Программа — рецепт.** Как рецепт торта: «возьми яйца, перемешай, испеки». Код — такой же рецепт для компьютера.
- **Переменная — коробочка с наклейкой.** Ты пишешь имя на коробочке ( `age` ) и кладёшь внутрь значение ( `25` ). Потом обращаешься по имени.
- **Тип — размер и содержание коробочки.** `int` — коробочка для целых чисел, `string` — для текста, `double` — для дробей, `bool` — для «да/нет».
- **Консоль — телеграм-переписка с компьютером.** `Console.WriteLine("...")` — ты отправляешь сообщение компьютеру, `Console.ReadLine()` — компьютер спрашивает тебя.
- **Ошибка компиляции — компьютер говорит: «Я не понял твою записку».** Он честно указывает, где промах.

### Задачи (по пройденной теме)

1. **Hello, World!** — выведи приветствие. *Подсказка:* `Console.WriteLine("Привет!");` — *точка с запятой в конце обязательна.*
2. **Коробочки** — создай переменные: возраст (int), имя (string), температура (double), учится ли (bool). Выведи их. *Подсказка:* `int age = 25;` *потом* `Console.WriteLine(age);`
3. **Двестиог** — спроси имя и возраст, выведи «Привет, X! Тебе Y лет». *Подсказка:* `Console.ReadLine()` *возвращает строку; чтобы получить число, используй* `int.Parse(...)`.
4. **Хитрый калькулятор** — сложи два числа, выведи результат. *Подсказка:* `int sum = a + b;`
5. **Заведи «анкету»** — запроси город и любимый язык, выведи фразу. *Подсказка: интерполяция* `$"..."` *подставляет значение в строку.*

### Разбор (на что смотреть)

- У всех ли строк в конце стоит `;`? Отсутствие — №1 ошибка новичка.
- Имена объяснимы? `int x = 5` плохо, `int age = 5` хорошо.
- Не пытался ли написать `Console.WriteLine(age)` без объявления переменной?

### Чек-поинт

Создай проект сам, выведи «Привет, мир», спроси имя и выведи приветствие. Если получилось без подглядывания — уровень 0 пройден.

---

## Уровень 1 — Junior Starter (недели 3–6)

**Цель:** научить компьютер принимать решения и повторять действия.

### Теория «как для ребёнка»

- **Если/иначе — развилка на дороге.** `if (age >= 18)` — «если это правда, иди налево, иначе — направо». Компьютер — единственный, кто не может «почти»: либо да, либо нет.
- **Цикл — карусель.** `for` — «прокатись ровно 5 кругов», `while` — «крутись, пока музыка играет». `foreach` — «пройдись по каждому человеку в очереди».
- **Метод — инструкция с именем.** Написал один раз «как пожарить яйцо», а потом зовёшь `FryEgg()` сколько угодно раз. Дублирование кода — это «скопировал рецепт 10 раз и забыл, что в одном из них соли не было».
- **Массив — коробка с клетками.** `int[] numbers = { 1, 2, 3 };` — коробка, где первая клетка под номером 0.

### Задачи

1. **FizzBuzz** — числа 1–100: делятся на 3 → «Fizz», на 5 → «Buzz», на оба → «FizzBuzz». *Подсказка: проверяй «на оба» первым (`i % 15 == 0`).*
2. **Калькулятор с меню** — «+», «-», «*», «/» через `switch`. *Подсказка: `switch`-выражение читается лучше длинного `if`.*
3. **Таблица умножения** — вложенные циклы. *Подсказка: внешний цикл — строки, внутренний — столбцы.*
4. **Угадай число** — компьютер загадывает, ты угадываешь с подсказками «больше/меньше». *Подсказка: `Random.Shared.Next(1, 101)`, продолжай, пока не угадал.*
5. **Сумма цифр числа** — `123` → `6`. *Подсказка: `% 10` берёт последнюю цифру, `/ 10` отбрасывает её.*
6. **Метод `IsPalindrome`** — проверь, читается ли слово одинаково с двух концов. *Подсказка: сравни первый символ с последним, второй с предпоследним, и так до середины.*

### Разбор

- Нет ли «стрелочной кодировки»: `else` после `return` — лишний уровень вложенности.
- Нет ли дублирования условия в цепочке `if`.
- Может ли программа зациклиться навсегда? Есть ли выход из `while (true)`?

### Чек-поинт

Напиши игру «Угадай число» целиком: загадывание, подсказки, счётчик попыток, выход. Объясни вслух, чем `for` отличается от `while`.

---

## Уровень 2 — Junior (недели 7–12)

**Цель:** понять объектно-ориентированное мышление — главное отличие профессионала от любителя.

### Теория «как для ребёнка»

- **Класс — чертёж. Объект — изделие по чертёжу.** Чертёж «Автомобиль» (класс) описывает свойства (`Цвет`, `Скорость`) и действия (`Ехать()`). А конкретная машина на парковке — объект.
- **Инкапсуляция — закрытая приборная панель.** Ты не лезешь в мотор руками. Ты жмёшь педаль (метод). А если бы движок был открыт — кто-нибудь сломал бы его. Поэтому поля делают `private`, а доступ — только через методы.

- **Наследование — семейство.** «Собака» — это «Животное» + что-то своё. Базовый класс — общее, наследник — частное.
- **Полиморфизм — один пульт для всех.** Место для «Животного» может быть любым животным; команда `Meow()` у кошки скажет «мяу», у собаки — «гав».
- **Интерфейс — договор на услуги.** «Я умею платить» (`IPayable`). Неважно, кто это — человек или магазин. Главное — умеет платить.
- **Коллекция List<T> — динамический список.** Ты можешь добавлять и убирать записи, не задумываясь о размере.

### Задачи

1. **Класс `BankAccount`** — баланс скрыт (`private`), пополнение и снятие через методы с проверками. *Подсказка: число в минус уходить не должно.*
2. **Класс `Student`** — валидация в свойствах: возраст от 14 до 100, имя непустое. *Подсказка: сеттер бросает исключение при неверном значении.*
3. **Иерархия фигур** — абстрактный класс `Shape`, метод `Area()`, наследники `Circle`, `Rectangle`, `Triangle`. *Подсказка: арифметика для круга — $\pi r^2$.*
4. **Интерфейс `IPayable`** — `Salary` и `Contract` реализуют его по-разному. *Подсказка: у зарплаты — фиксированный оклад, у контракта — ставка × часы.*
5. **Список задач** — `List<Task>`: добавить, удалить, отметить выполненной. *Подсказка: для удаления по условию удобен `RemoveAll`.*
6. **Очередь печати** — `Queue<int>`: добавить задачу, обработать в порядке поступления. *Подсказка: `Enqueue` добавляет, `Dequeue` забирает первым.*

### Разбор

- Не открыл ли ты поле `public` напрямую (ломка инкапсуляции)?
- Не отдал ли наружу `List<T>` (когда внутри) — лучше `IReadOnlyList<T>`.
- Нет ли «God-класса»: всё в `Program` вместо отдельных классов.

### Чек-поинт

Спроектируй класс `Order` с инкапсуляцией, наследованием и интерфейсом. Объясни кому-нибудь (или резиновой утке), зачем нужна инкапсуляция.

---

## Уровень 3 — Middle (недели 13–20)

**Цель:** писать устойчивый код: обрабатывать ошибки, тестировать, работать с данными и асинхронностью.

## Теория «как для ребёнка»

- **Исключение — «звонок о помощи».** Когда метод не может выполнить работу, он не молчит, а бросает исключение (кричит: «Внимание, деление на ноль!»). А `try/catch` — это «страховочная сетка».
- **Тест — страховка перед ремонтом.** Прежде чем менять проводку, ты проверяешь, что лампочки горели. Тесты делают то же: доказывают, что старое поведение не сломалось.
- **Лямбда — функция-аноним.** «Вот эта функция — прямо здесь, без имени». Как записка: не нужна отдельная страница — пишешь выражение на ходу.
- **LINQ — вопросы к коллекциям.** Вместо циклов ты «спрашиваешь»: `Where` — «дай тех, кто подходит», `OrderBy` — «отсортируй», `Select` — «возьми только имена».
- **Async/await — разговор без блокировки.** Ты заказал пиццу и не стоишь часами у телефона — занимаешься своим делом и получаешь ответ, когда готово. `await` — «дождись, но не блокируй очередь».
- **JSON — «форма бланка» для обмена данными.** Все системы понимают этот формат: программа записывает данные в JSON, а читатель разбирает его.

## Задачи

1. **Собственное исключение `InvalidEmailException`** — выброси его при неверном email. *Подсказка: наследуйся от `Exception`, добавь своё сообщение.*
2. **Юнит-тесты на банковский счёт** — проверь пополнение, снятие, попытку уйти в минус. *Подсказка: `Assert.Throws<...>` — для проверки исключений.*
3. **LINQ-пайплайн** — из списка людей отбери взрослых, отсортируй по имени, возьми первые 3. *Подсказка: `Where(...).OrderBy(...).Take(3)`.*
4. **Асинхронная загрузка** — три «источника» данных через `Task.WhenAll`, дождись всех. *Подсказка: `await Task.WhenAll(t1, t2, t3)`.*
5. **Заметки в JSON** — сохрани список заметок в файл и загрузи при старте. *Подсказка: `JsonSerializer.Serialize/Deserialize`.*
6. **Файловый логгер** — пиши строки с датой в файл. *Подсказка: `File.AppendAllText(path, line + Environment.NewLine)`.*

## Разбор

- Глотаешь ли ошибки пустым `catch {}`? Так делать нельзя.
- Используешь ли `.Result` / `.Wait()` вместо `await`? Это путь к зависаниям.
- Понимаешь ли «отложенное выполнение» LINQ? `Where` не считается, пока не переберёшь.

## Чек-поинт

Собери мини-проект: консольная «книга контактов» с сохранением в JSON, валидацией email и юнит-тестами на логику.

---

## Уровень 4 — Senior (недели 21–30)

**Цель:** проектировать системы — архитектура, безопасность, производительность, наблюдаемость, работа в команде.

### Теория «как для ребёнка»

- **SOLID — 5 правил «не ломать».** Например, Single Responsibility: «каждый класс делает одну работу», чтобы менять одно, не задевая другое.
- **Архитектура — планировка дома.** Сначала рисуем план (слои, интерфейсы), потом строим. Дом по плану просто расширять; хаотично построенный — рушится при любой переделке.
- **Dependency Injection — «принеси мне кофе».** Сервис не идёт сам на кухню (не создаёт зависимости), а просит: «дай мне кофе» (передаётся извне). Так легче подменять для тестов.
- **Безопасность — замок и ключ.** JWT — пропуск с печатью: сервер проверяет подпись и пускает или нет. Пароли хранят только в виде кубиков хэша, не в открытом виде.
- **Наблюдаемость — приборная панель пилота.** Логи (что случилось), метрики (сколько), трейсы (где время) — чтобы понять самолёт до того, как он упадёт.
- **Производительность — «не возить кирпичи по одной штуке».** `List.Contains` в цикле — тяжёлый; `HashSet` — мгновенный. Думай про выбор структуры данных.

### Задачи

1. **Примени SOLID** — возьми «God-класс» и разбей на классы с одной ответственностью. *Подсказка: начни с принципа Single Responsibility.*
2. **Слои** — структурируй проект: Domain / Application / Infrastructure / Presentation, интерфейсы на границах. *Подсказка: домен не должен знать про EF Core.*
3. **JWT** — подключи аутентификацию в ASP.NET Core. *Подсказка:* `AddJwtBearer` + `TokenValidationParameters`.
4. **Структурированное логирование** — `ILogger` с параметрами вместо конкатенации. *Подсказка:* `LogInformation("... {OrderId}", order.Id)`.
5. **Метрики** — `Meter` + `Counter`. *Подсказка:* `CreateCounter<int>("orders.created")`.
6. **Профилирование** — найди медленный участок (например, `List.Contains` в цикле) и замени на `HashSet`. *Подсказка: сравни сложность O(n) и O(1).*

### Разбор

- Нет ли утечек секретов в код/логи?
- Зависит ли код от конкретной БД (нарушена абстракция)?

- Понятно ли логирование для «расследования»?
- Тянется ли лишнего в память?

### Чек-поинт

Спроектируй и построй небольшой Web API с слоями, JWT, логированием и тестами. Профилируй и ускорь хотя бы один участок.

---

## После Senior — путь дальше

- **Docker/DevOps** — упаковать приложение как контейнер, автоматизировать деплой.
- **Микросервисы и очереди** — RabbitMQ/Kafka, распределённые системы.
- **Системный дизайн** — масштабируемость, кэширование, отказоустойчивость.
- **Менторство** — объяснять другим: лучший способ углубить собственное понимание.

---

## Календарь (компактный)

| Недели | Уровень | Главное |
|---|---|---|
| 1–2 | 0 | переменные, вывод, ввод |
| 3–6 | 1 | if/циклы/методы/массивы |
| 7–12 | 2 | ООП: классы, наследование, интерфейсы, коллекции |
| 13–20 | 3 | исключения, тесты, LINQ, async, JSON |
| 21–30 | 4 | SOLID, архитектура, безопасность, наблюдаемость, производительность |

---

## Советы «как для ребёнка», но для тебя

1. **Объясняй вслух.** Не понял идею — объясни её утке или воображаемому собеседнику.
2. **Пиши каждый день.** 30 минут кода > 5 часов теории раз в неделю.
3. **Ломай нарочно.** Сломай программу и почини — так ты узнаешь, как она устроена.
4. **Читай ошибки.** Компилятор — не враг, а переводчик: он объясняет, что именно ты сделал не так.
5. **Не зубри, понимай.** Если можешь объяснить — значит, выучил.

---

*Этот план — маршрут. Содержание каждого уровня уже частично наполнено в модулях курса (00–21). Если хочешь, я расширю любой уровень детальными «детскими» уроками и новыми задачами с разбором.*

# Урок 00. Что такое программа и компьютер

## Цель

Понять, что такое программа, чем «язык» отличается от «платформы», и сделать первый запуск кода.

## Теория «как для ребёнка»

### Что такое компьютер?

Компьютер — **очень послушный, но глупый помощник**. Он умеет делать ровно то, что ему сказали, и не умеет «догадываться». Если ты напишешь «сделай вкусный обед» — он не поймёт: слишком расплывчато. А если дашь точный рецепт — выполнит без ошибок, но и без выдумки.

### Что такое программа?

**Программа — это рецепт.** Как рецепт торта:

```
1. Возьми 3 яйца.
2. Добавь 200 г муки.
3. Перемешай.
4. Испеки при 180° 40 минут.
```

Код на C# — такой же рецепт, только для компьютера. Каждая строка — один понятный шаг.

### Чем «язык C#» отличается от «платформы .NET»?

- **C#** — язык, на котором ты пишешь рецепт (слова и правила).
- **.NET** — «кухня»: печь, плита, посуда. Она превращает твой рецепт в готовое блюдо (работающую программу).

Без C# нечего готовить, без .NET негде готовить. Нужны оба.

### Из чего состоит первая программа?

```
Console.WriteLine("Привет, мир!");
```

Разбор по молекулам:

- `Console` — «телефон» для общения с компьютером.

- `WriteLine` — «напиши и перейди на новую строку».
- `("Привет, мир!")` — что именно написать (текст в кавычках).
- `;` — **точка в конце предложения**. Говорит: «шаг закончен».

## Примеры

```
Console.WriteLine("Привет, мир!");
Console.WriteLine("Я учу C#");
```

Что выведется?

```
Привет, мир!
Я учу C#
```

Каждая команда — на своей строке.

## Сделай сам (задачи с подсказками)

1. Выведи надпись «Я стану программистом».
   *Подсказка:* `Console.WriteLine("...");` *— только свой текст.*
2. Выведи две строки: «Первый шаг» и «Второй шаг».
   *Подсказка: две команды* `WriteLine` *одна за другой.*
3. Измени «Привет, мир!» на своё имя.
   *Подсказка: внутри кавычек можно писать что угодно.*
4. Напиши три строчки о себе (любимый цвет, еда, город).
   *Подсказка: три команды* `WriteLine`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `Console.WriteLine("Привет")` без `;` | «предложение не закончено» | добавить `;` |
| `console.WriteLine(...)` (маленькая c) | C# — регистрозависимый | `Console` с большой C |
| Забытые кавычки: `WriteLine(Привет)` | текст обязан быть в кавычках | `WriteLine("Привет")` |
| `WriteLine` вместо `Console.WriteLine` | не сказано, *куда* писать | всегда через `Console.` |

## Тест — проверь себя

**Вопрос 1.** Что делает команда `Console.WriteLine("Привет");`?

- А) Спрашивает у пользователя что-то.

- B) Выводит текст «Привет» на экран и переходит на новую строку.
- C) Сохраняет слово «Привет» в памяти.

✅ **Верно: B.**
Почему: `WriteLine` — «напиши строку». `Console.` указывает, что действие происходит на экране.
Почему неверны остальные:

- A — Это делает `Console.ReadLine()` (считывание), а не `WriteLine` (вывод).
- C — Сохранение в памяти — это переменная (следующий урок), а не вывод на экран.

**Вопрос 2.** Что произойдёт, если убрать `;` в конце команды?

- A) Ничего, программа всё равно заработает.
- B) Компилятор покажет ошибку: он не поймёт, где закончился шаг.
- C) Программа выведет точку с запятой на экран.

✅ **Верно: B.**
Почему: `;` — маркер конца шага, как точка в конце предложения. Без него компилятор «теряется».
Почему неверны остальные:

- A — Ошибка будет обязательно; `;` не необязательный.
- C — `;` — служебный символ, он не выводится на экран.

**Вопрос 3.** C# и .NET — это одно и то же?

- A) Да, слова-синонимы.
- B) Нет: C# — язык, .NET — платформа, которая выполняет код.
- C) C# — это платформа, а .NET — язык.

✅ **Верно: B.**
Почему: язык (правила написания) и платформа (среда выполнения) — разные вещи, работающие вместе.
Почему неверны остальные:

- A — Они связаны, но это разные сущности.
- C — Наоборот: C# — язык, .NET — платформа.

**Вопрос 4.** Команда `Console.WriteLine("Привет, мир!");` — это:

- A) Переменная.
- B) Рецепт полностью.
- C) Один шаг рецепта (одна команда).

✅ **Верно: C.**
Почему: одна команда — один шаг. Рецепт — это много таких шагов.
Почему неверны остальные:

- A — Переменная — «коробочка» для значения (следующий урок), а не команда вывода.
- B — Одна строка — лишь шаг, а не весь «рецепт» программы.

## Итог урока

- Программа = рецепт; каждая строка = один шаг.
- `Console.WriteLine(...)` — показать текст.
- `;` — знак конца шага.
- Учись читать ошибки компилятора — они подсказывают, что исправить.

# Урок 01. Переменные и типы

## Цель

Научиться хранить значения в переменных и выбирать правильный тип.

## Теория «как для ребёнка»

### Что такое переменная?

**Переменная — это коробочка с наклейкой.** Ты пишешь имя на коробочке (например, `age`), кладёшь внутрь значение (`25`), а потом обращаешься к коробочке по имени.

```
int age = 25;      // коробочка "age", внутри число 25
string name = "Иван"; // коробочка "name", внутри текст "Иван"
```

Читается так: **«коробочка age будет хранить целое число 25»**.

### Что такое тип?

**Тип — размер и «содержимое» коробочки.** Нельзя положить текст в коробочку для чисел — она для этого не предназначена, и компилятор скажет об ошибке.

Главные типы для старта:

| Тип | Что хранит | Пример | Аналогия |
|---|---|---|---|
| `int` | целые числа | `25`, `-7`, `0` | коробочка для «штук» |
| `double` | дробные числа | `3.14`, `0.5` | коробочка для «кусочков» |
| `string` | текст | `"Привет"` | коробочка для слов |
| `bool` | да/нет | `true`, `false` | коробочка с выключателем |

### Как вывести содержимое коробочки?

```
int age = 25;
Console.WriteLine(age);   // выведет 25
```

### `var` — «угадай сам»

Можно написать `var age = 25;` — компилятор сам поймёт, что это `int`. Но для начала лучше писать тип явно: так понятнее.

## Примеры

```
 int appleCount = 3;
double price = 49.90;
string greeting = "Добрый день!";
bool isLearning = true;

Console.WriteLine(appleCount);
Console.WriteLine(price);
Console.WriteLine(greeting);
Console.WriteLine(isLearning);
```

## Сделай сам (задачи с подсказками)

1. Создай переменные: возраст (int), имя (string), рост в метрах (double), учится ли (bool). Выведи все.
   *Подсказка: четыре переменные, четыре `Console.WriteLine(...)`.*
2. Создай `int year = 2026;` и выведи его.
   *Подсказка: переменная уже есть — осталось вывести.*
3. Заведи `string city = "Москва";` и выведи «Я живу в Москве».
   *Подсказка: `Console.WriteLine("Я живу в " + city);` — «+» склеивает строки.*
4. Попробуй положить `true` в `int` — что скажет компилятор? Запиши вывод ошибки.
   *Подсказка: это будет ошибка типа — текст не помещается в коробочку для чисел.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `int age = "Иван";` | текст в коробочке для чисел | `string age = "Иван";` |
| `Console.WriteLine(Age);` (большая A) | имя переменной чувствительно к регистру | `Console.WriteLine(age);` |
| Забыл объявить: `Console.WriteLine(age);` без `int age = ...` | коробочки ещё нет | сначала создать переменную |
| `string name = Иван;` без кавычек | текст обязан быть в кавычках | `string name = "Иван";` |

## Тест — проверь себя

**Вопрос 1.** Какой тип у значения `3.14`?

- A) `int`
- B) `double`
- C) `string`

✅ **Верно: B.**
Почему: 3.14 — дробное число → нужна коробочка `double`.
Почему неверны остальные:

- A — `int` хранит только целые (без запятой).
- C — `string` — это текст в кавычках, а не число.

**Вопрос 2.** Что хранит переменная `bool isStudent = true;`?

- A) Число.
- B) Текст.
- C) Логическое значение «да» (true).

✅ **Верно: C.**
Почему: `bool` — только `true` или `false`, словно выключатель.
Почему неверны остальные:

- A — Числа — это `int` / `double`.
- B — Текст — это `string`.

**Вопрос 3.** Почему `int age = "25";` — ошибка?

- A) Потому что 25 слишком большое число.
- B) Потому что `"25"` — это текст, а `int` ожидает число.
- C) Потому что имя `age` запрещено.

✅ **Верно: B.**
Почему: кавычки делают `"25"` текстом; положить текст в коробочку для чисел нельзя.
Почему неверны остальные:

- A — 25 вполне помещается в `int`.
- C — `age` — допустимое имя.

**Вопрос 4.** Что выведет код?

```
 int x = 5;
Console.WriteLine(x);
```

- A) `x`
- B) `5`
- C) Ошибку

✅ **Верно: B.**
Почему: `Console.WriteLine(x)` выводит *значение* из коробочки (5), а не её имя.
Почему неверны остальные:

- A — Имя переменной на экран не выводится.
- C — Ошибки нет: переменная объявлена и заполнена.

## Итог урока

- Переменная = коробочка с наклейкой-именем.
- Тип определяет, что можно положить внутрь.
- `Console.WriteLine(имя)` выводит значение.
- Компилятор строг: не клади текст в коробочку для чисел.

# Урок 02. Ввод и вывод

## Цель

Научиться «разговаривать» с программой: выводить текст и считывать то, что вводит пользователь.

## Теория «как для ребёнка»

### Консоль — телеграм-переписка с компьютером

Программа и пользователь общаются через консоль (тёмное окно):

- Ты **шлёшь сообщение** компьютеру → `Console.WriteLine("...")`.
- Компьютер **спрашивает тебя** → `Console.ReadLine()`.
- Он **ждёт, пока ты напишешь и нажмёшь Enter**.

### Ввод: `Console.ReadLine()`

```
string name = Console.ReadLine();
```

Команда ждёт, пока пользователь введёт текст и нажмёт Enter, и кладёт введённое в коробочку `name`.

> `Console.ReadLine()` всегда возвращает строку ( `string` ). Даже если человек ввел число, с точки зрения программы это пока текст.

### Склейка строк и интерполяция

Есть два способа собрать фразу из переменных:

1. Склейка через `+`:

```
Console.WriteLine("Привет, " + name + "!");
```

2. Интерполяция через `$"..."` (удобнее):

```
Console.WriteLine($"Привет, {name}!");
```

`{name}` — место, куда подставится значение переменной `name`.

### Что если пользователь ввёл число?

`Console.ReadLine()` вернёт текст. Если нужно число — превращаем текст в число:

```
int age = int.Parse(Console.ReadLine());
```

Но если человек введёт «пять», программа упадёт. Безопаснее `int.TryParse` — об этом в уровне 1. Пока просто знай про `int.Parse`.

## Примеры

```
Console.Write("Как тебя зовут? ");   // Write — без перехода на новую строку
string name = Console.ReadLine();
Console.WriteLine($"Рад познакомиться, {name}!");
```

```
Как тебя зовут? Анна
Рад познакомиться, Анна!
```

Чем `Write` отличается от `WriteLine`? `Write` не переносит курсор на новую строку. Поэтому вопрос и ответ оказываются на одной строке.

## Сделай сам (задачи с подсказками)

1. Спроси любимый цвет и выведи «Твой любимый цвет: ...».
   *Подсказка:* *`string color = Console.ReadLine();`* *затем* *`WriteLine($"... {color}")`*.
2. Спроси имя и возраст; выведи «Иван, тебе 25 лет».
   *Подсказка: возраст придётся превратить в число:* *`int.Parse(Console.ReadLine())`*.
3. Выведи вопрос и ответ на одной строке (используй `Write`).
   *Подсказка:* *`Console.Write("Сколько тебе лет? ");`* *без* *`Line`*.
4. Спроси город и страну, выведи: «Я живу в городе ... в стране ...».
   *Подсказка: две переменные и одна интерполяция с двумя* *`{}`*.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `Console.WriteLine()` без текста внутри и без скобок | скобки обязательны | `Console.WriteLine("...")` |
| `Console.ReadLine` без скобок | это метод — нужны `()` | `Console.ReadLine()` |

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `Console.WriteLine($"{name}")` без `$` | подстановка не сработает | `$"..."` с `$` перед строкой |
| `int.Parse(Console.ReadLine())` ввёл «пять» | упадёт на тексте | пока оставь `int.Parse`, но помни про риск |

## Тест — проверь себя

**Вопрос 1.** Что делает `string name = Console.ReadLine();`?

- A) Выводит имя на экран.
- B) Ждёт, пока пользователь введёт текст, и кладёт его в `name`.
- C) Создаёт новую строку с текстом «name».

✅ **Верно: B.**
Почему: `ReadLine` считывает ввод и возвращает его; мы кладём результат в переменную.
Почему неверны остальные:

- A — Вывод делает `WriteLine`, а не `ReadLine`.
- C — В переменную попадает то, что ввёл пользователь, а не слово «name».

**Вопрос 2.** Что выведет код?

```
 string city = "Казань";
Console.WriteLine($"Город: {city}");
```

- A) `Город: city`
- B) `Город: Казань`
- C) `Город: {city}`

✅ **Верно: B.**
Почему: `$` + `{city}` подставляют *значение* переменной (Казань).
Почему неверны остальные:

- A — Без `$` такая подстановка не работает, но здесь `$` стоит — значит, значение.
- C — `{city}` — это шаблон, который заменяется значением.

**Вопрос 3.** Чем `Console.Write` отличается от `Console.WriteLine`?

- A) Ничем, они одинаковы.
- B) `Write` не переходит на новую строку после вывода, а `WriteLine` — переходит.
- C) `Write` выводит только числа.

✅ **Верно: B.**
Почему: разница только в переносе строки после вывода.
Почему неверны остальные:

- A — Разница есть (перенос строки).
- C — `Write` выводит что угодно, не только числа.

**Вопрос 4.** `Console.ReadLine()` всегда возвращает:

- A) `int` — число.
- B) `double` — дробное число.
- C) `string` — строку.

✅ **Верно: C.**
Почему: ввод с консоли по умолчанию — это текст (строка).
Почему неверны остальные:

- A, B — Чтобы получить число, надо превратить строку ( `int.Parse` ), но сам `ReadLine` возвращает строку.

## Итог урока

- `Console.WriteLine` / `Console.Write` — показать текст.
- `Console.ReadLine()` — считать ввод пользователя.
- Интерполяция `$"..."` + `{переменная}` — самый удобный способ собрать фразу.
- Ввод — всегда строка; числа приходится «конвертировать».

## Ссылки на тесты

- 00 · Что такое программа
- 01 · Переменные и типы
- 03 · Арифметика
- 04 · Первая программа

# Урок 03. Арифметика и операторы

## Цель

Научиться выполнять вычисления и понимать «подводные камни» целых чисел.

## Теория «как для ребёнка»

### Компьютер умеет считать

Как и калькулятор, C# знает знаки:

| Знак | Что делает | Пример |
|---|---|---|
| `+` | сложение | `5 + 3` → `8` |
| `-` | вычитание | `5 - 3` → `2` |
| `*` | умножение | `5 * 3` → `15` |
| `/` | деление | `6 / 3` → `2` |
| `%` | остаток от деления | `7 % 3` → `1` |

### Главная ловушка: деление целых чисел

Если **оба числа целые** ( `int` ), то деление даёт **целое** число, отбрасывая дробную часть:

```
Console.WriteLine(7 / 2);   // 3, а НЕ 3.5!
Console.WriteLine(7.0 / 2); // 3.5 — если хоть одно число дробное
```

Почему? Компьютер спрашивает: «7 целых кусочков делим на 2 — сколько целых кусочков у каждого? 3, и один остаётся». Остаток можно узнать через `%`:

```
Console.WriteLine(7 % 2);   // 1 (остаток)
```

### Остаток `%` — волшебная палочка

`%` выдаёт остаток от деления. Это удобно, чтобы проверять «делится ли нацело»:

- `10 % 2 == 0` — правда (чётное).
- `10 % 3` → `1`.

### Порядок действий

Как в математике: сначала `( )`, потом `*` и `/`, потом `+` и `-`.

```
int result = (2 + 3) * 4;   // 20
```

## Примеры

```
 int a = 10;
int b = 3;

Console.WriteLine(a + b);   // 13
Console.WriteLine(a - b);   // 7
Console.WriteLine(a * b);   // 30
Console.WriteLine(a / b);   // 3 (целые!)
Console.WriteLine(a % b);   // 1 (остаток)
```

## Сделай сам (задачи с подсказками)

1. Вычисли площадь прямоугольника: ширина 5, высота 8. Выведи.
   *Подсказка:* `int s = 5 * 8;`
2. Вычисли среднее трёх чисел: 10, 20, 30.
   *Подсказка: сумма* `(10 + 20 + 30)` *и* `/ 3`*; но среднее бывает дробным — используй* `double`.
3. Проверь, чётное ли число 18 (выведи остаток от деления на 2).
   *Подсказка:* `18 % 2` *→ 0 — значит чётное.*
4. Калькулятор: сложи два числа, введённых пользователем.
   *Подсказка:* `int.Parse(Console.ReadLine())` *для каждого числа.*
5. Сколько полных недель в 100 днях и сколько дней останется?
   *Подсказка: недели =* `100 / 7`*, остаток =* `100 % 7`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `7 / 2` ожидаешь `3.5` | целые делятся нацело | сделай хоть одно число дробным: `7.0 / 2` |
| `5 * 3` без скобок и переменной | результат потерян | сохрани в переменную: `int p = 5 * 3;` |
| `10 = x` | слева можно только имя переменной | `x = 10;` |
| Делишь на ноль: `x / 0` | ошибка выполнения | проверяй делитель перед делением |

## Тест — проверь себя

**Вопрос 1.** Что выведет `Console.WriteLine(9 / 2);`?

- A) `4.5`
- B) `4`
- C) `1`

✅ **Верно: B.**
Почему: оба числа целые → деление даёт целое, дробная часть отбрасывается (9 / 2 = 4, остаток 1).
Почему неверны остальные:

- A — `4.5` получилось бы только при дробном операнде (например, `9.0 / 2`).
- C — `1` — это остаток, его даёт `%`, а не `/`.

**Вопрос 2.** Что вернёт `17 % 5`?

- A) `3`
- B) `2`
- C) `3.4`

✅ **Верно: B.**
Почему: 17 ÷ 5 = 3, остаток 2 (5×3=15, 17−15=2).
Почему неверны остальные:

- A — 3 — это целая часть от деления, а не остаток.
- C — 3.4 — обычное деление, а `%` возвращает только остаток.

**Вопрос 3.** Чем `10 % 2` полезно?

- A) Оно показывает 10-ю цифру числа.
- B) Оно показывает, чётное ли число (0 — да, 1 — нет).
- C) Оно умножает на 2.

✅ **Верно: B.**
Почему: остаток от деления на 2 равен 0 у чётных и 1 у нечётных.
Почему неверны остальные:

- A — `%` не связан с цифрами.
- C — Умножение — это `*`, а не `%`.

**Вопрос 4.** В каком порядке посчитает `2 + 3 * 4`?

- A) Слева направо: (2+3)*4 = 20.

- B) Сначала умножение: 2 + (3*4) = 14.
- C) Ошибка.

✅ **Верно: B.**
Почему: `*` имеет приоритет над `+`, как в математике.
Почему неверны остальные:

- A — 20 получится только со скобками: `(2 + 3) * 4`.
- C — Ошибки нет, порядок чётко определён.

## Итог урока

- Знаки: `+ - * / %`.
- Деление целых даёт целое (отбрасывает дробь).
- `%` — остаток, нужен для проверок «делится ли нацело».
- Скобки управляют порядком действий.

# Урок 04. Первая программа целиком

## Цель

Собрать всё изученное в одну работающую программу и запустить её.

## Теория «как для ребёнка»

### Программа — это рецепт из многих шагов

В прошлых уроках мы учили отдельные «кубики»:

- `Console.WriteLine(...)` — показать текст;
- `Console.ReadLine()` — спросить пользователя;
- переменные — «коробочки»;
- `int.Parse(...)` — превратить текст в число.

Теперь соберём их вместе. Один рецепт = несколько шагов подряд.

### Собираем анкету

```
 // Шаг 1: спросить имя
Console.Write("Как тебя зовут? ");
string name = Console.ReadLine();

// Шаг 2: спросить возраст
Console.Write("Сколько тебе лет? ");
int age = int.Parse(Console.ReadLine());

// Шаг 3: вывести результат
Console.WriteLine($"Привет, {name}! Тебе {age} лет.");
```

Разбор:

- `Console.Write` — вопрос без переноса строки (ответ окажется рядом).
- `Console.ReadLine()` — ждёт ввод.
- `int.Parse(...)` — превращает введённый текст в число.
- `$"..."` — интерполяция: подставляет `name` и `age`.

### Как запустить

1. Создай проект: `dotnet new console -n MyFirstApp`
2. Вставь код в `Program.cs`.
3. Запусти: `dotnet run`

## Примеры

**Калькулятор «в одно действие»:**

```
Console.Write("Первое число: ");
int a = int.Parse(Console.ReadLine());

Console.Write("Второе число: ");
int b = int.Parse(Console.ReadLine());

Console.WriteLine($"Сумма: {a + b}");
```

**Проверка чётности:**

```
Console.Write("Введи число: ");
int n = int.Parse(Console.ReadLine());

int remainder = n % 2;
Console.WriteLine($"Остаток от деления на 2: {remainder}");
```

## Сделай сам (задачи с подсказками)

1. **Анкета** — спроси имя, город, возраст; выведи: «Иван из Москвы, 25 лет».
   *Подсказка: три переменные, одна интерполяция с `{ }` для каждой.*
2. **Площадь прямоугольника** — спроси ширину и высоту, выведи площадь.
   *Подсказка: `double s = a * b;` — используй `double`, чтобы не терять дробь.*
3. **Конвертер минут** — спроси число минут; выведи, сколько это часов и остаток минут.
   *Подсказка: часы = `m / 60`, минуты = `m % 60`.*
4. **Зарплата** — спроси ставку за час и число часов; выведи итог.
   *Подсказка: `double total = rate * hours;`*
5. **Температура** — спроси градусы Цельсия, переведи в Фаренгейты (F = C × 9/5 + 32).
   *Подсказка: `double f = c * 9 / 5 + 32;` — но осторожно с целыми: используй `double`.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `int age = Console.ReadLine();` | `ReadLine` даёт строку, а `int` ждёт число | `int age = int.Parse(Console.ReadLine());` |

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Забыл `Console.Write` и сразу `ReadLine` | пользователь не поймёт, что вводить | всегда сначала выведи вопрос-подсказку |
| `9 / 5` в формуле температур | целые дадут 1, а не 1.8 | используй `double` (`c * 9.0 / 5`) |
| Все шаги в одной строке | нечитаемо | по одному шагу на строку, добавляй комментарии `//` |

## Тест — проверь себя

**Вопрос 1.** Что выведет программа?

```
 string name = "Том";
int age = 30;
Console.WriteLine($"{name} — {age} лет");
```

- A) `Том — 30 лет`
- B) `name — age лет`
- C) `Том — возраст лет`

✅ **Верно: A.**
Почему: интерполяция `$` подставляет значения переменных.
Почему неверны остальные:

- B — Имена переменных не выводятся; подставляются их значения.
- C — `age` — это число 30, а не слово «возраст».

**Вопрос 2.** Зачем `int.Parse(Console.ReadLine())`?

- A) Чтобы вывести число на экран.
- B) Чтобы превратить введённый текст в число.
- C) Чтобы остановить программу.

✅ **Верно: B.**
Почему: `ReadLine` возвращает строку; `int.Parse` превращает её в число для вычислений.
Почему неверны остальные:

- A — Вывод делает `Console.WriteLine`.
- C — Остановка программы — это не задача `Parse`.

**Вопрос 3.** Что будет, если при `int.Parse(Console.ReadLine())` пользователь введёт «пять»?

- A) Программа превратит «пять» в число 5.
- B) Программа упадёт с ошибкой (не сможет превратить текст в число).

- C) Программа выведет «пять».

✅ **Верно: B.**
Почему: `int.Parse` понимает только цифры (например, "5"). «пять» — текст, он не превратится в число, и выскочит ошибка.
Почему неверны остальные:

- A — C# не понимает русские названия чисел «из коробки».
- C — Вывода не будет: раньше произойдёт ошибка.

**Вопрос 4.** Какой порядок шагов правильный?

- A) Сначала вычислить, потом спросить данные.
- B) Сначала спросить данные, потом обработать и вывести результат.
- C) Порядок не важен.

✅ **Верно: B.**
Почему: нельзя суммировать числа, которых ещё нет; сначала получаем данные, потом считаем.
Почему неверны остальные:

- A — Без данных считать нечего.
- C — Порядок критичен.

## Итог уровня 0

Ты теперь умеешь:

- выводить текст ( `Console.WriteLine` );
- считывать ввод ( `Console.ReadLine` );
- хранить значения в переменных с типами;
- выполнять арифметику и понимать «ловушку» целых чисел;
- собирать из этого работающие программы.

Поздравляю — **Уровень 0 пройден!** Дальше — Уровень 1: научить компьютер принимать решения (if) и повторять действия (циклы).

# Урок 10. Условия if/else

## Цель

Научиться заставлять программу принимать решения: что-то делать только при определённом условии.

## Теория «как для ребёнка»

### Развилка на дороге

Представь, что ты подошёл к развилке. Если идёт дождь — возьми зонт. Если нет — иди без зонта. Это и есть `if` (если) и `else` (иначе).

```
 if (age >= 18)
{
    Console.WriteLine("Ты взрослый");
}
else
{
    Console.WriteLine("Ты ещё несовершеннолетний");
}
```

Читается: «**если** возраст ≥ 18 — вывести одно, **иначе** — другое».

### Как пишется

- После `if` — скобки с условием: `if (условие)`.
- Внутри фигурных скобок `{ }` — что делать, если условие истинно.
- `else` — необязателен: если он не нужен, можно его не писать.

### Что такое «условие»?

Условие — это выражение, которое даёт «да/нет» (тип `bool`):

| Сравнение | Что значит |
|---|---|
| `a == b` | a равно b |
| `a != b` | a не равно b |
| `a > b` | a больше b |
| `a < b` | a меньше b |

| Сравнение | Что значит |
|---|---|
| `a >= b` | a больше или равно b |
| `a <= b` | a меньше или равно b |

> `==` — это проверка равенства. А одинарный `=` — это присваивание (положить значение). Не путай!

### Вложенные условия

Условие может быть внутри другого условия. Но лучше не увлекаться — код становится трудно читать.

## Примеры

```
 Console.Write("Сколько тебе лет? ");
int age = int.Parse(Console.ReadLine());

if (age >= 18)
{
    Console.WriteLine("Доступ разрешён");
}
else
{
    Console.WriteLine("Доступ запрещён");
}
```

## Сделай сам (задачи с подсказками)

1. Спроси число; выведи «положительное», «отрицательное» или «ноль».
   *Подсказка: сначала* `if (n > 0)`, *потом* `else if (n < 0)`, *затем* `else`.
2. Спроси число; выведи, чётное ли оно.
   *Подсказка:* `n % 2 == 0` — *чётное.*
3. Спроси оценку: если ≥ 4 — «молодец», иначе — «нужно подтянуть».
   *Подсказка:* `if (score >= 4)`.
4. Спроси возраст: если < 7 — «садик», < 18 — «школа», иначе — «взрослая жизнь».
   *Подсказка: цепочка* `if / else if / else`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `if (age = 18)` | одинарный `=` присваивает, а не сравнивает | `if (age == 18)` |
| `if age >= 18` без скобок | условие обязательно в `()` | `if (age >= 18)` |
| `if (age >= 18);` | `;` после условия «обрезает» ветку | после `)` сразу `{` |
| Забыл `{ }` | выполнится только первая строка | всегда ставь `{ }` |

## Тест — проверь себя

**Вопрос 1.** Что выведет код?

```
 int x = 5;
if (x > 10) Console.WriteLine("большое");
else Console.WriteLine("маленькое");
```

- A) `большое`
- B) `маленькое`
- C) ничего

✅ **Верно: B.**
Почему: 5 не больше 10, значит условие ложно → выполняется ветка `else`.
Почему неверны остальные:

- A — Условие `x > 10` ложно, так что эта ветка пропускается.
- C — `else` всегда выполнится, поэтому вывод будет обязательно.

**Вопрос 2.** Что означает `a == b`?

- A) Положить b в коробочку a.
- B) Проверить, равны ли a и b.
- C) Сложить a и b.

✅ **Верно: B.**
Почему: двойное `==` — проверка равенства.
Почему неверны остальные:

- A — Присваивание — это одинарное `=`.
- C — Сложение — это `+`.

**Вопрос 3.** Как сделать проверку: «если число отрицательное, вывести предупреждение» (и ничего иначе)?

- A) `if (n < 0) { Console.WriteLine("Отрицательное"); }`

- B) `if (n < 0) Console.WriteLine("Отрицательное"); else Console.WriteLine("OK");`
- C) `Console.WriteLine("Отрицательное"); if (n < 0)`

✅ **Верно: A.**
Почему: условие с блоком `{ }`, `else` не обязателен — если его нет, при ложном условии просто ничего не происходит.
Почему неверны остальные:

- B — Здесь `else` выводит «OK», а по условию задачи этого быть не должно.
- C — Сначала выполняется вывод, проверка — позже и бессмысленно.

**Вопрос 4.** Что выведет код?

```
 int n = 0;
if (n > 0) Console.WriteLine("+");
else if (n < 0) Console.WriteLine("-");
else Console.WriteLine("0");
```

- A) `+`
- B) `-`
- C) `0`

✅ **Верно: C.**
Почему: 0 не больше 0 и не меньше 0 → срабатывает финальный `else`.
Почему неверны остальные:

- A — `n > 0` ложно для нуля.
- B — `n < 0` тоже ложно для нуля.

## Итог урока

- `if (условие) { ... }` — если условие верно, выполнить блок.
- `else { ... }` — иначе (необязателен).
- Сравнения: `==`, `!=`, `>`, `<`, `>=`, `<=`.
- Не путай `==` (сравнить) и `=` (присвоить).

# Урок 11. Логические операторы

## Цель

Научиться строить сложные условия из простых: «и», «или», «не».

## Теория «как для ребёнка»

### Как соединить два условия?

«Взять зонт, если идёт дождь **И** дует сильный ветер». Здесь два условия, соединённые словом «И». В C# «И» — это `&&`.

| Оператор | Что значит | Пример |
|---|---|---|
| `&&` | «и» — оба условия должны быть верны | `age >= 18 && age < 65` |
| `\|\|` | «или» — хотя бы одно верно | `day == "Сб" \|\| day == "Вс"` |
| `!` | «не» — отрицание | `!isAdmin` |

### `&&` — «и»

`( age >= 18 && age < 65 )` верно, только если **оба** условия истинны. Если хоть одно ложно — всё условие ложно.

Пример: «взрослый трудоспособный возраст» = от 18 до 65 (не включая 65).

### `||` — «или»

`( day == "Сб" || day == "Вс" )` верно, если **хотя бы одно** истинно. Дождик, выходной в субботу — правда, в воскресенье — правда, в понедельник — ложь.

### `!` — «не»

`!isAdmin` — «не админ». Инвертирует: было `true` → стало `false`, и наоборот.

## Примеры

```
 int age = 30;
bool isStudent = false;

if (age >= 18 && age < 65)
```

```
{
    Console.WriteLine("Взрослый трудоспособный возраст");
}

if (age < 18 || isStudent)
{
    Console.WriteLine("Есть скидка");
}
```

## Сделай сам (задачи с подсказками)

1. Спроси возраст: выведи «можно водить машину», если возраст от 18 до 100.
   *Подсказка:* `age >= 18 && age <= 100`.
2. Спроси день недели: выведи «выходной», если суббота или воскресенье.
   *Подсказка:* `day == "Сб" || day == "Вс"`.
3. Спроси число: выведи «в диапазоне», если оно от 10 до 20 включительно.
   *Подсказка:* `n >= 10 && n <= 20`.
4. Есть флаг `bool isBanned`. Выведи «доступ разрешён», если НЕ забанен.
   *Подсказка:* `if (!isBanned)`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `18 < age < 65` | C# не умеет двойные сравнения | `age >= 18 && age < 65` |
| `age >= 18 && age < 65` без скобок | при смешении операторов — читаемость | ставь скобки: `(age >= 18) && (age < 65)` |
| `day == "Сб" | | day == "Вс"` `всего один знак` =` |
| `!isBanned = true` | нельзя присваивать рядом с `!` | просто `!isBanned` |

## Тест — проверь себя

**Вопрос 1.** При каких значениях условие `n >= 5 && n <= 10` истинно?

- A) Только при n = 5.
- B) При n от 5 до 10 включительно.
- C) При любом n больше 10.

✅ **Верно: B.**
Почему: `&&` требует, чтобы оба условия были верны: n ≥ 5 и n ≤ 10 одновременно → диапазон 5–10.
Почему неверны остальные:

- A — Верно не только для 5, но и для 6,7,8,9,10.
- C — При n > 10 второе условие `n <= 10` ложно.

**Вопрос 2.** Когда `day == "Сб" || day == "Вс"` ложно?

- A) Когда day — суббота.
- B) Когда day — воскресенье.
- C) Когда day — любой другой день.

✅ **Верно: C.**
Почему: `||` истинно, если хотя бы одно верно. Ложно, только когда оба неверны — это любой день, кроме субботы и воскресенья.
Почему неверны остальные:

- A, B — В эти дни условие истинно.

**Вопрос 3.** Что делает `!isAdmin`?

- A) Проверяет, что isAdmin истинно.
- B) Инвертирует: true становится false и наоборот.
- C) Делает isAdmin всегда истинным.

✅ **Верно: B.**
Почему: `!` — отрицание.
Почему неверны остальные:

- A — `!` делает противоположное, а не повторяет.
- C — Ничего не «всегда истинным» не делается.

**Вопрос 4.** Выбери правильную запись: «age от 18 до 65, не включая 65»:

- A) `age >= 18 && age < 65`
- B) `age >= 18 || age < 65`
- C) `age == 18 && age == 65`

✅ **Верно: A.**
Почему: «и» (оба условия), причём верхняя граница строгая ( `< 65` ).
Почему неверны остальные:

- B — `||` означало бы «или», а это почти всегда верно, это не то.
- C — `==` оба сразу не могут быть истинны (18 ≠ 65).

## Итог урока

- `&&` — «и» (оба верны).
- `||` — «или» (хотя бы одно верно).
- `!` — «не» (отрицание).
- Для диапазонов используй `&&` и явные границы.

# Урок 12. switch — выбор по вариантам

## Цель

Научиться выбирать действие, когда вариантов несколько и каждый зависит от одного значения.

## Теория «как для ребёнка»

### Когда нужен switch?

Если у тебя есть одно значение (например, день недели) и для каждого значения — свой ответ, можно писать длинную цепочку `if`. Но удобнее и читаемее использовать `switch`.

### Классический switch

```
 string day = Console.ReadLine() ?? "";

switch (day)
{
    case "Пн":
        Console.WriteLine("Начало недели");
        break;
    case "Сб":
    case "Вс":
        Console.WriteLine("Выходной");
        break;
    default:
        Console.WriteLine("Не знаю такой день");
        break;
}
```

Как читать:

- `case "Пн"` — «если day равно "Пн"»;
- `break;` — «выйти из switch» (иначе код «провалится» дальше);
- несколько `case` подряд без кода — несколько значений попадают в один блок;
- `default` — «во всех остальных случаях» (как `else`).

### Сокращённый вариант — switch-выражение

```
 string message = day switch
{
```

```
    "Пн" => "Начало недели",
    "Сб" or "Вс" => "Выходной",
    _ => "Не знаю такой день"
};
```

- `=>` — «то-то»;
- `or` — «или»;
- `_` — «все остальные случаи».

## Примеры

```
 int month = 3;
string season = month switch
{
    12 or 1 or 2 => "Зима",
    3 or 4 or 5 => "Весна",
    6 or 7 or 8 => "Лето",
    9 or 10 or 11 => "Осень",
    _ => "Неверный месяц"
};
Console.WriteLine(season);
```

## Сделай сам (задачи с подсказками)

1. Спроси номер дня недели (1–7), выведи его название.
   *Подсказка:* *`switch (dayNumber)`* *с* *`case 1:`* *…* *`case 7:`*.
2. Спроси уровень доступа («user», «manager», «admin»); выведи, что доступно.
   *Подсказка:* *`case "user":`* *— «только чтение» и т.д.*
3. Спроси страну; выведи столицу.
   *Подсказка:* *`case "Россия": Console.WriteLine("Москва"); break;`* *и т.д.*
4. Перепиши задачу 3 через switch-выражение с `=>` и `_`.
   *Подсказка:* *`string capital = country switch { "Россия" => "Москва", _ => "не знаю" };`*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Забыл `break;` | код «провалится» в следующий case | всегда `break;` в конце блока |
| `case "Пн";` вместо `:` | после значения — двоеточие | `case "Пн":` |

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `default` поставлен не последним | порядок нарушен | `default` — в конце |
| Сравнение с `=` | `=` присваивает | в `case` уже сравнение, `=` не нужен |

## Тест — проверь себя

**Вопрос 1.** Что выведет код?

```
 string color = "красный";
switch (color)
{
    case "зелёный": Console.WriteLine("Идти"); break;
    case "красный": Console.WriteLine("Стоять"); break;
    default: Console.WriteLine("Осторожно"); break;
}
```

- A) `Идти`
- B) `Стоять`
- C) `Осторожно`

✅ **Верно: B.**
Почему: color равен «красный», значит срабатывает `case "красный"`.
Почему неверны остальные:

- A — «зелёный» не совпал.
- C — `default` срабатывает только если ни один `case` не совпал.

**Вопрос 2.** Зачем нужен `break;`?

- A) Чтобы завершить программу.
- B) Чтобы выйти из switch после выполнения выбранной ветки.
- C) Чтобы пропустить default.

✅ **Верно: B.**
Почему: `break` останавливает выполнение switch после найденной ветки.
Почему неверны остальные:

- A — Программа продолжает работать дальше после switch.
- C — `break` не про `default`, а про выход из switch.

**Вопрос 3.** Что означает `_` в switch-выражении?

- A) Ничего, это опечатка.
- B) «Все остальные случаи» (по умолчанию).
- C) «И» (как &&).

✅ **Верно: B.**
Почему: `_` — дискард, «любое другое значение».
Почему неверны остальные:

- A — Это законный синтаксис.
- C — «И» — это `and` / `&&`, а не `_`.

**Вопрос 4.** Что выведет код?

```
 int n = 2;
string s = n switch
{
    1 => "один",
    2 => "два",
    _ => "много"
};
Console.WriteLine(s);
```

- A) `один`
- B) `два`
- C) `много`

✅ **Верно: B.**
Почему: n = 2 → подходит ветка `2 => "два"`.
Почему неверны остальные:

- A — Для 1 была бы ветка 1, но у нас 2.
- C — `_` срабатывает только для значений, не совпавших с 1 и 2.

## Итог урока

- `switch` — выбор из нескольких вариантов по одному значению.
- `break;` — обязателен в классическом switch.
- `default` (или `_`) — «все остальные случаи».
- switch-выражение с `=>` и `or` — компактнее.

# Урок 13. Цикл for

## Цель

Научиться повторять действия заранее известное число раз.

## Теория «как для ребёнка»

### Карусель

`for` — это карусель: ты заранее знаешь, сколько кругов надо проехать.

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"Круг номер {i}");
}
```

Разбор трёх частей в скобках:

| Часть | Что делает | Пример |
|---|---|---|
| `int i = 1;` | старт (счётчик начинает с 1) | счётчик |
| `i <= 5;` | условие продолжения | крутимся, пока i ≤ 5 |
| `i++` | шаг после каждого круга | i увеличивается на 1 |

Работает так:

1. Делаем старт: `i = 1`.
2. Проверяем условие: `1 <= 5` → да → выполняем тело.
3. Выполняем `i++` → теперь `i = 2`.
4. Снова проверяем условие — и так, пока оно истинно.

### Шаблон «ровно N раз»

Самый частый случай — повторить ровно N раз:

```
for (int i = 0; i < 10; i++)
{
    // выполнится 10 раз: i = 0,1,...,9
}
```

Обрати внимание: счётчик идёт **от 0 до N-1**. Так принято в программировании.

## Примеры

**Сумма чисел от 1 до 100:**

```
 int sum = 0;
for (int i = 1; i <= 100; i++)
{
    sum += i;
}
Console.WriteLine(sum);
```

**Чётные числа до 20:**

```
 for (int i = 0; i <= 20; i += 2)
{
    Console.WriteLine(i);
}
```

## Сделай сам (задачи с подсказками)

1. Выведи числа от 1 до 10.
   *Подсказка:* `for (int i = 1; i <= 10; i++)`.
2. Выведи числа от 10 до 1 (убывание).
   *Подсказка:* `for (int i = 10; i >= 1; i--)` — *шаг вниз.*
3. Вычисли сумму чисел от 1 до N (N спрашиваешь).
   *Подсказка:* `sum += i;` *внутри цикла.*
4. Выведи квадраты чисел от 1 до 10.
   *Подсказка:* `Console.WriteLine(i * i);`.
5. Выведи «звёздочки»: 5 штук в одну строку.
   *Подсказка: внутри цикла используй* `Console.Write("*")` *(без Line), после цикла —* `Console.WriteLine()`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `for (int i = 1; i <= 10; i++)` но с `;` после скобок | пустое тело, цикл просто «крутится» | не ставь `;` сразу после `)` |
| `i < 10` и ждёшь 10 повторов | будет ровно 10 (0..9) | если нужно 10 — `i < 10` |
| Забыл `i++` | бесконечный цикл | всегда пиши шаг |

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `i` используешь после цикла | переменная живёт только внутри цикла | объяви счётчик вне цикла, если нужен потом |

## Тест — проверь себя

**Вопрос 1.** Сколько раз выполнится тело?

```
for (int i = 0; i < 5; i++) { }
```

- A) 4
- B) 5
- C) 6

✅ **Верно: B.**
Почему: i идёт 0,1,2,3,4 — ровно 5 значений меньше 5.
Почему неверны остальные:

- A — 4 было бы при `i < 4` или `i <= 3`.
- C — 6 было бы при `i < 6`.

**Вопрос 2.** Что выведет код?

```
for (int i = 1; i <= 3; i++) Console.WriteLine(i);
```

- A) `1 2 3`
- B) `0 1 2`
- C) `1 2`

✅ **Верно: A.**
Почему: начинает с 1, идёт до 3 включительно ( `<=` ).
Почему неверны остальные:

- B — Это было бы при старте с 0.
- C — 2 — это при `i < 3` (без включения 3), а у нас `<=`.

**Вопрос 3.** Что делает `i += 2` в шаге цикла?

- A) Увеличивает i на 2 каждый раз.
- B) Увеличивает i на 1.
- C) Уменьшает i на 2.

✅ **Верно: A.**
Почему: `+= 2` — «прибавь 2».
Почему неверны остальные:

- B — Прибавка на 1 — это `i++` или `i += 1`.
- C — Уменьшение — это `i -= 2`.

**Вопрос 4.** Чем опасен цикл без изменения счётчика (`i++`)?

- A) Ничем.
- B) Он станет бесконечным (условие никогда не станет ложным).
- C) Он выполнится один раз.

✅ **Верно: B.**
Почему: без шага i не меняется, условие остаётся истинным навсегда.
Почему неверны остальные:

- A — Это серьёзная ошибка.
- C — Один раз не выполнится — он зациклится.

## Итог урока

- `for` — повторение известное число раз.
- Три части: старт, условие, шаг.
- Счётчик от 0 до N-1 — стандарт.
- Не забывай шаг (`i++`), иначе цикл бесконечный.

# Урок 14. Циклы while и do-while

## Цель

Научиться повторять действия, пока условие истинно (даже если заранее неизвестно, сколько раз).

## Теория «как для ребёнка»

### Пока музыка играет — танцуй

`while` — «пока условие верно, делай». Сколько раз — заранее неизвестно.

```
 int x = 0;
while (x < 5)
{
    Console.WriteLine(x);
    x++;
}
```

Работает так:

1. Проверяем условие.
2. Если истинно — выполняем тело.
3. Возвращаемся к проверке.

### Особый случай: while (true)

Бесконечный цикл — «крутись всегда». Хорош, когда выход приходит изнутри через `break`:

```
 while (true)
{
    Console.Write("Команда (exit — выход): ");
    string cmd = Console.ReadLine() ?? "";

    if (cmd == "exit") break;   // выход из цикла
    Console.WriteLine($"Выполняю: {cmd}");
}
```

### do-while — «сначала сделай, потом проверь»

Разница: тело выполняется как минимум один раз, проверка — после.

```
 string answer;
do
{
    Console.Write("Введи 'да': ");
    answer = Console.ReadLine() ?? "";
}
while (answer != "да");
```

### break и continue

- `break` — полностью выйти из цикла.
- `continue` — перейти сразу к следующей итерации (пропустить остаток тела).

## Примеры

**Ввод числа, пока не введено корректное:**

```
 int age;
do
{
    Console.Write("Введи возраст: ");
}
while (!int.TryParse(Console.ReadLine(), out age));

Console.WriteLine($"Возраст: {age}");
```

## Сделай сам (задачи с подсказками)

1. Выведи числа от 1 до 10 через `while`.
   *Подсказка: счётчик `int i = 1;` и условие `while (i <= 10)`.*
2. Запрашивай число, пока пользователь не введёт 0; в конце выведи сумму введённых.
   *Подсказка: `while (true)` + `break` при 0.*
3. «Угадай число»: компьютер загадал, ты угадываешь, подсказки «больше/меньше».
   *Подсказка: `while (guess != secret)` внутри считывай и подсказывай.*
4. Выведи чётные числа до 20 через `while` с `continue`.
   *Подсказка: если число нечётное — `continue`.*
5. Перепиши задачу 1 через `do-while` и сравни поведение.
   *Подсказка: в `do-while` тело гарантированно выполнится хотя бы один раз.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `while (condition)` без изменения переменных | бесконечный цикл | обязательно меняй что-то в теле |
| `while (true)` без `break` | никогда не закончится | планируй выход изнутри |
| `while` с `;` сразу после скобок | пустое тело | не ставь `;` |
| Путаешь `while` и `do-while` | разные моменты проверки | `do-while` — проверка после, всегда хотя бы один раз |

## Тест — проверь себя

**Вопрос 1.** Сколько раз выполнится тело?

```
int i = 0;
while (i < 3) { Console.WriteLine(i); i++; }
```

- A) 2
- B) 3
- C) 4

✅ **Верно: B.**
Почему: i = 0,1,2 — три итерации (при i=3 условие ложно).
Почему неверны остальные:

- A — 2 было бы при `i < 2`.
- C — 4 было бы при `i <= 3` вместо `i < 3`.

**Вопрос 2.** Чем `do-while` отличается от `while`?

- A) Ничем.
- B) `do-while` проверяет условие после тела, поэтому выполняется хотя бы один раз.
- C) `do-while` нельзя использовать с `break`.

✅ **Верно: B.**
Почему: в `do-while` сначала тело, потом проверка → минимум один проход.
Почему неверны остальные:

- A — Разница есть.
- C — `break` работает в обоих.

**Вопрос 3.** Что произойдёт?

```
while (true) { Console.WriteLine("x"); }
```

- A) Выведется один раз.
- B) Будет печатать "x" бесконечно.
- C) Ошибка компиляции.

✅ **Верно: B.**
Почему: условие всегда истинно, выхода нет.
Почему неверны остальные:

- A — Не остановится на одном.
- C — Такой код компилируется, но зацикливается.

**Вопрос 4.** Что делает `break;` внутри цикла?

- A) Пропускает одну итерацию.
- B) Полностью завершает цикл.
- C) Начинает цикл заново.

✅ **Верно: B.**
Почему: `break` прерывает цикл целиком.
Почему неверны остальные:

- A — Пропуск одной итерации — это `continue`.
- C — Перезапуск цикла — это просто повторное вхождение, а не `break`.

## Итог урока

- `while` — проверка перед выполнением.
- `do-while` — выполнение хотя бы один раз.
- `while (true)` + `break` — вечный цикл с управляемым выходом.
- `break` — выход, `continue` — пропуск итерации.

# Урок 15. Методы

## Цель

Научиться упаковывать кусочки кода в «инструкции с именем» и переиспользовать их.

## Теория «как для ребёнка»

### Инструкция с именем

Представь, что ты записал на бумажке рецепт «как пожарить яйцо» и повесил на холодильник с надписью **ПожаритьЯйцо**. Теперь ты можешь говорить «ПожаритьЯйцо!» вместо того, чтобы повторять все шаги заново. Это и есть метод.

```
void SayHello()
{
    Console.WriteLine("Привет!");
}
```

Вызов:

```
SayHello();   // "Привет!"
SayHello();   // ещё раз
```

### Метод с параметрами (входными данными)

Метод умеет принимать данные:

```
void Greet(string name)
{
    Console.WriteLine($"Привет, {name}!");
}
```

Вызов:

```
Greet("Маша");   // "Привет, Маша!"
Greet("Петя");   // "Привет, Петя!"
```

### Метод, возвращающий результат

Некоторые методы не просто печатают, а возвращают значение (как «выдающий сдачу» автомат) — тип возврата пишется вместо `void`:

```
double Square(double x)
{
    return x * x;
}
```

Вызов и использование результата:

```
double result = Square(5);   // result = 25
Console.WriteLine(result);
```

- `void` — метод ничего не возвращает.
- `double` / `int` / `string` и т.д. — возвращает значение этого типа.
- `return` — вернуть значение и закончить метод.

### Зачем это нужно

1. **Не повторяться** — написал один раз, зовёшь много раз.
2. **Смысл** — имя метода объясняет, что он делает.
3. **Чистый код** — большую задачу дробишь на маленькие методы.

## Примеры

```
int Add(int a, int b) => a + b;   // сокращённая запись (expression-bodied)

// использование:
Console.WriteLine(Add(2, 3));      // 5
Console.WriteLine(Add(10, 5));     // 15
```

## Сделай сам (задачи с подсказками)

1. Напиши метод `PrintHeader()` — выводит «====» и текст. Вызови его 3 раза.
   *Подсказка:* `void PrintHeader() { ... }` *— без параметров.*
2. Напиши метод `Square(double x)`, возвращающий $x^2$. Выведи `Square(5)`.
   *Подсказка:* `double Square(double x) => x * x;`
3. Напиши метод `Max(int a, int b)` — возвращает большее из двух.
   *Подсказка:* `return a > b ? a : b;` *(тернарный оператор).*
4. Напиши метод `IsEven(int n)` — возвращает `bool`.
   *Подсказка:* `return n % 2 == 0;`

5. Метод `Describe(int age)` выводит «ребёнок», «подросток» или «взрослый».
   *Подсказка: внутри метода используй `if/else`.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Метод возвращает значение, но нет `return` | компилятор не увидит результата | добавь `return ...;` |
| `void`-метод с `return 5;` | нечего возвращать | убери значение или смени тип на `int` |
| Вызов `Square` без `()` | это вызов метода — нужны скобки | `Square(5)` |
| Имя метода с маленькой буквы | методы — в PascalCase | `Square`, `Greet` |

## Тест — проверь себя

**Вопрос 1.** Что выведет код?

```
void Print(int n) { Console.WriteLine(n); }
Print(7);
```

- A) `n`
- B) `7`
- C) Ошибку

✅ **Верно: B.**
Почему: Print(7) передаёт 7 в параметр n и выводит его значение.
Почему неверны остальные:

- A — Выводится значение, а не имя параметра.
- C — Всё корректно.

**Вопрос 2.** Что означает тип `void`?

- A) Метод возвращает число.
- B) Метод ничего не возвращает.
- C) Метод возвращает строку.

✅ **Верно: B.**
Почему: `void` — «пусто», метод выполняет действия, но ничего не возвращает.
Почему неверны остальные:

- A, C — Это были бы `int`, `double`, `string` и т.п.

**Вопрос 3.** Что вернёт `Square(4)`?

```
double Square(double x) => x * x;
```

- A) 8
- B) 16
- C) 4

✅ **Верно: B.**
Почему: 4 × 4 = 16.
Почему неверны остальные:

- A — 8 — это 2×4 (умножение на 2), а не квадрат.
- C — 4 — это входное значение, не результат.

**Вопрос 4.** В чём польза методов?

- A) Чтобы код был длиннее.
- B) Чтобы не повторять один и тот же код и давать блокам понятные имена.
- C) Чтобы программа стала красивее, но не работала.

✅ **Верно: B.**
Почему: методы убирают дублирование и добавляют смысл.
Почему неверны остальные:

- A — Длиннее — не цель, важно качество.
- C — Методы — это про работоспособность и читаемость, а не украшение.

## Итог урока

- Метод — поименованная инструкция, которую можно звать много раз.
- Параметры — входные данные; `return` — результат.
- `void` — ничего не возвращает.
- Пиши методы в PascalCase, смыслово.

# Урок 16. Массивы

## Цель

Научиться хранить много значений в одной «коробке с клетками» и перебирать их.

## Теория «как для ребёнка»

### Коробка с клетками

Массив — это коробка, разделённая на пронумерованные клетки. Номер клетки называется **индексом**, и нумерация начинается с **0**.

```
int[] numbers = { 10, 20, 30 };
```

Теперь:

- `numbers[0]` — первая клетка (значение 10);
- `numbers[1]` — вторая клетка (значение 20);
- `numbers[2]` — третья клетка (значение 30).

### Как создать массив

```
// с готовыми значениями:
int[] a = { 1, 2, 3 };

// пустой с заданным размером:
string[] names = new string[5];
```

### Длина массива

```
Console.WriteLine(numbers.Length);   // 3
```

### Перебор через цикл

```
int[] numbers = { 10, 20, 30 };

for (int i = 0; i < numbers.Length; i++)
{
```

```
    Console.WriteLine(numbers[i]);
}
```

Или `foreach` — «для каждого элемента» (не нужно думать об индексах):

```
foreach (int n in numbers)
{
    Console.WriteLine(n);
}
```

### Изменить элемент

```
numbers[1] = 99;   // теперь { 10, 99, 30 }
```

## Примеры

**Сумма элементов:**

```
int[] numbers = { 4, 7, 2 };
int sum = 0;
foreach (int n in numbers) sum += n;
Console.WriteLine(sum);   // 13
```

## Сделай сам (задачи с подсказками)

1. Создай массив из 5 чисел и выведи его элементы через `for`.
   *Подсказка:* `for (int i = 0; i < arr.Length; i++)`.
2. Выведи тот же массив через `foreach`.
   *Подсказка:* `foreach (int x in arr)`.
3. Найди максимальный элемент массива.
   *Подсказка: заведи* `int max = arr[0];` *и сравнивай с остальными.*
4. Посчитай сумму элементов без `for` — через `foreach`.
   *Подсказка:* `sum += x;` *внутри цикла.*
5. Выведи элементы массива в обратном порядке.
   *Подсказка: цикл от* `arr.Length - 1` *вниз до 0.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `numbers[3]` при длине 3 | индексы идут 0,1,2 — 3 уже за границей | помни: последний индекс = длина − 1 |

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `{ 1, 2, 3 }` без типа | C# не поймёт тип | укажи тип: `int[] numbers = { ... };` |
| Цикл `i <= numbers.Length` | выход за границу (IndexOutOfRangeException) | используй `i < numbers.Length` |
| `int[] numbers = new int[3];` и сразу `numbers[0]` | значение по умолчанию 0 | 0 — значение по умолчанию для int, это нормально |

## Тест — проверь себя

**Вопрос 1.** Какое значение у `numbers[1]`?

```
int[] numbers = { 10, 20, 30 };
```

- A) 10
- B) 20
- C) 30

✅ **Верно: B.**
Почему: индексы с 0: numbers[0]=10, numbers[1]=20, numbers[2]=30.
Почему неверны остальные:

- A — 10 — это numbers[0].
- C — 30 — это numbers[2].

**Вопрос 2.** Чему равна длина `{ 1, 2, 3, 4 }`?

- A) 3
- B) 4
- C) 5

✅ **Верно: B.**
Почему: четыре элемента → Length = 4.
Почему неверны остальные:

- A — 3 — это был бы массив из трёх элементов.
- C — 5 — это был бы массив из пяти элементов.

**Вопрос 3.** Какой индекс последнего элемента массива длиной 5?

- A) 5
- B) 4
- C) 6

✅ **Верно: B.**
Почему: индексы с 0, последний = длина − 1 = 4.
Почему неверны остальные:

- A — 5 — это длина, а не индекс последнего.
- C — 6 — вообще за границей.

**Вопрос 4.** Что выведет код?

```
int[] a = { 2, 4, 6 };
int s = 0;
foreach (int x in a) s += x;
Console.WriteLine(s);
```

- A) 12
- B) 6
- C) 2

✅ **Верно: A.**
Почему: 2 + 4 + 6 = 12.
Почему неверны остальные:

- B — 6 — это просто последний элемент.
- C — 2 — это первый элемент.

## Итог урока

- Массив — упорядоченная «коробка с клетками»; индексы с 0.
- `Length` — число элементов.
- `foreach` — удобный перебор всех элементов.
- Последний индекс всегда `Length - 1`; не выходи за границы.

## Итог уровня 1

Ты научился:

- принимать решения (`if/else`, `switch`);
- составлять сложные условия (`&&`, `||`, `!`);
- повторять действия (`for`, `while`, `do-while`);
- писать методы и возвращать значения;
- хранить много значений в массивах.

Поздравляю — **Уровень 1 пройден!** Дальше — Уровень 2: классы и объектно-ориентированное программирование.

# Урок 17. Отладка и чтение ошибок

## Цель

Научиться понимать сообщения об ошибках и находить, что сломалось.

## Словарик новых слов

- **Компилятор** — переводчик с C# на исполняемый код. Если он не понял — показывает ошибку.
- **Ошибка компиляции** — «статическая» ошибка: код неверен, программа не соберётся.
- **Ошибка во время выполнения (runtime)** — код собрался, но упал в процессе.
- **Трассировка стека (stack trace)** — список вызовов, который показывает, где случилась ошибка.
- **Отладка** — процесс поиска и исправления ошибок.

## Теория «с нуля»

### Аналогия: переводчик и шеф-повар

Компилятор — капризный переводчик: если ты сделал опечатку — он не «догадывается», а честно указывает строку. А ошибка во время выполнения — как рецепт, который собрался, но при готовке что-то пошло не так. Главное — **читать сообщения**, а не пугаться.

### Виды ошибок

#### 1. Ошибка компиляции — программа не собирается

```
Console.WriteLine("Привет")   // забыли ;
```

Сообщение компилятора: `; expected` (ожидалась точка с запятой) и номер строки. Ошибку видно сразу.

#### 2. Ошибка во время выполнения — программа упала

```
int x = int.Parse("пять");   // FormatException
```

Программа *собралась*, но при запуске бросила исключение.

### Как читать ошибку

Сообщение обычно содержит:

- **тип ошибки** (например, `FormatException`);
- **объяснение** («Input string was not in a correct format»);
- **строку и файл**, где случилось.

### Трассировка стека

Когда падает, C# показывает цепочку вызовов — от места падения вверх. Читай **самую верхнюю строку** — это место, где реально случилась ошибка, а не вся цепочка.

### Приёмы отладки без отладчика

- Выводи промежуточные значения: `Console.WriteLine($"debug: {variable}")`.
- Проверяй входные данные перед использованием.
- Читай строчку, на которую указывает ошибка, буквально.

## Примеры

#### Частая ошибка: выход за границы массива

```
 int[] a = { 1, 2, 3 };
Console.WriteLine(a[5]);   // IndexOutOfRangeException
```

Сообщение укажет на строку с `a[5]`. Индекс 5 не существует (есть 0..2).

## Сделай сам (задачи с подсказками)

1. Сделай 3 намеренные ошибки (забытый `;`, опечатка в `Console`, неверный тип) и запиши, что говорит компилятор.
   *Подсказка: компилятор почти всегда указывает строку и номер.*
2. Вызови `int.Parse("abc")` без try/catch и прочитай тип исключения.
   *Подсказка: это FormatException.*
3. Обратись к `array[10]` в массиве из 3 элементов; запиши сообщение.
   *Подсказка: это IndexOutOfRangeException.*
4. Добавь `Console.WriteLine` для отладки внутри метода и найди, где значение становится неверным.
   *Подсказка: печатай значение до и после операции.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Игнорировать сообщение об ошибке | теряешь главную подсказку | читай тип, объяснение и строку |
| Паниковать при стек-трейсе | большая простыня пугает | смотри только верхнюю строку |
| «Мне кажется, ошибка где-то тут» | бессистемный поиск | выводи отладочные значения и сужай круг |
| Менять всё подряд наугад | ломаешь рабочее | правь одну строку и проверяй заново |

## Тест — проверь себя

**Вопрос 1.** Где компилятор указывает место ошибки?

- A) Нигде, надо искать самому.
- B) В сообщении: файл и номер строки.
- C) Всегда в конце программы.

✅ **Верно: B.**
Почему: компилятор сообщает файл и строку.
Почему неверны остальные:

- A — Указывает.
- C — Не «всегда в конце».

**Вопрос 2.** Чем ошибка компиляции отличается от ошибки во время выполнения?

- A) Ничем.
- B) Компиляция: код не собрался; runtime: собрался, но упал при запуске.
- C) Runtime бывает только на Linux.

✅ **Верно: B.**
Почему: это два разных этапа.
Почему неверны остальные:

- A — Разница есть.
- C — Не связано с ОС.

**Вопрос 3.** Что такое трассировка стека?

- A) Список команд компьютера.
- B) Цепочка вызовов, показывающая, где случилась ошибка.
- C) Резервная копия кода.

✅ **Верно: B.**
Почему: стек-трейс показывает путь вызовов.

Почему неверны остальные:

- A — Это не «команды компьютера».
- C — Это не копия.

**Вопрос 4.** `int.Parse("abc")` — это:

- A) Ошибка компиляции.
- B) Ошибка во время выполнения (FormatException).
- C) Валидный код без ошибок.

✅ **Верно: B.**
Почему: код компилируется, но при запуске падает.
Почему неверны остальные:

- A — Компилируется нормально.
- C — При запуске упадёт.

## Итог урока

- Ошибки бывают на этапе компиляции и во время выполнения.
- Читай тип ошибки, объяснение и строку.
- В стек-трейсе смотри верхнюю строку.
- Используй `Console.WriteLine` для отладки.

# Урок 18. Структура проекта и пространства имён

## Цель

Понять, как устроен проект на диске, что такое namespace и как организовать код в файлы.

## Словарик новых слов

- **Проект** — папка с кодом и файл `.csproj` (настройки).
- **Namespace (пространство имён)** — «папка» для имён классов, чтобы они не конфликтовали.
- `using` — «подключить» пространство имён, чтобы не писать его целиком.
- **Solution (.sln)** — объединение нескольких проектов.

## Теория «с нуля»

### Аналогия: квартира и адрес

Представь, что классы — это жильцы. У них есть имена, но «Иван» может быть в разных домах. Адрес (namespace) уточняет, где именно живёт «Иван». Без адресов была бы неразбериха.

### Как выглядит проект на диске

```
MyApp/
├── MyApp.csproj     # настройки: целевая версия .NET и т.п.
├── Program.cs        # входная точка
├── Utils.cs          # ещё один файл с кодом
└── bin/ obj/         # результат сборки (генерируется, в git не нужен)
```

### Зачем namespace

Сгруппировать логически связанные классы и избежать пересечения имён:

```
namespace MyApp.Helpers
{
    public static class MathUtils
    {
        public static int Double(int x) => x * 2;
    }
}
```

### using — подключение

Чтобы использовать класс, либо пишем полное имя, либо подключаем через `using`:

```
using MyApp.Helpers;

int result = MathUtils.Double(5);
```

### Файлы и namespace

Хорошая практика: каждый публичный класс — в отдельном файле, папки повторяют namespace:

```
Utils/
  MathUtils.cs   → namespace MyApp.Helpers
```

### ImplicitUsings

В современных проектах часть стандартных `using` (например, `System`) подключается автоматически (флаг `<ImplicitUsings>enable</ImplicitUsings>` в csproj).

## Примеры

```
// Utils/MathUtils.cs
namespace MyApp.Helpers;

public static class MathUtils
{
    public static int Add(int a, int b) => a + b;
}
```

```
// Program.cs
using MyApp.Helpers;
Console.WriteLine(MathUtils.Add(2, 3));   // 5
```

## Сделай сам (задачи с подсказками)

1. Создай класс в namespace `MyApp.Helpers` с методом `Triple(int)`.
   *Подсказка:* `namespace MyApp.Helpers { public static class ... }`
2. Подключи этот namespace через `using` и вызови метод.
   *Подсказка:* `using MyApp.Helpers;` *в начале файла.*
3. Разложи два класса по разным файлам и namespace.
   *Подсказка: один класс — один файл.*

4. Объясни, зачем нужен namespace, на примере двух классов с одним именем.
   *Подсказка: разные namespace → нет конфликта.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Забыл `using` | имя не найдено | добавь `using` или пиши полное имя |
| Всё в одном файле | трудно ориентироваться | один публичный класс — один файл |
| Папки не соответствуют namespace | путаница | держи папки и namespace согласованными |
| Забыл `;` после namespace | ошибка компиляции | `namespace ...;` |

## Тест — проверь себя

**Вопрос 1.** Что такое namespace?

- A) Тип данных.
- B) «Пространство имён» — группировка имён классов.
- C) Папка на диске.

✅ **Верно: B.**
Почему: namespace логически группирует имена.
Почему неверны остальные:

- A — Это не тип.
- C — Необязательно совпадает с папкой, хотя принято.

**Вопрос 2.** Зачем `using MyApp.Helpers;`?

- A) Чтобы создать проект.
- B) Чтобы не писать полное имя класса каждый раз.
- C) Чтобы удалить namespace.

✅ **Верно: B.**
Почему: using подключает пространство имён.
Почему неверны остальные:

- A — Это не создание проекта.
- C — Это не удаление.

**Вопрос 3.** Что хранит файл `.csproj`?

- A) Исходный код.
- B) Настройки проекта (целевая версия .NET и т.п.).
- C) Готовую программу.

✅ **Верно: B.**
Почему: csproj — это «паспорт» проекта.
Почему неверны остальные:

- A — Код — в .cs файлах.
- C — Готовая программа — в bin/.

**Вопрос 4.** Что обычно игнорируется в git?

- A) `Program.cs`
- B) `bin/` и `obj/`
- C) `*.csproj`

✅ **Верно: B.**
Почему: bin/obj — результат сборки, они генерируются.
Почему неверны остальные:

- A — Program.cs — исходник, хранится.
- C — csproj — исходник, хранится.

## Итог урока

- Проект = папка + `csproj` + исходники.
- `namespace` — группировка имён; `using` — подключение.
- Один публичный класс — один файл; папки ~ namespace.
- `bin/ obj/` — в `.gitignore`.

# Урок 20. Классы и объекты

## Цель

Понять главную идею ООП — «класс» и «объект» — и создать свой первый класс.

## Словарик новых слов

- **Класс** — «чертёж» или «форма». Описывает, какие данные и действия будут у создаваемых «изделий». Сам по себе он ещё не вещь.
- **Объект** — конкретное «изделие», созданное по чертёжу (классу). Ему можно дать имя и использовать.
- **Поле** — переменная внутри класса, хранящая данные объекта.
- **Метод** — действие, которое объект умеет выполнять.

## Теория «с нуля»

### Аналогия: чертёж и автомобиль

Представь завод:

- **Чертёж** «Автомобиль» — это класс. На нём нарисовано: есть руль (поле), есть колёса (поле), есть действие «завестись» (метод). Но чертёж сам по себе не ездит.
- **Автомобиль** на парковке — это объект. Его сделали по чертёжу. Таких машин может быть много, но все они — по одному чертёжу.

### Класс в коде

```
 class Car
{
    // поля — данные объекта
    public string Color;
    public int Speed;

    // метод — действие объекта
    public void Honk()
    {
        Console.WriteLine("Бип!");
    }
}
```

### Создание объекта

Объект создаётся словом `new`:

```
Car myCar = new Car();
myCar.Color = "Красный";   // заполняем поле
myCar.Speed = 60;
myCar.Honk();              // вызываем метод — "Бип!"
```

### Класс ≠ объект — главное различие

```
// Car — это "чертёж" (тип). myCar — конкретный объект.
Car myCar = new Car();
```

Если сказать «создай класс» — мы описываем чертёж. Если «создай объект» — делаем конкретное изделие.

## Сделай сам (задачи с подсказками)

1. Создай класс `Dog` с полем `string Name` и методом `Bark()` (выводит «Гав!»).
   *Подсказка:* `class Dog { public string Name; public void Bark() { Console.WriteLine("Гав!"); } }`
2. Создай два объекта `Dog` с разными именами и вызови у обоих `Bark()`.
   *Подсказка:* `Dog d1 = new Dog(); d1.Name = "Рекс";` *и т.д.*
3. Создай класс `Person` с полями `Name` и `Age` и методом `Introduce()` (выводит «Привет, я ...»).
   *Подсказка: внутри метода используй* `Console.WriteLine($"Привет, я {Name}");`
4. Создай класс `Rectangle` с полями `Width`, `Height` и методом `Area()`, возвращающим площадь.
   *Подсказка:* `public double Area() { return Width * Height; }`

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `Car myCar;` без `new` | объект не создан (myCar = null) | `= new Car();` |
| `new Car()` без переменной | объект создан, но мы его не можем использовать | сохрани в переменную |
| Обращение к полю `myCar.Color` до создания | ошибка (null) | сначала создай объект |
| Имя метода `Bark()` с прописной | методы — с большой буквы | `Bark()`, не `bark()` |

## Тест — проверь себя

**Вопрос 1.** Что такое класс?

- A) Конкретная вещь в памяти.
- B) «Чертёж», описывающий данные и действия.
- C) Переменная.

✅ **Верно: B.**
Почему: класс — описание (тип), а не конкретная вещь.
Почему неверны остальные:

- A — Конкретная вещь — это объект.
- C — Переменная хранит значение; класс — это тип/шаблон.

**Вопрос 2.** Что такое объект?

- A) Чертёж.
- B) Конкретное «изделие», созданное по классу.
- C) Имя класса.

✅ **Верно: B.**
Почему: объект — реальный экземпляр, созданный через `new`.
Почему неверны остальные:

- A — Чертёж — это класс.
- C — Имя класса — не объект.

**Вопрос 3.** Как создать объект класса `Car`?

- A) `Car myCar = new Car();`
- B) `Car myCar;`
- C) `new Car myCar;`

✅ **Верно: A.**
Почему: `new Car()` создаёт объект, и мы сохраняем его в переменную.
Почему неверны остальные:

- B — Объявлена только переменная, объект не создан (будет `null`).
- C — Неверный порядок слов; `new` идёт перед типом.

**Вопрос 4.** Что выведет код?

```
 class Cat { public void Meow() { Console.WriteLine("Мяу!"); } }
Cat c = new Cat();
```

```
c.Meow();
```

- A) Ничего.
- B) `Мяу!`
- C) Ошибку.

✅ **Верно: B.**
Почему: создали объект `c` и вызвали его метод `Meow()`, который выводит «Мяу!».
Почему неверны остальные:

- A — Метод выводит текст.
- C — Ошибки нет: класс и метод описаны верно.

## Итог урока

- **Класс** — чертёж (тип). **Объект** — изделие по чертёжу.
- Объект создаётся через `new`.
- Поля — данные; методы — действия объекта.
- Чтобы использовать объект, сохрани его в переменную.

# Урок 21. Инкапсуляция и свойства

## Цель

Понять, зачем «прятать» данные внутри объекта, и научиться делать это правильно — через свойства.

## Словарик новых слов

- **Инкапсуляция** — «упаковка»: скрытие внутренних данных объекта от постороннего вмешательства. (От слова «капсула» — оболочка.)
- **Поле** — переменная внутри класса (обычно скрытая).
- **Свойство** — «дверца» к полю с проверками: способ дать доступ на чтение/запись, но под контролем.
- **Модификатор доступа** — пометка «кто может видеть». `public` — все, `private` — только сам класс.

## Теория «с нуля»

### Аналогия: приборная панель и двигатель

В машине ты не лезешь руками в двигатель. Ты жмёшь педаль (метод), а за тебя решает механика внутри. Если бы двигатель был открыт — любой мог бы его сломать.
Так и в коде: **внутренние данные лучше прятать**, а доступ давать через специальные «двери» — методы и свойства.

### Что будет без инкапсуляции

```
 class BankAccount
{
    public decimal Balance;   // поле открыто — меняем как хотим
}

BankAccount acc = new BankAccount();
acc.Balance = -1000;   // отрицательный баланс! никто не помешал
```

Это плохо: кто угодно может испортить данные.

### Как правильно: скрытое поле + методы

```
class BankAccount
{
    private decimal _balance;   // скрытое поле

    public decimal Balance       // свойство: только чтение
    {
        get { return _balance; }
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Сумма должна быть положительной");
        _balance += amount;
    }
}
```

Теперь баланс нельзя испортить — он меняется только через `Deposit` с проверкой.

### Свойства с записью

```
class Person
{
    private string _name;

    public string Name
    {
        get { return _name; }
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Имя не может быть пустым");
            _name = value;
        }
    }
}
```

- `get` — что вернуть при чтении.
- `set` — что сделать при записи; слово `value` — то значение, которое присваивают.
- Проверка в `set` — защита от мусора.

### Сокращённая запись (auto-property)

Если проверки не нужны:

```
public string Email { get; set; }
```

Компилятор сам создаст скрытое поле. Это самый частый вид свойства.

### Модификаторы

| Ключевое слово | Кто видит |
|---|---|
| `public` | все |
| `private` | только внутри класса |
| `protected` | внутри класса и его наследников |

Правило: **поле — `private`, доступ наружу — через свойства/методы.**

## Сделай сам (задачи с подсказками)

1. Сделай класс `BankAccount` с скрытым полем и методами `Deposit` / `Withdraw` (нельзя уйти в минус).
   *Подсказка:* `private decimal _balance;` + *проверки в методах.*
2. Класс `Student`: свойство `Age` с проверкой «от 14 до 100».
   *Подсказка: в* `set` *проверяй* `value`*, при ошибке бросай исключение.*
3. Класс `Counter`: скрытое поле `_value`, метод `Increment()`, свойство `Value` (только чтение).
   *Подсказка:* `public int Value { get { return _value; } }`
4. Перепиши класс из прошлого урока `Car`: сделай поле `Color` как auto-property.
   *Подсказка:* `public string Color { get; set; }`

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Поле `public decimal Balance` | данные открыты, можно испортить | скрой в `private`, дай методы/свойства |
| Свойство без `get` | читать нельзя | добавь `get` |
| `set` без проверки | можно записать мусор | добавляй проверки, где важно |
| `_balance += amount` в методе без условия | можно добавить отрицательное | проверяй `amount` |

## Тест — проверь себя

**Вопрос 1.** Зачем инкапсуляция?

- A) Чтобы программа была красивее.
- B) Чтобы скрыть внутренние данные и защитить их от неправильного изменения.
- C) Чтобы запретить вызывать методы.

✅ **Верно: B.**
Почему: скрытие данных + контроль через проверки защищает состояние объекта.
Почему неверны остальные:

- A — Это не про красоту, а про надёжность.
- C — Методы как раз и остаются доступными.

**Вопрос 2.** Что делает `set` в свойстве?

- A) Возвращает значение.
- B) Обрабатывает запись значения (в `value` лежит то, что присваивают).
- C) Удаляет поле.

✅ **Верно: B.**
Почему: `set` срабатывает при присваивании, `value` — новое значение.
Почему неверны остальные:

- A — Возврат — это `get`.
- C — `set` ничего не удаляет.

**Вопрос 3.** Модификатор `private` означает:

- A) Поле видят все.
- B) Поле видно только внутри этого класса.
- C) Поле видно только внутри сборки.

✅ **Верно: B.**
Почему: `private` — доступ только из самого класса.
Почему неверны остальные:

- A — «Все» — это `public`.
- C — Это описание `internal`, а не `private`.

**Вопрос 4.** Что произойдёт при `acc.Balance = -5;`, если `Balance` — только чтение (только `get`)?

- A) Баланс станет −5.
- B) Произойдёт ошибка: писать нельзя.
- C) Ничего.

✅ **Верно: B.**

Почему: без `set` свойство нельзя присваивать — компилятор даст ошибку.

Почему неверны остальные:

- A — Запись невозможна без `set`.
- C — Ошибка всё-таки будет (на этапе компиляции).

## Итог урока

- **Инкапсуляция** — прячем внутренности, даём контрольный доступ.
- Поля — `private`; наружу — свойства/методы.
- Свойство: `get` (чтение) + `set` (запись) с проверкой через `value`.
- Auto-property `{ get; set; }` — когда проверки не нужны.

# Урок 22. Конструкторы

## Цель

Понять, как объект «рождается» и как задать его начальное состояние сразу при создании.

## Словарик новых слов

- **Конструктор** — специальный метод, который вызывается автоматически при создании объекта ( `new` ). Он «настраивает» объект с самого начала.
- **Параметр** — входное значение метода.
- **Инициализация** — задание начальных значений.

## Теория «с нуля»

### Аналогия: регистрация новорождённого

Когда рождается человек, его сразу регистрируют: дают имя, записывают дату рождения. Так и объект: при создании (через `new` ) вызывается **конструктор**, который может сразу задать его данные.

### Конструктор по умолчанию

Если ты ничего не пишешь, C# создаёт пустой конструктор: поля остаются со значениями «по умолчанию» (у чисел — 0, у строк — null).

```
 class Person { public string Name { get; set; } }

Person p = new Person();   // вызвался конструктор по умолчанию
```

### Свой конструктор

```
 class Person
{
    public string Name { get; }

    public Person(string name)   // конструктор с параметром
    {
        Name = name;
    }
}
```

Теперь при создании **обязательно** передаём имя:

```
Person p = new Person("Иван");   // сразу имя задано
```

Имя конструктора совпадает с именем класса, и у него нет типа возврата.

### Почему это полезно

1. Объект не может появиться «наполовину настроенным».
2. Нельзя забыть важные данные — они передаются сразу.
3. Можно поставить проверки в конструкторе.

### Проверка в конструкторе

```
public Person(string name)
{
    if (string.IsNullOrWhiteSpace(name))
        throw new ArgumentException("Имя не может быть пустым");
    Name = name;
}
```

### Несколько конструкторов

Можно сделать несколько конструкторов (с разными наборами параметров):

```
public Person()                        { Name = "Без имени"; }
public Person(string name)             { Name = name; }
```

## Примеры

```
class Rectangle
{
    public double Width { get; }
    public double Height { get; }

    public Rectangle(double width, double height)
    {
        Width = width;
        Height = height;
    }

    public double Area() => Width * Height;
}
```

```
 Rectangle r = new Rectangle(4, 5);
Console.WriteLine(r.Area());   // 20
```

## Сделай сам (задачи с подсказками)

1. Класс `Car` с конструктором, принимающим цвет. Создай красную машину.
   *Подсказка:* *`public Car(string color) { Color = color; }`*
2. Класс `Student` с конструктором (имя, возраст). Запрети пустое имя.
   *Подсказка: проверка* *`string.IsNullOrWhiteSpace(name)`*.
3. Класс `Point` с конструктором (x, y) и методом `DistanceTo(Point other)`.
   *Подсказка: формула расстояния — квадратный корень из суммы квадратов разностей.*
4. Сделай два конструктора у `Timer`: пустой (старт с 0) и с начальным значением секунд.
   *Подсказка:* *`public Timer() {}`* *и* *`public Timer(int seconds) { Seconds = seconds; }`*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `Person(string name)` без `public` | не виден снаружи | `public` |
| Конструктор с типом возврата ( `void Person(...)` ) | у конструктора нет типа возврата | просто `Person(...)` |
| `new Person()` при единственном конструкторе с параметром | такого конструктора нет | передай параметр: `new Person("И")` |
| Забыл `{ }` у конструктора | пустой код | добавь тело |

## Тест — проверь себя

**Вопрос 1.** Когда вызывается конструктор?

- A) Когда вызываешь метод `Run()`.
- B) Автоматически при создании объекта ( `new` ).
- C) При выходе из программы.

✅ **Верно: B.**
Почему: конструктор срабатывает в момент `new`.
Почему неверны остальные:

- A — `Run()` — обычный метод.
- C — При выходе вызывается деструктор/очистка, не конструктор.

**Вопрос 2.** У конструктора:

- A) Есть тип возврата `void`.
- B) Нет типа возврата.
- C) Обязательно есть `return`.

✅ **Верно: B.**
Почему: конструктор не возвращает значение, поэтому типа возврата нет.
Почему неверны остальные:

- A — `void` в конструкторе писать нельзя.
- C — `return` там не нужен.

**Вопрос 3.** Что произойдёт при `Person p = new Person();`, если есть только конструктор `Person(string name)`?

- A) Создастся объект с пустым именем.
- B) Ошибка компиляции: такого конструктора без параметров нет.
- C) Создастся объект с именем null.

✅ **Верно: B.**
Почему: раз объявлен конструктор с параметром, «пустого» конструктора больше нет — вызов без аргументов не сработает.
Почему неверны остальные:

- A, C — Без сствующего конструктора объект не создастся вовсе.

**Вопрос 4.** Зачем задавать данные через конструктор?

- A) Чтобы объект сразу был корректно настроен, и нельзя было «забыть» важное.
- B) Чтобы программа работала медленнее.
- C) Чтобы поля были скрыты.

✅ **Верно: A.**
Почему: конструктор гарантирует начальное состояние.
Почему неверны остальные:

- B — Это не про скорость.
- C — Скрытие полей — это инкапсуляция (прошлый урок), не цель конструктора.

## Итог урока

- **Конструктор** — автоматический метод при `new`, имя = имя класса, без типа возврата.
- Позволяет сразу задать данные и проверить их.
- Если объявил конструктор — «пустого» больше нет.

# Урок 23. Наследование

## Цель

Понять, как сделать «семейство» классов: общее — в базовом, особенное — в наследниках.

## Словарик новых слов

- **Наследование** — механизм, при котором класс получает (наследует) члены другого класса.
- **Базовый класс** — «родитель», от которого наследуют.
- **Наследник (производный класс)** — «ребёнок», который берёт родительское и добавляет своё.
- `base` — ссылка на базовый класс (например, чтобы вызвать его конструктор).

## Теория «с нуля»

### Аналогия: семейство животных

Опишем «Животное»: у него есть имя и умение есть. Теперь «Собака» — это Животное + умеет лаять. «Кошка» — Животное + умеет мурчать. Общее (имя, есть) кладём в базовый класс, а особенное — в наследников.

### Базовый класс

```
class Animal
{
    public string Name { get; set; }

    public void Eat()
    {
        Console.WriteLine($"{Name} ест");
    }
}
```

### Наследник

```
class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Гав!");
```

```
    }
}
```

Символ `:` означает «наследуется от». Теперь `Dog` имеет всё от `Animal` плюс своё:

```
Dog dog = new Dog();
dog.Name = "Рекс";
dog.Eat();    // из базового класса
dog.Bark();   // своё
```

### Конструкторы и `base`

Если у базового класса есть конструктор с параметрами, наследник обязан его вызвать:

```
class Animal
{
    public string Name { get; }
    public Animal(string name) { Name = name; }
}

class Dog : Animal
{
    public Dog(string name) : base(name) { }   // вызываем конструктор Animal
}
```

`: base(name)` — «вызови родительский конструктор, передав ему name».

### Одиночное наследование

В C# класс может иметь **только одного** «родителя». (А вот интерфейсов — можно много, см. следующий урок.)

## Примеры

```
class Shape
{
    public virtual double Area() => 0;
}

class Circle : Shape
{
    public double Radius { get; }
    public Circle(double radius) { Radius = radius; }
```

```
    public override double Area() => Math.PI * Radius * Radius;
}
```

(Про `virtual` / `override` — подробно в следующем уроке.)

## Сделай сам (задачи с подсказками)

1. Создай базовый класс `Vehicle` (поле `Speed`) и наследника `Car`, добавляющего метод `Honk()`.
   *Подсказка:* `class Car : Vehicle { public void Honk() { ... } }`
2. Создай `Animal` с конструктором `Animal(string name)` и наследника `Cat : Animal` с `: base(name)`.
   *Подсказка:* `public Cat(string name) : base(name) { }`
3. Создай иерархию: `Shape` → `Rectangle` (поля Width/Height + площадь) и `Circle` (радиус).
   *Подсказка: общий метод* `Area()` *— в базовом, переопределяй в наследниках.*
4. Покажи, что объект `Car` можно положить в переменную типа `Vehicle` (наследник — это и базовый тип тоже).
   *Подсказка:* `Vehicle v = new Car();` *— это легально.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `class Dog : Animal` но без конструктора `base` | если у Animal конструктор с параметром — ошибка | добавь `: base(...)` |
| Наследование «ради переиспользования» кода | не родственная связь | лучше композиция (поле внутри) |
| `class Dog : Animal, Another` | множественное наследование классов запрещено | один базовый класс |
| Забыл `public` у членов | недоступны наследнику/извне | `public` или `protected` |

## Тест — проверь себя

**Вопрос 1.** Что означает `class Dog : Animal`?

- A) Dog содержит объект Animal.
- B) Dog наследует (получает) члены Animal.
- C) Dog и Animal — одно и то же.

✅ **Верно: B.**
Почему: `:` — знак наследования.
Почему неверны остальные:

- A — «Содержит объект» — это композиция, не наследование.
- C — Это разные классы, просто связанные.

**Вопрос 2.** Что унаследует `Dog` от `Animal`?

- A) Только методы.
- B) Все `public` (и `protected`) члены Animal.
- C) Ничего.

✅ **Верно: B.**
Почему: доступные члены базового класса становятся доступны наследнику.
Почему неверны остальные:

- A — Наследуются и поля/свойства, и методы.
- C — Наследование — это и есть передача членов.

**Вопрос 3.** Как вызвать конструктор базового класса?

- A) `base(name)`
- B) `super(name)`
- C) Никак, он вызывается сам всегда с теми же параметрами.

✅ **Верно: A.**
Почему: в C# — `base(...)`.
Почему неверны остальные:

- B — `super` — это из Java/Python, в C# так нельзя.
- C — Параметры нужно передать явно.

**Вопрос 4.** Можно ли сделать `class Dog : Animal, Machine`?

- A) Да.
- B) Нет: в C# у класса один базовый класс.
- C) Да, если Machine — абстрактный.

✅ **Верно: B.**
Почему: множественное наследование классов в C# запрещено.
Почему неверны остальные:

- A — Нельзя.
- C — Абстрактность не меняет количество базовых классов.

## Итог урока

- **Наследование** — класс получает члены другого класса (`:`).

- Базовый класс — общее; наследник — добавляет своё.
- Конструктор базового класса вызывается через `base(...)`.
- Базовый класс — один.

# Урок 24. Полиморфизм

## Цель

Понять, как один и тот же код может вести себя по-разному для разных наследников.

## Словарик новых слов

- **Полиморфизм** — (буквально «много форм») способность разных классов отвечать на одну и ту же команду по-своему.
- **Виртуальный метод** ( `virtual` ) — метод, который можно переопределить в наследнике.
- **Переопределение** ( `override` ) — замена поведения метода базового класса в наследнике.

## Теория «с нуля»

### Аналогия: один пульт, много устройств

У тебя есть пульт с кнопкой «Пуск». Для телевизора она включает экран, для микроволновки — нагрев, для кондиционера — обдув. Одна команда — но каждое устройство отвечает по-своему. Это полиморфизм.

### Проблема без полиморфизма

Хочется заставить всех животных «говорить», но по-разному:

```
 class Animal { public string Speak() => "..."; }
class Dog : Animal { public string Speak() => "Гав!"; }
class Cat : Animal { public string Speak() => "Мяу!"; }
```

Если держать собаку как `Animal`, вызовется `Speak` из `Animal` (потому что метод не виртуальный) — и мы получим «...», а не «Гав!».

### Решение: virtual + override

```
 class Animal
{
    public virtual string Speak() => "...";
}

class Dog : Animal
{
```

```
    public override string Speak() => "Гав!";
}

class Cat : Animal
{
    public override string Speak() => "Мяу!";
}
```

Теперь:

```
Animal[] animals = { new Dog(), new Cat() };
foreach (Animal a in animals)
{
    Console.WriteLine(a.Speak());   // "Гав!" затем "Мяу!"
}
```

Ключевой момент: **во время выполнения** программа смотрит на фактический тип объекта (Dog, Cat) и вызывает именно его переопределённый метод. Это и есть полиморфизм.

### Для чего это нужно

- Новое животное = новый класс, а не правка всех `if`.
- Код работает с базовым типом `Animal`, а поведение у каждого своё.

### Абстрактные методы и классы

Если у базового класса нет смысла иметь реализацию, можно сделать метод `abstract` (без тела) и класс `abstract` (нельзя создавать объекты):

```
abstract class Shape
{
    public abstract double Area();
}
class Circle : Shape
{
    public double Radius { get; }
    public Circle(double r) { Radius = r; }
    public override double Area() => Math.PI * Radius * Radius;
}
```

- `abstract class` — нельзя `new Shape()`, только наследники.
- `abstract` метод — обязан быть переопределён в наследниках.

## Сделай сам (задачи с подсказками)

1. Создай `Animal` с `virtual Speak()` и наследников `Dog`, `Cat`, `Cow` с `override Speak()`.
   *Подсказка:* `public override string Speak() => "Гав!";`
2. Создай массив `Animal[]` со всеми тремя и выведи `Speak()` каждого через `foreach`.
   *Подсказка: foreach по* `Animal` *— вывод будет разный.*
3. Создай `abstract class Shape` с `abstract double Area()` и наследников `Circle`, `Rectangle`, `Triangle`.
   *Подсказка: в наследниках* `override double Area()`.
4. Вычисли сумму площадей всех фигур в массиве `Shape[]`.
   *Подсказка:* `sum += s.Area();` *в foreach.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `override` без `virtual` в базовом | нечего переопределять — ошибка | добавь `virtual` в базовый класс |
| `virtual` без `override` в наследнике | метод скрывается, полиморфизм не работает | добавь `override` |
| `new Shape()` для abstract | нельзя создавать объекты абстрактного класса | создавай наследников |
| Забыл переопределить abstract-метод | класс останется абстрактным (или ошибка) | определи `override` |

## Тест — проверь себя

**Вопрос 1.** Что такое полиморфизм?

- A) Возможность одному коду вести себя по-разному для разных типов.
- B) Способ скрыть данные.
- C) Ускорение программы.

✅ **Верно: A.**
Почему: полиморфизм — «много форм» одного вызова.
Почему неверны остальные:

- B — Скрытие данных — это инкапсуляция.
- C — Полиморфизм не про скорость.

**Вопрос 2.** Зачем `virtual` в базовом классе?

- A) Чтобы его нельзя было наследовать.
- B) Чтобы разрешить переопределение метода в наследниках.
- C) Чтобы скрыть метод.

✅ **Верно: B.**
Почему: `virtual` «открывает» метод для переопределения.
Почему неверны остальные:

- A — Запрет наследования — это `sealed`.
- C — Скрытие — это `private`.

**Вопрос 3.** Что выведет код?

```
class Animal { public virtual string Speak() => "..."; }
class Dog : Animal { public override string Speak() => "Гав!"; }
Animal a = new Dog();
Console.WriteLine(a.Speak());
```

- A) `...`
- B) `Гав!`
- C) Ошибку

✅ **Верно: B.**
Почему: несмотря на тип переменной Animal, фактический объект — Dog, и вызывается его `override`.
Почему неверны остальные:

- A — Так было бы без `virtual/override`.
- C — Ошибки нет.

**Вопрос 4.** Можно ли создать объект абстрактного класса ( `new Shape()` )?

- A) Да.
- B) Нет — абстрактный класс нельзя инстанцировать.
- C) Да, но только один раз.

✅ **Верно: B.**
Почему: `abstract class` — только шаблон для наследников.
Почему неверны остальные:

- A — Нельзя.
- C — «Один раз» не имеет значения — нельзя вовсе.

## Итог урока

- **Полиморфизм** — один вызов, разное поведение.
- `virtual` + `override` — разрешение и замена поведения.
- Абстрактный класс/метод — шаблон без реализации, для наследников.

# Урок 25. Интерфейсы

## Цель

Понять «договор на услуги» — интерфейс — и чем он отличается от наследования.

## Словарик новых слов

- **Интерфейс** — «договор»: список того, что объект обязано уметь. Без реализации — только «шапка».
- **Реализация интерфейса** — класс, который выполняет договор (пишет конкретный код методов).
- **Контракт** — то же, что договор: набор обязательств.

## Теория «с нуля»

### Аналогия: договор

Подрядчик и заказчик подписывают договор: «должен сделать ремонт до 1 числа». В договоре написано, **что** сделать, но не как именно — каждый подрядчик делает по-своему. Интерфейс — это такой договор для классов: «ты обязан уметь вот это» — но как именно, класс решает сам.

### Объявление интерфейса

```
interface IPayable
{
    decimal CalculateAmount();
}
```

- Имя обычно начинается с `I` (от interface).
- Внутри — только «шапки» методов, без тела.

### Реализация классом

```
class Salary : IPayable
{
    public decimal Monthly { get; set; }

    public decimal CalculateAmount() => Monthly;
}
```

```
class Contract : IPayable
{
    public decimal HourlyRate { get; set; }
    public int Hours { get; set; }

    public decimal CalculateAmount() => HourlyRate * Hours;
}
```

Оба класса «подписали договор»: они умеют `CalculateAmount()`, но считают по-разному.

### Зачем это нужно

Можно обращаться ко всем «платежам» единообразно:

```
 IPayable[] payments = { new Salary { Monthly = 50000m }, new Contract { HourlyRate = 1500m,
Hours = 80 } };

foreach (IPayable p in payments)
{
    Console.WriteLine(p.CalculateAmount());
}
```

### Интерфейс против наследования

| Вопрос | Абстрактный класс | Интерфейс |
|---|---|---|
| Сколько можно «взять»? | один | несколько |
| Может содержать данные/поля? | да | нет (только контракты) |
| Когда выбирать? | есть общая логика + `IS-A` | нужен только контракт «умеет» `CAN-DO` |

Ключевая мысль: интерфейс описывает **что умеет** объект, а не **чем он является**.

### Несколько интерфейсов

Класс может реализовывать много интерфейсов:

```
class Robot : IWorkable, IChargeable { ... }
```

## Сделай сам (задачи с подсказками)

1. Создай интерфейс `IPayable` с `CalculateAmount()` и две реализации (Salary, Contract).
   *Подсказка: имя интерфейса начинается с `I`.*

2. Создай `ILoggable` с методом `Log(string message)` и класс `ConsoleLogger : ILoggable`.
   *Подсказка:* `public void Log(string message) => Console.WriteLine(message);`
3. Создай интерфейс `IShape` с `Area()` и реализуй его в `Circle` и `Square`.
   *Подсказка:* `class Circle : IShape { ... }`
4. Покажи, что массив типа `IShape[]` может содержать и круг, и квадрат.
   *Подсказка:* `IShape[] shapes = { new Circle(...), new Square(...) };`

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Метод интерфейса с телом | в интерфейсе нет тела | только «шапка» |
| Класс не реализовал все методы интерфейса | компилятор потребует | реализуй все методы |
| Интерфейс с данными/полями | интерфейсы не хранят состояние | не используй поля |
| «Реализация» с другим именем метода | должно совпадать с объявлением | точное совпадение имени/типов |

## Тест — проверь себя

**Вопрос 1.** Что такое интерфейс?

- A) Готовый класс с логикой.
- B) «Договор»: список методов, которые класс обязан реализовать.
- C) Переменная.

✅ **Верно: B.**
Почему: интерфейс задаёт контракт без реализации.
Почему неверны остальные:

- A — Готовый класс — это класс, не интерфейс.
- C — Это не переменная.

**Вопрос 2.** Сколько интерфейсов может реализовывать класс?

- A) Только один.
- B) Сколько угодно.
- C) Ни одного.

✅ **Верно: B.**
Почему: множественная реализация интерфейсов разрешена.

Почему неверны остальные:

- A — Один — это ограничение для классов (базовый класс один).
- C — Тогда зачем они нужны.

**Вопрос 3.** В чём главное отличие интерфейса от абстрактного класса?

- A) Интерфейс — только контракт (без данных и реализации); абстрактный класс может иметь реализацию и поля.
- B) Они полностью одинаковы.
- C) Абстрактный класс нельзя наследовать.

✅ **Верно: A.**
Почему: интерфейс — «что умеет», абстрактный класс может дать общую логику.
Почему неверны остальные:

- B — Есть различия.
- C — Абстрактный класс как раз создаётся для наследования.

**Вопрос 4.** Что обязан сделать класс, реализующий `IPayable`?

- A) Ничего.
- B) Реализовать все методы интерфейса (здесь — `CalculateAmount`).
- C) Создать объект интерфейса.

✅ **Верно: B.**
Почему: это и есть «выполнение договора».
Почему неверны остальные:

- A — Иначе компилятор выдаст ошибку.
- C — Объекты интерфейсов не создаются — интерфейс не класс.

## Итог урока

- **Интерфейс** — договор о возможностях (список методов, имя с `I`).
- Класс **реализует** интерфейс, выполняя все его методы.
- Интерфейсов может быть много; это про «умеет», а не «является».

## Переходим дальше

- 26 · Коллекции — списки и словари
- 27 · Перечисления enum — заранее заданные варианты

# Урок 26. Коллекции

## Цель

Познакомиться с коллекциями — «умными контейнерами» для множества значений: списком, словарём и множеством.

## Словарик новых слов

- **Коллекция** — контейнер, который хранит много значений и умеет с ними работать (добавлять, искать, удалять).
- `List<T>` — динамический список: «вереница» элементов, размер может расти.
- `Dictionary<TKey,TValue>` — словарь: пары «ключ → значение», мгновенный поиск по ключу.
- `HashSet<T>` — множество уникальных элементов, быстрая проверка «есть ли».
- **Обобщение (generic)** `<T>` — «тип-заполнитель»: подставляем реальный тип, например `List<int>`.

## Теория «с нуля»

### Почему не хватает массива

Массив — фиксированного размера. А часто нужно добавлять/убирать элементы. Для этого есть коллекции.

### List — список

```
 List<string> names = new List<string>();
names.Add("Иван");
names.Add("Мария");
Console.WriteLine(names[0]);    // "Иван"
names.Remove("Иван");
Console.WriteLine(names.Count);  // 1
```

- `Add` — добавить, `Remove` — удалить, `Count` — количество.
- Доступ по индексу, как у массива: `names[0]`.
- Удобный способ создать с данными: `List<int> nums = new List<int> { 1, 2, 3 };`

### Словарь Dictionary — «телефонная книга»

```
Dictionary<string, int> ages = new Dictionary<string, int>();
ages["Иван"] = 30;
ages["Мария"] = 28;

Console.WriteLine(ages["Иван"]);           // 30
if (ages.ContainsKey("Пётр"))
{
    Console.WriteLine(ages["Пётр"]);
}
else
{
    Console.WriteLine("Такого ключа нет");
}
```

Словарь хранит пары. По **ключу** ищет очень быстро. Ключи уникальны.

### Множество HashSet — «список с отсутствием повторов»

```
HashSet<string> tags = new HashSet<string>();
tags.Add("csharp");
tags.Add("csharp");   // дубликат игнорируется
Console.WriteLine(tags.Count);              // 1
Console.WriteLine(tags.Contains("csharp")); // True
```

`HashSet` гарантирует уникальность и быстро отвечает «есть ли элемент».

### Перебор foreach

Любую коллекцию можно перебрать:

```
foreach (string name in names)
{
    Console.WriteLine(name);
}
```

## Примеры

```
List<int> numbers = new List<int> { 4, 1, 3 };
numbers.Sort();                     // 1, 3, 4
int total = 0;
foreach (int n in numbers) total += n;
Console.WriteLine(total);           // 8
```

## Сделай сам (задачи с подсказками)

1. `List<string>`: добавь три имени, выведи количество и второе имя.
   *Подсказка:* *`names.Add("...")`* *и* *`names[1]`*.
2. Словарь-«телефонная книга»: добавь пары «имя → номер», найди номер по имени.
   *Подсказка:* *`dict[name]`* *и* *`ContainsKey`*.
3. Подсчитай частоту слов в строке через `Dictionary<string,int>`.
   *Подсказка:* *`dict[word] = dict.GetValueOrDefault(word) + 1;`*
4. `HashSet<int>`: добавь числа и выведи, есть ли в нём 5.
   *Подсказка:* *`Contains(5)`*.
5. Перебери словарь через foreach и выведи ключи и значения.
   *Подсказка:* *`foreach (var pair in dict) Console.WriteLine($"{pair.Key}: {pair.Value}");`*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `dict["нет_такого"]` без проверки | ошибка KeyNotFoundException | используй `ContainsKey` или `TryGetValue` |
| Изменение коллекции во время foreach | ошибка | собери изменения отдельно, примени потом |
| Дубликаты в HashSet и ожидание двух | их будет один | HashSet — для уникальности |
| `List<T>` там, где нужен быстрый поиск | ищет долго (O(n)) | для поиска — Dictionary/HashSet |

## Тест — проверь себя

**Вопрос 1.** Что подходит для быстрого поиска по ключу?

- A) `List<T>`
- B) `Dictionary<TKey,TValue>`
- C) Массив

✅ **Верно: B.**
Почему: словарь ищет по ключу очень быстро.
Почему неверны остальные:

- A, C — Список/массив ищут перебором (медленнее).

**Вопрос 2.** Что будет, если в `HashSet<int>` добавить одно и то же число дважды?

- A) Оно попадёт дважды.
- B) Останется одно вхождение (дубликат отброшен).
- C) Ошибка.

✅ **Верно: B.**
Почему: `HashSet` хранит только уникальные элементы.
Почему неверны остальные:

- A — Известных повторов не будет.
- C — Ошибки нет, просто дубликат игнорируется.

**Вопрос 3.** Чем `List<string>` отличается от массива `string[]`?

- A) Ничем.
- B) Список умеет расти (Add/Remove); массив фиксирован.
- C) Массив умеет расти.

✅ **Верно: B.**
Почему: `List` динамический, массив — фиксированный.
Почему неверны остальные:

- A — Разница есть.
- C — Наоборот.

**Вопрос 4.** Что выведет код?

```
 List<int> list = new List<int> { 1, 2, 3 };
list.Add(4);
Console.WriteLine(list.Count);
```

- A) 3
- B) 4
- C) 7

✅ **Верно: B.**
Почему: было 3 элемента, добавили 1 → стало 4.
Почему неверны остальные:

- A — 3 — это до добавления.
- C — 7 — это сумма значений, а не количество.

## Итог урока

- `List<T>` — динамический список с Add/Remove.

- `Dictionary<TKey,TValue>` — пары «ключ→значение», быстрый поиск.
- `HashSet<T>` — уникальные элементы.
- `foreach` перебирает любую коллекцию.

## Переходим дальше

- 27 · Перечисления enum — заранее заданные варианты

# Урок 27. Перечисления enum

## Цель

Научиться задавать «список заранее известных вариантов» и использовать его вместо магических чисел и строк.

## Словарик новых слов

- **Перечисление (enum)** — тип, который перечисляет набор именованных вариантов.
- **Магическое число** — число в коде без пояснений (например, `3` — «а что значит 3?»).
- **Строка-статус** — статус, хранимый как текст (легко ошибиться в регистре/опечатке).

## Теория «с нуля»

### Аналогия: дни недели

В неделе заранее заданы 7 дней. «Пятница» — это определённый вариант, а не произвольный текст. Если бы мы записывали день просто строкой, легко было бы написать «пятнца» с опечаткой. Перечисление исключает такие ошибки.

### Объявление enum

```
enum OrderStatus
{
    New,
    Paid,
    Shipped,
    Delivered
}
```

Теперь `OrderStatus` — тип, у которого возможны только эти значения.

### Использование

```
OrderStatus status = OrderStatus.Paid;

if (status == OrderStatus.Paid)
{
    Console.WriteLine("Заказ оплачен");
}
```

### Почему это лучше «магических чисел»

Плохо:

```
if (orderStatus == 2) { ... }   // что такое 2?
```

Хорошо:

```
if (orderStatus == OrderStatus.Paid) { ... }
```

Имя `Paid` объясняет смысл, и нельзя перепутать с другой веткой.

### enum в switch

```
string message = status switch
{
    OrderStatus.New => "Новый",
    OrderStatus.Paid => "Оплачен",
    OrderStatus.Shipped => "Отправлен",
    OrderStatus.Delivered => "Доставлен",
    _ => "Неизвестно"
};
```

### Парсинг

Превращение строки в enum:

```
if (Enum.TryParse("Paid", out OrderStatus s))
{
    Console.WriteLine(s);   // Paid
}
```

## Примеры

```
enum Priority { Low, Medium, High }

Priority p = Priority.High;

if (p == Priority.High) Console.WriteLine("Срочно!");
```

## Сделай сам (задачи с подсказками)

1. Создай `enum Weekday { Monday, Tuesday, ... }` и выведи, что `Weekday.Friday` — это валидное значение.
   *Подсказка:* `enum Weekday { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }`
2. Создай `enum TaskStatus { Todo, InProgress, Done }` и используй его в `if`.
   *Подсказка:* `if (status == TaskStatus.Done)`.
3. Создай `enum Priority { Low, Medium, High }` и switch-выражение, возвращающее строку.
   *Подсказка:* `Priority.High => "Высокий"`.
4. Перепиши код, где статус был строкой "done"/"DONE", на enum.
   *Подсказка: теперь сравнение безопасно:* `status == TaskStatus.Done`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Сравнение `status == "done"` для enum | типы не совпадают (enum vs string) | `status == TaskStatus.Done` |
| `int` вместо enum | магические числа | используй enum |
| Забыл `_` в switch-выражении | может быть не покрыто | добавляй `_ => ...` |
| Имя enum с маленькой буквы | типы — с большой | `OrderStatus`, не `orderStatus` |

## Тест — проверь себя

**Вопрос 1.** Что такое `enum`?

- A) Структура с методами.
- B) Тип с заранее заданным набором именованных вариантов.
- C) Переменная для чисел.

✅ **Верно: B.**
Почему: enum перечисляет допустимые варианты.
Почему неверны остальные:

- A — Это не структура с логикой.
- C — Это не про «любые числа».

**Вопрос 2.** Зачем использовать enum вместо строки-статуса?

- A) Чтобы код был длиннее.
- B) Чтобы исключить опечатки и перепутывание вариантов.
- C) Чтобы быстрее работало.

✅ **Верно: B.**
Почему: набор вариантов фиксирован, нельзя случайно написать «doneed».
Почему неверны остальные:

- A — Это не про длину.
- C — Главное — надёжность, а не скорость.

**Вопрос 3.** Как сравнить, что `status` равен `Paid`?

- A) `status == "Paid"`
- B) `status == OrderStatus.Paid`
- C) `status.Paid`

✅ **Верно: B.**
Почему: сравниваем значение enum с элементом типа.
Почему неверны остальные:

- A — Строку нельзя сравнить с enum напрямую (типы разные).
- C — Такой синтаксис не существует.

**Вопрос 4.** Что делает `Enum.TryParse("Paid", out OrderStatus s)`?

- A) Выводит «Paid».
- B) Пытается превратить строку в значение enum.
- C) Создаёт новый enum.

✅ **Верно: B.**
Почему: это безопасный разбор строки в enum.
Почему неверны остальные:

- A — Это не вывод.
- C — Enum не «создаётся» так.

## Итог урока

- `enum` — набор именованных вариантов.
- Заменяет «магические числа» и строки-статусы.
- Сравнение — через `==` с элементом enum.
- `Enum.TryParse` — безопасный разбор строки.

## Итог уровня 2

Ты освоил основы ООП:

- классы и объекты, поля и методы;
- инкапсуляцию и свойства;
- конструкторы;
- наследование и полиморфизм;
- интерфейсы;
- коллекции (List, Dictionary, HashSet);
- перечисления enum.

Поздравляю — **Уровень 2 пройден!** Дальше — Уровень 3 (Middle): исключения, тесты, LINQ, асинхронность, JSON.

# Урок 28. Null-безопасность

## Цель

Понять, что такое `null`, почему обращение к нему ломает программу, и как безопасно с ним работать.

## Словарик новых слов

- **`null`** — «ничего», отсутствие ссылки. Переменная ссылочного типа может не указывать ни на что.
- **`NullReferenceException`** — ошибка: попытались использовать `null` как объект.
- **Nullable `?`** — пометка «может быть null».
- **`??`** — «если null, возьми что-то другое».
- **`?.`** — «если не null, обратись к члену».

## Теория «с нуля»

### Аналогия: пустая коробка

Представь коробку без содержимого (`null`). Если ты попробуешь взять из неё предмет (`box.Length`) — ничего нет, и программа упадёт. `null` — это «коробка с надписью: внутри ничего нет».

### Что такое NullReferenceException

```
 string s = null;
Console.WriteLine(s.Length);   // NullReferenceException
```

`s` — null, а мы просим `Length`. Это как открыть пустую коробку и требовать «достань предмет».

### Как предотвратить: проверка

```
 if (s != null)
{
    Console.WriteLine(s.Length);
}
```

### Оператор `??` — «если null, то...»

```
string name = Console.ReadLine();      // может быть null
string safe = name ?? "Гость";        // если name == null → "Гость"
```

### Оператор `?.` — «если не null, обращайся»

```
int? length = s?.Length;   // если s == null → length = null (без падения)
```

### Пометка nullable `?`

В современном C# мы «говорим» компилятору, может ли быть null:

```
string? maybe = null;    // может быть null
string required = "текст"; // не должен быть null
```

Компилятор предупредит, если ты разыменуешь `maybe` без проверки.

### Чего не делать

- Не «глотать» NullReferenceException надеждой, что не случится.
- Проверяй входные данные (особенно извне: консоль, файлы, сеть).

## Примеры

```
// Безопасное чтение
string? input = Console.ReadLine();
Console.WriteLine(input?.Trim() ?? "(пусто)");
```

## Сделай сам (задачи с подсказками)

1. Создай `string s = null;` и попробуй `s.Length` — запиши тип ошибки.
   *Подсказка: это NullReferenceException.*
2. Исправь это через `if (s != null)`.
   *Подсказка: проверка перед использованием.*
3. Используй `??` для значения по умолчанию.
   *Подсказка:* `string x = s ?? "по умолчанию";`
4. Используй `?.` и сравни результат с null.
   *Подсказка:* `int? len = s?.Length;`

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `s.Length` при s == null | падение | проверь или используй `?.` |
| Пометить `string` как nullable, а потом не проверять | компилятор предупредит | проверяй |
| Использовать `??` на null-объекте, не думая о смысле | иногда маскирует ошибку | выбирай осознанно |

## Тест — проверь себя

**Вопрос 1.** Что такое `null`?

- A) Ноль.
- B) Отсутствие ссылки («ничего»).
- C) Пустая строка.

✅ **Верно: B.**
Почему: null — отсутствие объекта, а не 0 и не "".
Почему неверны остальные:

- A — 0 — это число.
- C — "" — пустая, но *существующая* строка.

**Вопрос 2.** Что будет при `string s = null; Console.WriteLine(s.Length);`?

- A) Выведет 0.
- B) NullReferenceException.
- C) Выведет «null».

✅ **Верно: B.**
Почему: обращение к члену null-объекта падает.
Почему неверны остальные:

- A, C — Программа упадёт до вывода.

**Вопрос 3.** Что вернёт `s?.Length`, если `s == null`?

- A) 0.
- B) null.
- C) Ошибку.

✅ **Верно: B.**
Почему: `?.` безопасно возвращает null, не падая.
Почему неверны остальные:

- A — Не 0.
- C — Ошибки нет.

**Вопрос 4.** Что делает `string name = input ?? "Гость";`?

- A) Всегда присваивает «Гость».
- B) Если input == null, присваивает «Гость»; иначе — значение input.
- C) Бросает ошибку.

✅ **Верно: B.**
Почему: `??` — «если null, возьми правую часть».
Почему неверны остальные:

- A — Не «всегда».
- C — Ошибки нет.

## Итог урока

- `null` — отсутствие ссылки; обращение к нему падает.
- `?` — пометка «может быть null».
- `??` — значение по умолчанию при null.
- `?.` — безопасное обращение.
- Проверяй данные извне.

# Урок 30. Исключения и try/catch

## Цель

Понять, как программа сообщает об ошибках и как не дать ей «рухнуть».

## Словарик новых слов

- **Исключение (exception)** — специальный сигнал «что-то пошло не так». Программа «бросает» его в момент ошибки.
- **Бросить исключение** (`throw`) — создать и послать сигнал об ошибке.
- **Поймать исключение** (`catch`) — перехватить сигнал и среагировать.
- **try/catch/finally** — конструкция для «страховочной сетки».

## Теория «с нуля»

### Аналогия: будильник на кухне

Ты готовишь и вдруг — пожар (ошибка). Если ничего не делать, дом сгорит (программа упадёт). Но есть пожарная сигнализация: она «ловит» сигнал и вызывает МЧС. В коде роль сигнализации играет `try/catch`.

### Что происходит при ошибке без обработки

```
int x = int.Parse("пять");   // ошибка: текст не превращается в число
Console.WriteLine(x);        // сюда не дойдём — программа упадёт
```

Программа «упадёт» с исключением. Чтобы это предотвратить — оборачиваем в try/catch.

### try/catch — страховочная сетка

```
try
{
    int x = int.Parse("пять");
    Console.WriteLine(x);
}
catch (FormatException)
{
    Console.WriteLine("Это не число!");
}
```

- `try` — код, который может упасть.
- `catch (FormatException)` — что делать, если упал с ошибкой именно этого типа.
- После `catch` программа продолжает работу — не рушится.

### Какие бывают типы исключений

| Тип | Когда возникает |
|---|---|
| `FormatException` | неверный формат (например, «пять» в число) |
| `DivideByZeroException` | деление на ноль |
| `ArgumentNullException` | на входе null |
| `ArgumentOutOfRangeException` | значение вне диапазона |
| `FileNotFoundException` | файл не найден |
| `InvalidOperationException` | операция запрещена в текущем состоянии |

### finally — «выполнится в любом случае»

```
try
{
    // рискованный код
}
catch (Exception ex)
{
    Console.WriteLine($"Ошибка: {ex.Message}");
}
finally
{
    Console.WriteLine("Это выполнится всегда — была ошибка или нет");
}
```

`finally` — для «уборки»: закрыть файл, освободить что-то.

### Один try — несколько catch

Можно ловить разные типы отдельно:

```
try { ... }
catch (FormatException) { Console.WriteLine("Неверный формат"); }
catch (OverflowException) { Console.WriteLine("Слишком большое число"); }
```

## Примеры

```
 Console.Write("Введи число: ");
try
{
    int n = int.Parse(Console.ReadLine() ?? "");
    Console.WriteLine($"Квадрат: {n * n}");
}
catch (FormatException)
{
    Console.WriteLine("Это не число!");
}
```

## Сделай сам (задачи с подсказками)

1. Оберни `int.Parse(Console.ReadLine())` в try/catch и выведи «Это не число» при ошибке.
   *Подсказка:* *`catch (FormatException)`*.
2. Раздели два числа, обработай `DivideByZeroException`.
   *Подсказка:* *`catch (DivideByZeroException)`*.
3. Добавь `finally`, который пишет «операция завершена».
   *Подсказка: блок* *`finally { ... }`* *идёт после catch.*
4. Сделай два `catch`: для FormatException и для OverflowException.
   *Подсказка: разные блоки catch подряд.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `catch { }` пустой | ошибка молча глотается | хотя бы логируй/выводи |
| `catch (Exception ex)` первым | перехватит всё, остальные бесполезны | конкретные типы — впереди |
| Ошибка вне try | не будет поймана | оборачивай именно рискованный код |
| `throw ex;` внутри catch | теряется стек вызовов | используй голый `throw;` |

## Тест — проверь себя

**Вопрос 1.** Что такое исключение?

- A) Число.
- B) Сигнал о том, что произошла ошибка.
- C) Имя метода.

✅ **Верно: B.**
Почему: исключение — механизм сообщения об ошибке.
Почему неверны остальные:

- A — Это не число.
- C — Это не имя метода.

**Вопрос 2.** Что делает `catch`?

- A) Бросает ошибку.
- B) Перехватывает ошибку и позволяет среагировать.
- C) Останавливает программу навсегда.

✅ **Верно: B.**
Почему: catch ловит исключение и выполняет свой код.
Почему неверны остальные:

- A — Бросает — это `throw`.
- C — После catch программа обычно продолжается.

**Вопрос 3.** Для чего нужен `finally`?

- A) Выполнить код, только если не было ошибки.
- B) Выполнить код всегда — была ошибка или нет.
- C) Завершить программу.

✅ **Верно: B.**
Почему: finally гарантированно выполняется.
Почему неверны остальные:

- A — Это было бы частью try без ошибки.
- C — finally не завершает программу.

**Вопрос 4.** Какой результат `int.Parse("пять")` без try/catch?

- A) Вернёт 5.
- B) Бросит исключение (FormatException), программа упадёт.
- C) Вернёт 0.

✅ **Верно: B.**
Почему: «пять» не превращается в число — возникает FormatException.
Почему неверны остальные:

- A — C# не понимает русские названия чисел «из коробки».
- C — 0 не вернётся, упадёт исключение.

## Итог урока

- Исключение — сигнал об ошибке, «бросается» при сбое.
- `try/catch` — перехват и реакция.
- `finally` — «уборка» в любом случае.
- Лови конкретные типы, не глотай ошибки молча.

# Урок 31. Свои исключения

## Цель

Научиться бросать исключения самому и создавать свои типы ошибок.

## Словарик новых слов

- **throw** — оператор: «бросить исключение».
- **Своё исключение** — класс, унаследованный от `Exception`, для особой ошибки твоей программы.
- **Конструктор** — метод, вызываемый при создании объекта (из урока 22).

## Теория «с нуля»

### Аналогия: часовой

Ты — часовой на проходной. Если кто-то пытается войти без пропуска, ты не должен молча пропускать. Ты «бросаешь сигнал» — поднимаешь тревогу (исключение). А дежурный (catch) реагирует.

### Почему бросать самому

Метод должен сообщать, если входные данные плохие, а не молча делать неправильную работу:

```
 void SetAge(int age)
{
    if (age <= 0)
        throw new ArgumentOutOfRangeException(nameof(age), "Возраст должен быть
положительным");

    // дальше работаем с корректным возрастом
}
```

`throw new ...` — создать исключение и бросить. После `throw` выполнение метода прекращается.

### Стандартные типы — когда какой

- `ArgumentException` — аргумент неверный в целом.
- `ArgumentNullException` — передан `null`.

- `ArgumentOutOfRangeException` — значение вне диапазона.
- `InvalidOperationException` — действие запрещено текущим состоянием.

### Своё исключение

Иногда стандартных мало — нужна «своя» ошибка со смыслом:

```
class InvalidEmailException : Exception
{
    public string Email { get; }

    public InvalidEmailException(string email)
        : base($"Некорректный email: {email}")
    {
        Email = email;
    }
}
```

- Наследуемся от `Exception`.
- Имя заканчивается на `Exception`.
- В конструкторе передаём сообщение в базовый (`base`).
- Можно добавить свои свойства (например, `Email`).

Использование:

```
void ValidateEmail(string email)
{
    if (!email.Contains('@'))
        throw new InvalidEmailException(email);
}
```

### Перехват своего исключения

```
try
{
    ValidateEmail("не-почта");
}
catch (InvalidEmailException ex)
{
    Console.WriteLine(ex.Message);
}
```

## Примеры

```
 class AccountNotFound : Exception
{
    public AccountNotFound(int id) : base($"Счёт {id} не найден") { }
}

void FindAccount(int id)
{
    if (id != 42) throw new AccountNotFound(id);
}
```

## Сделай сам (задачи с подсказками)

1. Напиши метод `Withdraw`, бросающий `InvalidOperationException`, если денег не хватает.
   *Подсказка:* `if (amount > balance) throw new InvalidOperationException("Недостаточно средств");`
2. Создай своё исключение `InvalidPasswordException` и брось его при коротком пароле.
   *Подсказка: класс-наследник* `Exception`*, сообщение в* `base`.
3. Создай `InvalidEmailException` с полем `Email`. Поймай его через catch.
   *Подсказка: в catch обратись к* `ex.Email`.
4. Добавь `nameof` в сообщения стандартных исключений.
   *Подсказка:* `throw new ArgumentException("...", nameof(age));`

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `throw new Exception("...")` | слишком общий тип | используй конкретный или свой |
| Имя своего исключения без `Exception` | нарушает конвенцию | `...Exception` |
| `throw` после выполняющейся работы | «бросаем» слишком поздно | проверяй в начале метода |
| Забыл `nameof` в сообщении | сообщение без имени параметра | добавляй `nameof(...)` |

## Тест — проверь себя

**Вопрос 1.** Что делает оператор `throw`?

- A) Ловит ошибку.
- B) Бросает исключение, прекращая выполнение метода.
- C) Выводит ошибку на экран.

✅ **Верно: B.**
Почему: `throw` создаёт и «бросает» исключение.
Почему неверны остальные:

- A — Ловит — это `catch`.
- C — Вывод на экран — не задача throw.

**Вопрос 2.** Каким должно быть имя своего исключения?

- A) Любым.
- B) Заканчиваться на `Exception`.
- C) Начинаться с `I`.

✅ **Верно: B.**
Почему: это стандарт именования в .NET.
Почему неверны остальные:

- A — Есть конвенция.
- C — `I` — для интерфейсов.

**Вопрос 3.** От чего наследуется своё исключение?

- A) От `object`.
- B) От `Exception` (или его наследника).
- C) От `String`.

✅ **Верно: B.**
Почему: чтобы быть настоящим исключением и ловиться через catch.
Почему неверны остальные:

- A — `object` — корень всех, но не даёт поведения исключения.
- C — Не от строки.

**Вопрос 4.** Что произойдёт после `throw new InvalidPasswordException(...)` в методе?

- A) Метод продолжит выполняться.
- B) Выполнение метода прекратится, исключение уйдёт наверх.
- C) Программа выведет пароль.

✅ **Верно: B.**
Почему: `throw` немедленно останавливает метод.
Почему неверны остальные:

- A — Код после throw не выполнится.
- C — Вывод пароля не происходит.

## Итог урока

- `throw` — бросаем исключение, метод останавливается.
- Стандартные типы — под конкретные ситуации.
- Своё исключение = класс-наследник `Exception` с именем `...Exception`.
- Сообщение передаём в `base`.

# Урок 32. Файлы

## Цель

Научиться читать и записывать файлы.

## Словарик новых слов

- **Путь** — адрес файла на диске (например, `data.txt`).
- `File` — класс-помощник для простых операций с файлами.
- `File.ReadAllText` — прочитать весь файл как текст.
- `File.WriteAllText` — записать текст в файл (перезаписав его).
- `File.AppendAllText` — дописать текст в конец файла.

## Теория «с нуля»

### Аналогия: блокнот и папка

Файл — это лист в блокноте. `ReadAllText` — прочитать весь лист. `WriteAllText` — стереть и написать заново. `AppendAllText` — дописать ещё строку снизу, не стирая ранее написанное.

### Запись файла

```
File.WriteAllText("note.txt", "Привет, файл!");
```

Создаст файл `note.txt` с текстом. Если файл уже был — он будет перезаписан.

### Дописывание

```
File.AppendAllText("log.txt", "Новая запись" + Environment.NewLine);
```

`Environment.NewLine` — «перевод строки» (перенос). Так каждая запись — с новой строки.

### Чтение

```
 string content = File.ReadAllText("note.txt");
Console.WriteLine(content);
```

### Проверка существования

```
if (File.Exists("note.txt"))
{
    Console.WriteLine(File.ReadAllText("note.txt"));
}
else
{
    Console.WriteLine("Файла нет");
}
```

### Чтение по строкам

```
string[] lines = File.ReadAllLines("notes.txt");
foreach (string line in lines)
{
    Console.WriteLine(line);
}
```

### Пути

Вместо зашитых путей используют `Path.Combine`:

```
string path = Path.Combine(Environment.CurrentDirectory, "data", "notes.txt");
```

Это соберёт корректный путь для твоей операционной системы.

## Примеры

**Логгер (запись с датой):**

```
string line = $"[{DateTime.Now:HH:mm:ss}] Пользователь вошёл";
File.AppendAllText("log.txt", line + Environment.NewLine);
```

## Сделай сам (задачи с подсказками)

1. Запиши «Привет» в `hello.txt` и прочитай его обратно.
   *Подсказка:* *`WriteAllText`* *затем* *`ReadAllText`*.
2. Допиши три строки в `log.txt` через `AppendAllText`.
   *Подсказка: не забывай* *`Environment.NewLine`*.
3. Если файл существует — выведи его содержимое, иначе создай с текстом «пусто».
   *Подсказка:* *`File.Exists`*.
4. Прочитай файл по строкам и выведи количество строк.
   *Подсказка:* *`File.ReadAllLines(...).Length`*.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Чтение несуществующего файла | `FileNotFoundException` | проверяй `File.Exists` |
| Указан только `WriteAllText` и ждёшь, что допишется | он перезаписывает | для дописывания — `AppendAllText` |
| Путь «C:\Users\...» зашит жёстко | не работает на другой машине | `Path.Combine` |
| Строки без переносов | всё сливается в одну строку | `Environment.NewLine` |

## Тест — проверь себя

**Вопрос 1.** Что делает `File.WriteAllText("a.txt", "x")`?

- A) Дописывает «x» в конец.
- B) Записывает «x», перезаписав файл.
- C) Читает файл.

✅ **Верно: B.**
Почему: WriteAllText перезаписывает содержимое.
Почему неверны остальные:

- A — Дописывание — это `AppendAllText`.
- C — Чтение — это `ReadAllText`.

**Вопрос 2.** Что будет при `File.ReadAllText("нет.txt")`, если файла нет?

- A) Вернётся пустая строка.
- B) Будет брошено исключение (`FileNotFoundException`).
- C) Файл создастся автоматически.

✅ **Верно: B.**
Почему: чтение отсутствующего файла — ошибка.
Почему неверны остальные:

- A — Пустой строки не будет, будет исключение.
- C — Чтение не создаёт файл.

**Вопрос 3.** Зачем `Path.Combine`?

- A) Чтобы объединить части пути корректно для любой ОС.
- B) Чтобы прочитать файл.
- C) Чтобы удалить файл.

✅ **Верно: A.**
Почему: `Path.Combine` собирает путь безопасно.
Почему неверны остальные:

- B — Чтение — это `ReadAllText`.
- C — Удаление — это `File.Delete`.

**Вопрос 4.** Как дописать строку, не стирая прежнее содержимое?

- A) `File.WriteAllText`
- B) `File.AppendAllText`
- C) `File.ReadAllText`

✅ **Верно: B.**
Почему: `AppendAllText` добавляет в конец.
Почему неверны остальные:

- A — WriteAllText перезаписывает.
- C — ReadAllText читает.

## Итог урока

- `WriteAllText` — запись (перезапись), `AppendAllText` — дописывание.
- `ReadAllText` / `ReadAllLines` — чтение.
- `File.Exists` — проверка.
- `Path.Combine` — правильные пути.

# Урок 33. JSON

## Цель

Понять, как превращать объекты в текст (для сохранения/передачи) и обратно.

## Словарик новых слов

- **JSON** — текстовый формат для хранения данных. Читается и людьми, и программами.
- **Сериализация** — превращение объекта в текст (JSON).
- **Десериализация** — превращение текста (JSON) обратно в объект.
- `JsonSerializer` — встроенный в .NET помощник для этого.

## Теория «с нуля»

### Аналогия: анкета

Анкета в бумажном виде — это «JSON»: набор полей и значений. Ты заполняешь анкету (объект), потом её можно отправить по факсу (сериализация в текст), а получатель расшифрует (десериализация) и снова увидит аккуратные поля.

### Как выглядит JSON

```
{
  "name": "Иван",
  "age": 30
}
```

### Сериализация объекта в JSON

```
record Person(string Name, int Age);

Person p = new Person("Иван", 30);
string json = JsonSerializer.Serialize(p);
Console.WriteLine(json);   // {"Name":"Иван","Age":30}
```

- `record` — компактный тип для данных (из урока про продвинутые типы).
- `JsonSerializer.Serialize(obj)` — объект → строка.

### Десериализация JSON в объект

```
string json = "{\"Name\":\"Иван\",\"Age\":30}";
Person? p = JsonSerializer.Deserialize<Person>(json);
Console.WriteLine(p!.Name);   // Иван
```

- `Deserialize<Person>(json)` — строка → объект типа Person.
- `?` и `!` — про возможность null (модуль 04).

### Список объектов

```
List<Person> people = new List<Person> { new("Иван", 30), new("Мария", 28) };
string json = JsonSerializer.Serialize(people);          // в строку
List<Person>? back = JsonSerializer.Deserialize<List<Person>>(json);
```

### Красивый вывод

```
string pretty = JsonSerializer.Serialize(p, new JsonSerializerOptions { WriteIndented = true
});
Console.WriteLine(pretty);
```

`WriteIndented = true` — делает JSON читаемым («с отступами»).

### JSON + файлы (связка с уроком 32)

```
// сохранить объект в файл
File.WriteAllText("person.json", JsonSerializer.Serialize(p));

// прочитать из файла
Person? loaded = JsonSerializer.Deserialize<Person>(File.ReadAllText("person.json"));
```

## Примеры

```
record Product(string Name, decimal Price);

Product phone = new Product("Phone", 499.99m);
File.WriteAllText("product.json", JsonSerializer.Serialize(phone, new JsonSerializerOptions {
WriteIndented = true }));

Product? loaded = JsonSerializer.Deserialize<Product>(File.ReadAllText("product.json"));
Console.WriteLine(loaded!.Name);
```

## Сделай сам (задачи с подсказками)

1. Создай `record Student(string Name, int Age)` и сериализуй объект в строку.
   *Подсказка:* `JsonSerializer.Serialize(student)`.
2. Десериализуй строку `{"Name":"Аня","Age":20}` обратно в Student.
   *Подсказка:* `JsonSerializer.Deserialize<Student>(json)`.
3. Сохрани список студентов в файл и прочитай обратно.
   *Подсказка:* `WriteAllText` + `Deserialize<List<Student>>`.
4. Выведи JSON красиво (с отступами).
   *Подсказка:* `WriteIndented = true`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `Deserialize<Person>` без проверки на null | может вернуть null | используй `?` и проверяй |
| JSON без кавычек у полей | невалидный JSON | поля в кавычках |
| Забыл `using System.Text.Json;` | JsonSerializer не найден | добавь using |
| Чтение несуществующего файла JSON | исключение | проверяй File.Exists |

## Тест — проверь себя

**Вопрос 1.** Что такое сериализация?

- A) Превращение объекта в текст (JSON).
- B) Превращение текста в объект.
- C) Удаление объекта.

✅ **Верно: A.**
Почему: сериализация — объект → текст.
Почему неверны остальные:

- B — Это десериализация.
- C — Это не удаление.

**Вопрос 2.** Что делает `JsonSerializer.Deserialize<Person>(json)`?

- A) Превращает Person в строку.
- B) Превращает строку JSON в объект Person.
- C) Печатает JSON.

✅ **Верно: B.**
Почему: Deserialize — текст → объект.
Почему неверны остальные:

- A — Это Serialize.
- C — Печать — не задача Deserialize.

**Вопрос 3.** Зачем `WriteIndented = true`?

- A) Чтобы JSON стал меньше.
- B) Чтобы JSON был читаемым, с отступами.
- C) Чтобы JSON ускорился.

✅ **Верно: B.**
Почему: отступы делают JSON наглядным.
Почему неверны остальные:

- A — Отступы, наоборот, увеличивают размер, а не уменьшают.
- C — Это не про скорость.

**Вопрос 4.** Как сохранить объект в файл как JSON?

- A) `File.WriteAllText("p.json", JsonSerializer.Serialize(p));`
- B) `File.ReadAllText("p.json");`
- C) `JsonSerializer.Deserialize<Person>(json);`

✅ **Верно: A.**
Почему: сериализуем и записываем текст в файл.
Почему неверны остальные:

- B — ReadAllText — чтение.
- C — Deserialize — разбор строки, а не запись файла.

## Итог урока

- JSON — текстовый формат данных.
- `Serialize` — объект → JSON; `Deserialize` — JSON → объект.
- `WriteIndented` — красивый вывод.
- JSON легко сочетается с файлами.

# Урок 34. LINQ

## Цель

Научиться красиво и коротко «спрашивать» коллекции вместо длинных циклов.

## Словарик новых слов

- **LINQ** — встроенный язык запросов к коллекциям.
- `Where` — «отфильтруй» (оставь подходящих).
- `Select` — «возьми только нужное поле» (проекция).
- `OrderBy` — «отсортируй».
- `Take` — «возьми первые N».
- `ToList()` — превратить результат в список.

## Теория «с нуля»

### Аналогия: вопрос к очереди

Представь список людей. Ты можешь задать вопросы:

- «Дай всех, кому ≥ 18» → `Where(p => p.Age >= 18)`.
- «Дай только их имена» → `Select(p => p.Name)`.
- «Отсортируй по имени» → `OrderBy(p => p.Name)`.
- «Возьми первых трёх» → `Take(3)`.

LINQ — это такие же «вопросы», только в коде.

### Цепочка вопросов

```
List<Person> adults = people
    .Where(p => p.Age >= 18)
    .OrderBy(p => p.Name)
    .ToList();
```

Читается как предложение: «возьми людей, отфильтруй >= 18, отсортируй по имени, преврати в список».

### Где взять данные

```
record Person(string Name, int Age);

List<Person> people = new()
{
    new("Иван", 30),
    new("Аня", 17),
    new("Мария", 25)
};
```

### Самые частые операторы

| Оператор | Что делает | Пример |
|---|---|---|
| `Where(...)` | фильтр | `.Where(p => p.Age >= 18)` |
| `Select(...)` | проекция | `.Select(p => p.Name)` |
| `OrderBy(...)` | сортировка | `.OrderBy(p => p.Name)` |
| `Take(n)` | первые n | `.Take(3)` |
| `Skip(n)` | пропустить n | `.Skip(2)` |
| `FirstOrDefault()` | первый или null | `.FirstOrDefault()` |
| `Count()` | количество | `.Count()` |
| `Any(...)` | есть ли хоть один | `.Any(p => p.Age > 100)` |
| `Sum(...)` | сумма | `.Sum(p => p.Age)` |

### Лямбда `p => ...`

Запись `p => p.Age` — «для каждого p — верни p.Age». Это стрелочная функция (из урока про делегаты, можно просто воспринимать как «что взять от элемента»).

### Почему LINQ удобен

- Короче и читаемее циклов.
- Операторы можно соединять цепочкой.
- Легче понять намерение: «фильтр», «сортировка».

## Примеры

**Имена взрослых по алфавиту:**

```
List<string> names = people
    .Where(p => p.Age >= 18)
    .OrderBy(p => p.Name)
```

```
    .Select(p => p.Name)
    .ToList();
```

**Средний возраст:**

```
double avg = people.Average(p => p.Age);
```

## Сделай сам (задачи с подсказками)

1. Из списка чисел `{ 5, 12, 7, 20 }` выбери больше 10.
   *Подсказка:* `Where(n => n > 10)`.
2. Отсортируй по убыванию.
   *Подсказка:* `OrderByDescending(n => n)`.
3. Возьми первые 2 числа.
   *Подсказка:* `Take(2)`.
4. Из людей выбери имена тех, кому ≥ 18, по алфавиту.
   *Подсказка:* `Where` → `OrderBy` → `Select` → `ToList`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Забыл `ToList()` и перебираешь несколько раз | запрос пересчитывается | материализуй `ToList()` |
| `First()` на пустом | исключение | `FirstOrDefault()` |
| `Select` в цикле вместо `SelectMany` | вложенные списки | `SelectMany` (для списков списков) |
| `Where(...).ToList()` без нужды | лишняя аллокация | оставляй ленивым при однократном переборе |

## Тест — проверь себя

**Вопрос 1.** Что делает `.Where(p => p.Age >= 18)`?

- A) Сортирует.
- B) Оставляет только тех, у кого Age ≥ 18.
- C) Удаляет всех.

✅ **Верно: B.**
Почему: Where — фильтр по условию.
Почему неверны остальные:

- A — Сортировка — это OrderBy.

- C — Фильтр не удаляет всех.

**Вопрос 2.** Что вернёт `.Select(p => p.Name)`?

- A) Объекты Person.
- B) Только имена (строки).
- C) Числа.

✅ **Верно: B.**
Почему: Select «берёт» поле Name.
Почему неверны остальные:

- A — Это, не проекция.
- C — Если только имена не числа, но здесь они строки.

**Вопрос 3.** Зачем `.ToList()` в конце цепочки?

- A) Чтобы красиво вывести.
- B) Чтобы «материализовать» результат в список (выполнить запрос).
- C) Чтобы отсортировать.

✅ **Верно: B.**
Почему: без ToList запрос «ленивый» и выполняется при переборе; ToList фиксирует результат.
Почему неверны остальные:

- A — Это не про вывод.
- C — Сортировка — это OrderBy.

**Вопрос 4.** Как найти первый элемент, или null, если никого нет?

- A) `.First()`
- B) `.FirstOrDefault()`
- C) `.Take(1)`

✅ **Верно: B.**
Почему: FirstOrDefault возвращает первый или default/null без исключения.
Почему неверны остальные:

- A — First на пустом бросит исключение.
- C — Take(1) вернёт коллекцию из одного элемента, а не сам элемент.

## Итог урока

- LINQ — «вопросы» к коллекциям: Where, Select, OrderBy, Take...
- Цепочки операторов читаются как предложение.

- `ToList()` — материализация результата.
- `FirstOrDefault()` — безопасное взятие первого.

# Урок 35. Асинхронность

## Цель

Понять, как не «замораживать» программу при долгих операциях (сеть, файлы, БД).

## Словарик новых слов

- **Асинхронность** — выполнение долгой операции, не блокируя остальное.
- `Task` — «обещание» будущего результата.
- `async` — пометка метода, что он асинхронный.
- `await` — «дождись результата, но не блокируй».
- `CancellationToken` — сигнал об отмене.

## Теория «с нуля»

### Аналогия: заказ пиццы

Ты заказал пиццу. Если ты будешь **блокирующе** ждать у телефона — ты не сможешь делать другие дела. Асинхронность — как «заказал и занимаешься своим делом; когда пицца готова — тебе позвонят». `await` — «жди звонка, но продолжай жить».

### Синхронно (блокирует)

```
string data = File.ReadAllText("big.txt");   // пока читаем — программа «стоит»
```

### Асинхронно

```
async Task<string> ReadFileAsync()
{
    string data = await File.ReadAllTextAsync("big.txt");
    return data;
}
```

- `async Task<string>` — метод асинхронный, вернёт строку.
- `await` — приостанавливает метод, НЕ блокируя поток.

### Что такое Task

`Task` — «обещание»: работа идёт, а результат будет. `Task<string>` — результат-строка.

### await и результат

```
string result = await ReadFileAsync();
```

`await` «разворачивает» `Task<string>` в строку.

### async Task vs async void

- `async Task` — правильно (можно await, ошибки ловятся).
- `async void` — только для обработчиков событий; в остальных случаях опасно.

### Отмена (CancellationToken)

```
async Task ProcessAsync(CancellationToken token)
{
    for (int i = 0; i < 100; i++)
    {
        token.ThrowIfCancellationRequested();   // проверить отмену
        await Task.Delay(100, token);
    }
}
```

### Сколько задач параллельно: Task.WhenAll

```
Task<int> t1 = FetchAsync(1);
Task<int> t2 = FetchAsync(2);
int[] results = await Task.WhenAll(t1, t2);
```

## Примеры

```
async Task<int> FetchAsync(int id)
{
    await Task.Delay(300);          // имитация задержки
    return id * 10;
}

// в Main:
int result = await FetchAsync(1);
Console.WriteLine(result);          // 10
```

## Сделай сам (задачи с подсказками)

1. Напиши `async Task<string> GreetAsync()`: `await Task.Delay(200);` затем возврат строки.
   *Подсказка:* `return "Привет";` *после await.*
2. Вызови его через `await` и выведи результат.
   *Подсказка:* `var s = await GreetAsync(); Console.WriteLine(s);`
3. Запусти два async-метода параллельно через `Task.WhenAll`.
   *Подсказка:* `await Task.WhenAll(t1, t2)`.
4. Добавь `CancellationToken` и отмену через `CancelAfter`.
   *Подсказка:* `using var cts = new CancellationTokenSource(); cts.CancelAfter(100);`

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `.Result` / `.Wait()` вместо await | блокировка, риск дедлока | `await` |
| `async void` не в обработчике | ошибки теряются | `async Task` |
| Метод `async` без `await` | предупреждение CS1998 | убери async или добавь await |
| Забыл `await` перед Task | получишь Task, а не результат | добавь `await` |

## Тест — проверь себя

**Вопрос 1.** Что делает `await`?

- A) Блокирует программу до результата.
- B) Приостанавливает метод, не блокируя поток, и дожидается результата.
- C) Ускоряет код.

✅ **Верно: B.**
Почему: await освобождает поток, ожидая результат.
Почему неверны остальные:

- A — Блокировка — это `.Result` / `.Wait()`.
- C — Это не про ускорение, а про неблокирование.

**Вопрос 2.** Что такое `Task<T>`?

- A) Число.
- B) «Обещание» будущего результата типа T.
- C) Массив.

✅ **Верно: B.**
Почему: Task — представление асинхронной работы.
Почему неверны остальные:

- A, C — Это не число и не массив.

**Вопрос 3.** Почему `async void` опасен вне обработчиков событий?

- A) Он медленнее.
- B) Ошибки в нём теряются, его нельзя await.
- C) Он всегда блокирует.

✅ **Верно: B.**
Почему: async void не возвращает Task, поэтому нельзя дождаться и перехватить ошибку.
Почему неверны остальные:

- A — Это не про скорость.
- C — Он не «всегда блокирует».

**Вопрос 4.** Что делает `Task.WhenAll(t1, t2)`?

- A) Ждёт первый завершившийся.
- B) Ждёт все переданные задачи.
- C) Отменяет задачи.

✅ **Верно: B.**
Почему: WhenAll ждёт завершения всех.
Почему неверны остальные:

- A — Первый завершившийся — это `Task.WhenAny`.
- C — Отмена — это CancellationToken.

## Итог урока

- Асинхронность — не блокировать поток при долгих операциях.
- `async Task` + `await` — правильный паттерн.
- `Task.WhenAll` — ждать несколько задач.
- `CancellationToken` — отмена.
- Не используй `.Result` / `async void` без нужды.

# Урок 36. Юнит-тесты

## Цель

Понять, как автоматически проверять, что код работает, и не ломать старое при изменениях.

## Словарик новых слов

- **Тест** — маленькая программа, которая проверяет, что метод ведёт себя ожидаемо.
- **Юнит-тест** — тест одного «кирпичика» (метода/класса) в изоляции.
- **xUnit** — популярный фреймворк для тестов в .NET.
- `[Fact]` — пометка одиночного теста.
- `[Theory]` / `[InlineData]` — тест с несколькими наборами данных.
- **Assert** — «утверждение»: проверка ожидания.

## Теория «с нуля»

### Аналогия: проверка перед ремонтом

Прежде чем менять проводку, ты проверяешь, что лампочки горели. Тесты — то же самое: перед изменениями мы убеждаемся, что старое поведение не сломалось.

### Как выглядит тест

```
public class Calculator
{
    public int Add(int a, int b) => a + b;
}
```

Тест:

```
public class CalculatorTests
{
    [Fact]
    public void Add_TwoNumbers_ReturnsSum()
    {
        // Arrange (подготовка)
        var calc = new Calculator();

        // Act (действие)
        int result = calc.Add(2, 3);
```

```
        // Assert (проверка)
        Assert.Equal(5, result);
    }
}
```

Три части (AAA): Arrange → Act → Assert.

### Утверждения Assert

| Метод | Проверяет |
|---|---|
| `Assert.Equal(ожидаемо, фактически)` | равенство |
| `Assert.NotEqual(...)` | неравенство |
| `Assert.True(...)` / `Assert.False(...)` | истинность |
| `Assert.Null(...)` / `Assert.NotNull(...)` | null |
| `Assert.Throws<T>(() => ...)` | ожидаемое исключение |

### Theory с несколькими случаями

```
[Theory]
[InlineData(1, 2, 3)]
[InlineData(-1, 1, 0)]
public void Add_ReturnsSum(int a, int b, int expected)
{
    Assert.Equal(expected, new Calculator().Add(a, b));
}
```

`[Theory]` — тест запускается для каждого набора `[InlineData]`.

### Зачем это нужно

- Страховка при правках (регрессии).
- Документация поведения.
- Можно рефакторить без страха.

### Запуск тестов

```
dotnet test
```

## Примеры

```
 public class PasswordValidator
{
    public static bool IsValid(string? password) =>
        password is not null && password.Length >= 8;
}

public class PasswordValidatorTests
{
    [Fact]
    public void IsValid_ShortPassword_ReturnsFalse()
    {
        Assert.False(PasswordValidator.IsValid("abc"));
    }

    [Fact]
    public void IsValid_LongPassword_ReturnsTrue()
    {
        Assert.True(PasswordValidator.IsValid("abcdefgh"));
    }
}
```

## Сделай сам (задачи с подсказками)

1. Напиши тест для `Add(2, 3)` через `Assert.Equal(5, ...)`.
   *Подсказка: AAA + `Assert.Equal`.*
2. Напиши `[Theory]` с `[InlineData]` для нескольких случаев.
   *Подсказка: параметры метода соответствуют значениям InlineData.*
3. Проверь, что `Divide` бросает `DivideByZeroException`.
   *Подсказка: `Assert.Throws<DivideByZeroException>(() => ...)`.*
4. Напиши тест на `IsValid(null)` → false.
   *Подсказка: `Assert.False(...)`.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Тест без `Assert` | ничего не проверяет | всегда завершай Assert-ом |
| Один тест со множеством актов | труднее локализовать | дроби на отдельные тесты |
| Тест зависит от `DateTime.Now` | плавает | передавай время параметром |
| Тест-«дубликат» | лишний прогон | один тест на сценарий |

## Тест — проверь себя

**Вопрос 1.** Что делает `Assert.Equal(5, result)`?

- A) Проверяет, что result равен 5.
- B) Присваивает result = 5.
- C) Выводит 5.

✅ **Верно: A.**
Почему: Assert.Equal — проверка равенства.
Почему неверны остальные:

- B — Это не присваивание.
- C — Это не вывод.

**Вопрос 2.** Что значит `[Fact]`?

- A) Тест всегда падает.
- B) Обычный одиночный тест.
- C) Тест с параметрами.

✅ **Верно: B.**
Почему: [Fact] — самый простой тест.
Почему неверны остальные:

- A — Он не обязан падать.
- C — Параметры — это [Theory].

**Вопрос 3.** Чем `[Theory]` отличается от `[Fact]`?

- A) Theory запускается с несколькими наборами данных ([InlineData]).
- B) Theory не имеет Assert.
- C) Theory запускается один раз.

✅ **Верно: A.**
Почему: Theory + InlineData — много случаев в одном тесте.
Почему неверны остальные:

- B — Assert там есть.
- C — Запускается несколько раз, по одному на InlineData.

**Вопрос 4.** Зачем тесты?

- A) Чтобы программа была длиннее.
- B) Чтобы ловить поломки при изменениях.
- C) Чтобы скрыть код.

✅ **Верно: B.**
Почему: тесты — страховка от регрессий.
Почему неверны остальные:

- A — Не для длины.
- C — Не для скрытия.

## Итог урока

- Юнит-тест проверяет один метод.
- AAA: Arrange → Act → Assert.
- `Assert.*` — утверждения.
- `[Fact]` — одиночный, `[Theory]` + `[InlineData]` — с данными.
- `dotnet test` — запуск.

## Итог уровня 3

Ты научился:

- обрабатывать ошибки (try/catch, свои исключения);
- работать с файлами и JSON;
- писать красивые запросы LINQ;
- не блокировать программу (async/await);
- защищать код тестами.

Поздравляю — **Уровень 3 пройден!** Дальше — Уровень 4 (Senior): SOLID, архитектура, безопасность, наблюдаемость.

# Урок 37. Экскурсия по стандартной библиотеке

## Цель

Познакомиться с часто используемыми готовыми помощниками .NET: `string`, `Math`, `Random`, `DateTime`, `Guid`.

## Словарик новых слов

- **BCL (Base Class Library)** — стандартная библиотека классов .NET. В ней — сотни готовых инструментов.
- **Класс-помощник** — класс со статическими методами (например, `Math`).

## Теория «с нуля»

### Зачем это нужно

Не нужно писать всё самому. Многие задачи уже решены за нас — важно знать, что они существуют.

### string — работа с текстом

```
 string s = "Привет, мир!";
Console.WriteLine(s.Length);         // 13
Console.WriteLine(s.ToUpper());      // ПРИВЕТ, МИР!
Console.WriteLine(s.ToLower());      // привет, мир!
Console.WriteLine(s.Trim());         // убирает пробелы с краёв
Console.WriteLine(s.Replace("мир", "C#")); // Привет, C#!
string[] parts = s.Split(", ");      // ["Привет", "мир!"]
Console.WriteLine(s.StartsWith("Привет")); // True
```

### Math — математика

```
 Console.WriteLine(Math.Max(3, 9));    // 9
Console.WriteLine(Math.Min(3, 9));    // 3
Console.WriteLine(Math.Abs(-5));      // 5
Console.WriteLine(Math.Pow(2, 10));   // 1024
Console.WriteLine(Math.Sqrt(81));     // 9
Console.WriteLine(Math.Round(3.14159, 2)); // 3.14
```

### Random — случайные числа

```
Random rnd = new Random();
Console.WriteLine(rnd.Next(1, 101));   // целое от 1 до 100
Console.WriteLine(rnd.NextDouble());   // дробное от 0 до 1
```

`Random.Shared` — один общий генератор (удобно): `Random.Shared.Next(1, 101)`.

### DateTime — даты и время

```
DateTime now = DateTime.Now;        // текущее (локальное)
DateTime utc = DateTime.UtcNow;     // текущее (UTC — для логики)
Console.WriteLine(now.ToString("dd.MM.yyyy"));
TimeSpan days = DateTime.Today - new DateTime(2000, 5, 17);
Console.WriteLine(days.Days);       // сколько дней прошло
```

### Guid — уникальные идентификаторы

```
Guid id = Guid.NewGuid();
Console.WriteLine(id);   // например, 3f2a...-...-...
```

`Guid` — почти гарантированно уникальный идентификатор (для id записей).

## Примеры

```
// Случайный пароль из 8 символов (маленькая демонстрация)
Random rnd = Random.Shared;
string chars = "abcdefghijklmnopqrstuvwxyz0123456789";
string password = "";
for (int i = 0; i < 8; i++) password += chars[rnd.Next(chars.Length)];
Console.WriteLine(password);
```

## Сделай сам (задачи с подсказками)

1. Выведи длину, верхний регистр и слово по индексу для `"C# — это круто"`.
   *Подсказка:* `Split(' ')` → `parts[0]`.
2. Посчитай `2^10` и `√(144)` через Math.
   *Подсказка:* `Math.Pow(2,10)`, `Math.Sqrt(144)`.
3. Выведи 3 случайных числа от 1 до 6.
   *Подсказка:* `Random.Shared.Next(1,7)` *в цикле.*

4. Выведи текущую дату в формате «дд.мм.гггг».
   *Подсказка:* `DateTime.Today.ToString("dd.MM.yyyy")`.
5. Создай случайный Guid и выведи его.
   *Подсказка:* `Guid.NewGuid()`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| `rnd.Next(1, 101)` и ожидание 101 | верхняя граница НЕ включается | `Next(1, 100)` для 1..100 |
| `DateTime.Now` в логике | локальное время, лучше Utc | `DateTime.UtcNow` |
| `Split(" ")` с несколькими пробелами | пустые элементы | `Split(' ', StringSplitOptions.RemoveEmptyEntries)` |
| Складывать строки в цикле без нужды | медленно | `StringBuilder` /join |

## Тест — проверь себя

**Вопрос 1.** Что вернёт `Math.Max(4, 7)`?

- A) 4
- B) 7
- C) 11

✅ **Верно: B.**
Почему: Max возвращает большее.
Почему неверны остальные:

- A — 4 меньше.
- C — 11 — это сумма.

**Вопрос 2.** `Random.Shared.Next(1, 7)` даёт числа:

- A) от 1 до 7 включительно.
- B) от 1 до 6 включительно.
- C) от 0 до 6.

✅ **Верно: B.**
Почему: верхняя граница не включается → 1..6.
Почему неверны остальные:

- A — 7 не включается.

- C — нижняя граница 1, не 0.

**Вопрос 3.** Что делает `s.Trim()`?

- A) Делит строку.
- B) Убирает пробелы с краёв.
- C) Делает строку заглавной.

✅ **Верно: B.**
Почему: Trim обрезает пробелы по краям.
Почему неверны остальные:

- A — Деление — это Split.
- C — Заглавные — это ToUpper.

**Вопрос 4.** Для уникального идентификатора лучше всего использовать:

- A) `Math.Max`
- B) `Guid.NewGuid()`
- C) `DateTime.Now`

✅ **Верно: B.**
Почему: Guid почти гарантированно уникален.
Почему неверны остальные:

- A — Max — это не про уникальность.
- C — Время не уникально.

## Итог урока

- `string` — методы для работы с текстом.
- `Math` — математические функции.
- `Random` — случайные числа (`Next(1, 7)` не включает 7).
- `DateTime` — даты (в логике — `UtcNow`).
- `Guid.NewGuid()` — уникальные id.

# Урок 40. SOLID · Single Responsibility (SRP)

## Цель

Понять первый и главный принцип качества кода: у каждого класса — одна работа.

## Словарик новых слов

- **SRP (Single Responsibility Principle)** — «принцип единой ответственности».
- **Ответственность** — причина, по которой класс может меняться.
- **God-класс** — класс, который делает слишком много всего.

## Теория «с нуля»

### Аналогия: рабочий-«всё-руки»

Представь рабочего, который и строит, и красит, и проводит электрику, и считает бюджет, и звонит клиентам. Если поменялись правила покраски — придётся трогать «всё», а заодно можно случайно сломать электрику. Лучше разделить: строитель, маляр, электрик, бухгалтер. У каждого — одна работа.

### Что такое «ответственность»

Если у класса есть **две причины для изменения** — значит, у него две ответственности. Например:

- «считает цену» (меняется при изменении правил ценообразования);
- «сохраняет в БД» (меняется при смене БД).

Это разные причины — класс стоит разделить.

### Плохо: God-класс

```
 class OrderProcessor
{
    public void Process(Order order)
    {
        // 1. проверить заказ
        // 2. посчитать итог
        // 3. сохранить в БД
        // 4. отправить email
    }
}
```

Всё в одном месте. Любое изменение — риск сломать остальное.

### Хорошо: раздельно

```
 class OrderValidator { public bool IsValid(Order o) => /* проверка */; }
class OrderCalculator { public decimal Calculate(Order o) => /* расчёт */; }
class OrderRepository { public void Save(Order o) => /* хранение */; }
class EmailNotifier { public void Send(Order o) => /* письмо */; }
```

У каждого класса — одна причина для изменения.

### Как понять, что SRP нарушен

Задай вопрос: «Какую одну работу делает этот класс?» Если ответ содержит «и ещё...» — пора разделять.

## Сделай сам (задачи с подсказками)

1. Найди в классе `Report` две ответственности: формирование отчёта и отправка по почте. Раздели.
   *Подсказка: `ReportBuilder` и `ReportMailer`.*
2. Класс `UserManager`: регистрация + логирование. Раздели.
   *Подсказка: вынеси логирование в отдельный `Logger`.*
3. Приведи пример своего «God-класса» и разбей его на 3 класса.
   *Подсказка: по одной обязанности на класс.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Класс из 500 строк | много ответственностей | разбить |
| Метод делает 5 дел | трудно тестировать | декомпозиция на методы/классы |
| Ставить «всё» в один класс «для простоты» | дальше будет больно | сразу разделять по смыслу |
| Прятать логику в `Program` | главный класс-«свалка» | выносить в классы |

## Тест — проверь себя

**Вопрос 1.** Что означает SRP?

- A) У класса должна быть только одна причина для изменения.
- B) У класса должно быть много ответственностей.
- C) Класс должен быть маленьким по количеству строк.

✅ **Верно: A.**
Почему: единая ответственность = одна причина меняться.
Почему неверны остальные:

- B — Много ответственностей — это нарушение SRP.
- C — SRP не про количество строк, а про смысл.

**Вопрос 2.** Почему God-класс — это плохо?

- A) Он слишком быстрый.
- B) Изменение одной части рискует сломать другие.
- C) Он не может быть унаследован.

✅ **Верно: B.**
Почему: всё связано в одном месте, правки задевают друг друга.
Почему неверны остальные:

- A — Это не про скорость.
- C — Наследование тут ни при чём.

**Вопрос 3.** Класс делает «считает налог» и «сохраняет в БД». Это:

- A) Отлично, всё в одном.
- B) Нарушение SRP: две ответственности.
- C) Нельзя так делать вообще.

✅ **Верно: B.**
Почему: две разные причины для изменения → стоит разделить.
Почему неверны остальные:

- A — Это и есть нарушение.
- C — Так делают, но это плохая практика.

**Вопрос 4.** Какое правило помогает проверить SRP?

- A) «Сколько строк в классе?»
- B) «Какую одну работу делает класс?» (если ответ: «и ещё…» — разделяй)
- C) «Сколько методов в классе?»

✅ **Верно: B.**
Почему: вопрос про одну работу отражает суть принципа.
Почему неверны остальные:

- A, C — Строки и методы — не критерий SRP.

## Итог урока

- SRP: **одна ответственность = одна причина для изменения**.
- God-класс — анти-паттерн; дроби на классы по смыслу.
- Проверка: «Какую одну работу это делает?»

# Урок 41. SOLID · Open/Closed (OCP) и Лисков (LSP)

## Цель

Понять: новое поведение добавляем, не переписывая старое (OCP), а наследники не ломают обещания базового класса (LSP).

## Словарик новых слов

- **OCP (Open/Closed)** — классы открыты для расширения, но закрыты для изменения.
- **LSP (Liskov Substitution)** — наследник не должен ломать поведение базового класса.

## Теория «с нуля»

### OCP: аналогия с розеткой

Розетка «открыта» для подключения новых приборов, но «закрыта» для изменения самой розетки. Так и код: мы хотим **добавлять** новое (новый класс), не **переписывая** существующее.

### Плохо: switch по типу

```
 double Area(Shape s) => s switch
{
    Circle c => Math.PI * c.Radius * c.Radius,
    Rectangle r => r.Width * r.Height,
    // новая фигура → приходится ПРАВИТЬ этот метод
};
```

Каждая новая фигура заставляет менять этот метод — это «не закрыто для изменения».

### Хорошо: полиморфизм

```
 abstract class Shape { public abstract double Area(); }

class Circle : Shape
{
    public double Radius { get; }
    public Circle(double r) { Radius = r; }
    public override double Area() => Math.PI * Radius * Radius;
}
```

Новая фигура = **новый класс**, существующий код не трогаем. Это «открыто для расширения».

### LSP: наследник не должен ломать

Если базовый класс обещает «площадь всегда ≥ 0», наследник не должен возвращать −5. Если можно переопределение бросает исключение там, где базовый работает, — иерархия спроектирована плохо.

Пример плохого LSP:

```
 class Rectangle { public virtual double Area() => Width * Height; }
class Square : Rectangle
{
    public override double Area() => Side * Side; // может вести себя иначе
}
```

Пример опасного наследника:

```
 class Bird { public virtual void Fly() => ...; }
class Penguin : Bird { public override void Fly() => throw new Exception("не летает"); }
```

Использование «птицы» на пингвине сломается. Лучше не делать `Penguin` наследником летающего `Bird`.

## Сделай сам (задачи с подсказками)

1. Перепиши `switch` по фигурам через полиморфизм (абстрактный класс + override).
   *Подсказка:* `abstract double Area();` + *наследники.*
2. Добавь новую фигуру `Triangle` одним новым классом, не трогая остальной код.
   *Подсказка: просто создай* `class Triangle : Shape`.
3. Найди в коде место, где наследник «ломает» обещание, и исправь (разнеси классы).
   *Подсказка: если наследник бросает исключение в методе — пересмотри иерархию.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| switch по типу в 5 местах | правки в каждом месте | полиморфизм |
| Наследник бросает исключение в override | нарушение LSP | не наследовать там, где поведение несовместимо |
| Ожидание «базовых» гарантий от наследника | нарушено | гарантии должны сохраняться |

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Переписывать рабочий класс ради нового | не закрыт для изменения | добавлять новый класс |

## Тест — проверь себя

**Вопрос 1.** Что значит «открыто для расширения, закрыто для изменения» (OCP)?

- A) Новое добавляем новыми классами, старое не переписываем.
- B) Запрещено добавлять новое.
- C) Запрещено менять класс через наследование.

✅ **Верно: A.**
Почему: расширяем, не модифицируя существующий код.
Почему неверны остальные:

- B — Открыто для расширения — значит добавлять можно.
- C — Наследование как раз и есть способ расширения.

**Вопрос 2.** Почему switch по типу фигуры — плохо?

- A) Он работает медленно.
- B) Новая фигура требует править метод.
- C) Он не компилируется.

✅ **Верно: B.**
Почему: каждое добавление ломает/меняет существующий код.
Почему неверны остальные:

- A — Это не про скорость.
- C — Компилируется, но плохо масштабируется.

**Вопрос 3.** Что такое нарушение LSP?

- A) Наследник ведёт себя несовместимо с обещанием базового класса.
- B) Наследник переопределяет метод.
- C) Наследник добавляет новый метод.

✅ **Верно: A.**
Почему: LSP требует, чтобы наследник не ломал поведение родителя.
Почему неверны остальные:

- B, C — Переопределение/добавление сами по себе не нарушают LSP.

**Вопрос 4.** `Penguin : Bird` с `override Fly()` бросающим исключение — это:

- A) Отлично.
- B) Нарушение LSP.
- C) Нельзя объявить override.

✅ **Верно: B.**
Почему: «птица» для пингвина не летает — сломан контракт.
Почему неверны остальные:

- A — Это нарушение.
- C — Объявить можно, но это плохой дизайн.

## Итог урока

- OCP: **расширяем новыми классами, не правя старое**.
- LSP: **наследник не ломает обещания базового класса**.
- Полиморфизм — главный инструмент OCP.

# Урок 42. SOLID · ISP и DIP

## Цель

Понять два финальных принципа SOLID: тонкие интерфейсы (ISP) и зависимость от абстракций (DIP).

## Словарик новых слов

- **ISP (Interface Segregation)** — клиент не должен зависеть от методов, которые не использует.
- **DIP (Dependency Inversion)** — полагайся на абстракции, а не на конкретные реализации.
- **Инъекция зависимостей (DI)** — передача зависимости извне (например, через конструктор).

## Теория «с нуля»

### ISP: аналогия с меню

Представь меню, где в каждом блюде все ингредиенты сразу. Тебе нужно только яйцо, но «блюдо» тянет всё остальное. Лучше отдельные блюда (отдельные интерфейсы) под конкретную нужду.

### Плохо: «жирный» интерфейс

```
 interface IWorker
{
    void Work();
    void Eat();
    void Sleep();
    void PrintReport();
}
```

Класс `Robot` должен реализовать и `Eat()`, и `Sleep()`, хотя не умеет. Это нарушение ISP.

### Хорошо: тонкие интерфейсы

```
 interface IWorker { void Work(); }
interface IReportable { void PrintReport(); }
```

`Robot` реализует только то, что нужно.

### DIP: аналогия — розетка и прибор

Прибор (класс) не должен сам себя подключать к конкретной розетке. Он «ожидает» питание извне. В коде: класс не создаёт свои зависимости внутри ( `new` ), а получает их через конструктор (инъекция).

### Плохо: new внутри

```
 class OrderService
{
    private readonly SqlOrderRepository _repo = new();   // жёсткая связь
}
```

Сменить хранилище сложно, тестировать без БД нельзя.

### Хорошо: инъекция через конструктор

```
 interface IOrderRepository { void Save(Order o); }

class OrderService
{
    private readonly IOrderRepository _repo;
    public OrderService(IOrderRepository repo) => _repo = repo;   // извне
}
```

Теперь можно подставить любую реализацию — и для тестов, и для смены БД. Зависимость направлена на абстракцию (интерфейс).

## Сделай сам (задачи с подсказками)

1. Разбей «жирный» интерфейс IWorker на тонкие.
   *Подсказка:* `IWorker`, `IEater`, `IReportable`.
2. Перепиши класс с `new SqlRepository()` на конструкторную инъекцию интерфейса.
   *Подсказка: создай* `IOrderRepository`*, принимай в конструкторе.*
3. Напиши тест, подставив фейк-репозиторий.
   *Подсказка: класс-фейк реализует* `IOrderRepository`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Интерфейс с 15 методами | много лишнего для клиентов | дробить |
| `new` сервиса внутри другого сервиса | жёсткая связь | инъекция через конструктор |

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Глобальный `static`-сервис | скрытая зависимость | DI |
| Реализация «вхолостую» (throw) у ненужных методов | запах ISP | токие интерфейсы |

## Тест — проверь себя

**Вопрос 1.** Что означает ISP?

- A) Клиент не должен зависеть от ненужных ему методов.
- B) Интерфейсы должны быть сложными.
- C) Наследование запрещено.

✅ **Верно: A.**
Почему: тонкие интерфейсы под нужды клиента.
Почему неверны остальные:

- B — Наоборот, тонкие.
- C — ISP не про запрет наследования.

**Вопрос 2.** Что означает DIP?

- A) Зависеть от конкретных классов.
- B) Зависеть от абстракций (интерфейсов), а не от реализаций.
- C) Запретить абстракции.

✅ **Верно: B.**
Почему: полагаемся на контракты.
Почему неверны остальные:

- A — Это противоположность DIP.
- C — Абстракции как раз нужны.

**Вопрос 3.** Почему `new SqlOrderRepository()` внутри сервиса — плохо?

- A) Это быстро.
- B) Жёсткая связь: нельзя подменить для тестов и смены БД.
- C) Это не компилируется.

✅ **Верно: B.**
Почему: конкретная реализация прибита гвоздями.
Почему неверны остальные:

- A — Это не про скорость.

- C — Компилируется, но сцепление плохое.

**Вопрос 4.** Что такое инъекция зависимостей (DI)?

- A) Создание зависимостей внутри класса.
- B) Передача зависимостей извне (например, через конструктор).
- C) Удаление зависимостей.

✅ **Верно: B.**
Почему: DI — зависимость приходит снаружи.
Почему неверны остальные:

- A — Это противоположность DI.
- C — Это не удаление.

## Итог урока

- ISP: **тонкие интерфейсы под нужды клиента**.
- DIP: **зависимость от абстракций**, реализация приходит извне (DI).
- Инъекция через конструктор — основа тестируемости.

# Урок 43. Архитектура и слои

## Цель

Понять, как организовывать большой проект: слои, направление зависимостей и структуру папок.

## Словарик новых слов

- **Архитектура** — «планировка» кода: как разделены части и как они связаны.
- **Слой** — логическая группа классов со своей ролью.
- **Domain (домен)** — бизнес-правила (ядро).
- **Infrastructure (инфраструктура)** — БД, внешние сервисы.
- **Зависимость** — ситуация, когда один класс использует другой.

## Теория «с нуля»

### Аналогия: планировка дома

Прежде чем строить дом, делают план: где кухня, где спальня, где коммуникации. Если строить хаотично — переделка одной комнаты рушит всё. Архитектура — план кода: каждому слою — своё место.

### Слои

Популярная схема (Clean Architecture, упрощённо):

```
 Presentation (API/UI)  — ввод/вывод, контроллеры
        ↓
Application (сервисы)   — use cases, бизнес-сценарии
        ↓
Domain (сущности)       — правила, модели
        ↓
Infrastructure (БД)     — EF Core, файлы, внешние вызовы
```

### Главное правило: зависимости «внутрь»

- `Domain` **не знает** про `Infrastructure` (не использует EF Core).
- `Infrastructure` реализует интерфейсы, объявленные в `Domain` / `Application`.
- Это тот самый DIP из прошлого урока — теперь на уровне проекта.

### Пример структуры папок

```
src/Shop/
├── Domain/
│   ├── Entities/Product.cs
│   └── Interfaces/IProductRepository.cs
├── Application/
│   ├── Services/ProductService.cs
│   └── Dtos/ProductDto.cs
├── Infrastructure/
│   ├── Data/AppDbContext.cs
│   └── Repositories/EfProductRepository.cs
└── Presentation/
    └── Api/Program.cs
```

- Интерфейс репозитория лежит в `Domain` (или Application).
- Реализация — в `Infrastructure`.
- `Application` работает с интерфейсом, а не с EF.

### Зачем это нужно

- Смена БД не трогает домен/приложение.
- Лёгкое тестирование (подмена репозитория).
- Читаемость и предсказуемость.

## Сделай сам (задачи с подсказками)

1. Спроектируй структуру папок для «Магазина»: Domain, Application, Infrastructure, Presentation.
   *Подсказка: по примеру выше.*
2. Объяви `IProductRepository` в Domain, реализуй `EfProductRepository` в Infrastructure.
   *Подсказка: интерфейс — в Domain, реализация — в Infrastructure.*
3. Сделай `ProductService` (Application), принимающий `IProductRepository` через конструктор.
   *Подсказка: инъекция через конструктор.*
4. Какой слой НЕ должен знать про EF Core?
   *Подсказка: Domain и Application.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Domain использует DbContext | домен привязан к БД | убрать зависимость |
| Вся логика в Program | нет слоёв | разложить по слоям |
| Сервис создаёт репозиторий через new | сильная связь | DI через конструктор |

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Папки есть, но зависимости нарушены | формальность | следить за направлением |

## Тест — проверь себя

**Вопрос 1.** Какой слой — «ядро» с бизнес-правилами?

- A) Infrastructure
- B) Domain
- C) Presentation

✅ **Верно: B.**
Почему: Domain — сущности и правила.
Почему неверны остальные:

- A — Infrastructure — детали (БД).
- C — Presentation — ввод/вывод.

**Вопрос 2.** Куда направлены зависимости?

- A) От Domain к Infrastructure.
- B) К центру: к Domain (внутрь).
- C) В любую сторону.

✅ **Верно: B.**
Почему: внутренние слои не знают про внешние.
Почему неверны остальные:

- A — Это нарушение направления.
- C — Направление строгое.

**Вопрос 3.** Где обычно объявляется интерфейс репозитория?

- A) В Infrastructure.
- B) В Domain (или Application) — как контракт.
- C) В Presentation.

✅ **Верно: B.**
Почему: контракт принадлежит внутреннему слою.
Почему неверны остальные:

- A — Реализация — в Infrastructure, а контракт — внутри.
- C — Presentation — не место для контрактов данных.

**Вопрос 4.** Какая польза от слоёв?

- A) Код становится длиннее.
- B) Замена БД и тестирование упрощаются.
- C) Программа быстрее.

✅ **Верно: B.**
Почему: слои дают гибкость и тестируемость.
Почему неверны остальные:

- A — Не про длину.
- C — Не про скорость.

## Итог урока

- Архитектура — план, как разделить код.
- Слои: Presentation → Application → Domain ← Infrastructure (зависимости внутрь).
- Контракты — внутри, реализации — снаружи.
- Слои упрощают смену БД и тесты.

# Урок 44. Безопасность

## Цель

Понять основы защиты приложений: аутентификация, секреты и валидация входных данных.

## Словарик новых слов

- **Аутентификация** — «кто ты?» (проверка личности).
- **Авторизация** — «что тебе можно?» (права).
- **JWT** — подписанный токен-«пропуск» для API.
- **Секрет** — ключ/пароль, который нельзя раскрывать.
- **Валидация** — проверка входных данных.

## Теория «с нуля»

### Аналогия: пропуск на завод

Чтобы войти на завод, нужен **пропуск** (аутентификация — кто ты). Но даже с пропуском не все двери открыты: в серверную пускают только администраторов (авторизация — что тебе можно).

### Аутентификация vs авторизация

- Аутентификация отвечает на вопрос «кто?».
- Авторизация — на вопрос «что можно?».

### JWT — «пропуск с печатью»

JWT — это строка (токен), которую сервер выдаёт после успешного логина. Клиент шлёт её в каждом запросе, а сервер проверяет **подпись** и пускает.

```
// клиент посылает заголовок:
// Authorization: Bearer <токен>
```

Проверка на сервере (настройка JWT в ASP.NET Core):

```
builder.Services.AddAuthentication()
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
```

```
            ValidIssuer = "myapp",
            ValidAudience = "myapp",
            IssuerSigningKey = new SymmetricSecurityKey(keyBytes)
        };
    });
```

- `keyBytes` — секрет, полученный из переменной окружения.

### Секреты: где хранить

**Никогда** не пиши ключи и пароли в коде или в репозитории.

```
string? key = Environment.GetEnvironmentVariable("JWT_SECRET");
```

Храни секреты в переменных окружения, в user-secrets или в Vault.

### Валидация входных данных

Не доверяй тому, что прислал клиент. Проверяй на границе API:

```
if (string.IsNullOrWhiteSpace(request.Title) || request.Title.Length > 100)
    return Results.BadRequest("Некорректное название");
```

### Пароли

Храни только **хэш** пароля, а не сам пароль:

```
var hasher = new PasswordHasher<object>();
string hash = hasher.HashPassword(null!, "секретный пароль");
```

Сравнение — через `VerifyHashedPassword`.

## Примеры

```
// проверка секрета перед использованием
string? secret = Environment.GetEnvironmentVariable("JWT_SECRET");
if (secret is null) throw new InvalidOperationException("JWT_SECRET не задан");
```

## Сделай сам (задачи с подсказками)

1. Объясни разницу между аутентификацией и авторизацией своими словами.
   *Подсказка: «кто» против «что можно».*

2. Получи секрет из переменной окружения.
   *Подсказка:* `Environment.GetEnvironmentVariable("...")`.
3. Покажи, как проверить входные данные перед обработкой.
   *Подсказка:* `if (string.IsNullOrWhiteSpace(...)) return BadRequest;`
4. Почему нельзя хранить пароль открыто? Приведи причину.
   *Подсказка: утечка БД = потеря паролей.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Секрет в коде/репо | утечка | env-переменные |
| Открытый эндпоинт без RequireAuthorization | доступ без проверки | защищать |
| Хранение пароля в открытом виде | кража паролей | хэшировать |
| Не проверять ввод | инъекции/мусор | валидация на границе |

## Тест — проверь себя

**Вопрос 1.** Аутентификация отвечает на вопрос:

- A) «Что тебе можно?»
- B) «Кто ты?»
- C) «Сколько времени?»

✅ **Верно: B.**
Почему: аутентификация — проверка личности.
Почему неверны остальные:

- A — Это авторизация.
- C — Это не связано.

**Вопрос 2.** Что такое JWT?

- A) Пароль.
- B) Подписанный токен-«пропуск».
- C) База данных.

✅ **Верно: B.**
Почему: JWT — токен с проверяемой подписью.
Почему неверны остальные:

- A — Это не пароль, а токен.
- C — Это не БД.

**Вопрос 3.** Где хранить секреты?

- A) В коде.
- B) В переменных окружения / безопасном хранилище.
- C) В readme.

✅ **Верно: B.**
Почему: секреты — вне кода и репозитория.
Почему неверны остальные:

- A, C — Код и readme попадают в git — это утечка.

**Вопрос 4.** Зачем валидировать входные данные?

- A) Чтобы код был длиннее.
- B) Чтобы не доверять клиенту и защититься от мусора/атак.
- C) Чтобы ускорить программу.

✅ **Верно: B.**
Почему: валидация — защита на границе.
Почему неверны остальные:

- A — Не для длины.
- C — Не для скорости.

## Итог урока

- Аутентификация — «кто», авторизация — «что можно».
- JWT — подписанный токен-пропуск.
- Секреты — только в окружении/хранилище, не в git.
- Пароли хранят как хэши.
- Валидируй ввод на границе API.

# Урок 45. Наблюдаемость

## Цель

Понять, как «видеть» работающий сервис: логирование, метрики и трейсинг.

## Словарик новых слов

- **Наблюдаемость** — возможность понять, что происходит в системе.
- **Логи** — текстовые записи событий.
- **Метрики** — числа (сколько, как быстро).
- **Трейсинг** — цепочка «где потрачено время» по запросу.
- `ILogger<T>` — встроенное логирование в ASP.NET Core.

## Теория «с нуля»

### Аналогия: приборная панель пилота

Пилот видит: приборы (метрики), записи в журнале (логи), маршрут по точкам (трейсы). Без панели самолёт неуправляем. Так и сервис: без наблюдаемости сложно понять, почему он «лежит».

### Три столпа

| Столп | Вопрос | Инструмент |
|---|---|---|
| Логи | как произошло? | `ILogger<T>` |
| Метрики | сколько/как быстро? | `Meter`, `Counter` |
| Трейсы | где время? | `Activity` |

### Логирование через ILogger

```
 public class OrderService
{
    private readonly ILogger<OrderService> _logger;
    public OrderService(ILogger<OrderService> logger) => _logger = logger;

    public void Create(Order order)
    {
        _logger.LogInformation("Создание заказа {OrderId}", order.Id);
        // ...
```

```
    }
}
```

**Важно:** используй параметры `{OrderId}`, а не склейку строк:

```
// плохо:
_logger.LogInformation($"Создан заказ {order.Id}");
// хорошо:
_logger.LogInformation("Создан заказ {OrderId}", order.Id);
```

Параметры позволяют фильтровать и агрегировать логи.

### Уровни логирования

`Trace`, `Debug`, `Information`, `Warning`, `Error`, `Critical` — от самых подробных до самых серьёзных.

### Метрики: Meter и Counter

```
using System.Diagnostics.Metrics;

var meter = new Meter("Shop.Orders", "1.0");
var ordersCreated = meter.CreateCounter<int>("orders.created");
// при создании заказа:
ordersCreated.Add(1);
```

### Трейсинг: Activity

```
var source = new ActivitySource("Shop.Orders");
using var activity = source.StartActivity("CreateOrder");
// внутри activity можно вешать теги
```

## Примеры

```
try
{
    // ...
}
catch (Exception ex)
{
    _logger.LogError(ex, "Не удалось создать заказ {OrderId}", order.Id);
    throw;
}
```

## Сделай сам (задачи с подсказками)

1. Добавь `ILogger<T>` в класс и залогируй создание через `LogInformation` с параметром.
   *Подсказка:* `LogInformation("... {Id}", id)`.
2. Залогируй ошибку в catch через `LogError(ex, ...)`.
   *Подсказка: передавай исключение первым аргументом.*
3. Создай `Meter` и `Counter`, наращивай счётчик.
   *Подсказка:* `meter.CreateCounter<int>("...")`.
4. Почему параметры лучше склейки строк?
   *Подсказка: фильтрация и агрегация.*

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Склейка строк в логах | неструктурированно | параметры `{...}` |
| Пустой catch | тишина при ошибке | LogError |
| Логировать секреты | утечка | не логировать ключи/пароли |
| Всё через Information | шум | выбирать уровень |

## Тест — проверь себя

**Вопрос 1.** Что такое метрики?

- A) Текстовые записи.
- B) Числовые показатели (сколько, как быстро).
- C) Ошибки.

✅ **Верно: B.**
Почему: метрики — числа.
Почему неверны остальные:

- A — Это логи.
- C — Метрики не только про ошибки.

**Вопрос 2.** Зачем параметры `{OrderId}` в сообщении лога?

- A) Чтобы удобнее было склеивать.
- B) Чтобы можно было фильтровать и агрегировать логи по значению.
- C) Чтобы логи были короче.

✅ **Верно: B.**
Почему: параметры становятся отдельными полями.
Почему неверны остальные:

- A — Склейка — как раз то, чего избегаем.
- C — Не только про длину.

**Вопрос 3.** Как залогировать ошибку с исключением?

- A) `_logger.LogInformation("ошибка")`
- B) `_logger.LogError(ex, "не удалось {Id}", id)`
- C) `Console.WriteLine(ex)`

✅ **Верно: B.**
Почему: LogError принимает исключение и параметры.
Почему неверны остальные:

- A — Information не для ошибок.
- C — Console.WriteLine — не структурированное логирование.

**Вопрос 4.** Что из этого — НЕ столп наблюдаемости?

- A) Логи
- B) Метрики
- C) Имена переменных

✅ **Верно: C.**
Почему: имена переменных не относятся к наблюдаемости.
Почему неверны остальные:

- A, B — Логи, метрики (и трейсы) — столпы.

## Итог урока

- Наблюдаемость = логи + метрики + трейсы.
- `ILogger<T>` с параметрами — правильное логирование.
- `Meter` / `Counter` — метрики.
- `Activity` — трейсинг.
- Не логируй секреты, не глотай ошибки.

# Урок 46. Паттерны проектирования

## Цель

Познакомиться с проверенными решениями частых задач: Factory, Strategy, Observer.

## Словарик новых слов

- **Паттерн** — проверенное типовое решение повторяющейся задачи.
- **Factory (фабрика)** — централизованное создание объектов.
- **Strategy (стратегия)** — взаимозаменяемые алгоритмы за интерфейсом.
- **Observer (наблюдатель)** — один источник уведомляет многих подписчиков.

## Теория «с нуля»

### Factory: аналогия с заводом

Вместо того чтобы собирать машину каждый раз вручную, есть завод: передал параметры — получил готовый автомобиль. Фабрика скрывает детали создания.

```
interface IPaymentProcessor { void Pay(decimal amount); }

class PaymentProcessorFactory
{
    public static IPaymentProcessor Create(string method) => method switch
    {
        "card" => new CreditCardProcessor(),
        "paypal" => new PayPalProcessor(),
        _ => throw new ArgumentException("Неизвестный способ")
    };
}
```

Использование:

```
IPaymentProcessor p = PaymentProcessorFactory.Create("card");
p.Pay(1000);
```

### Strategy: аналогия с маршрутами

Можно добраться пешком, на автобусе, на такси. Способ «как добраться» взаимозаменяем. В коде — алгоритмы за одним интерфейсом.

```
interface ITransport { void Go(string from, string to); }
class Walk : ITransport { public void Go(string f, string t) => Console.WriteLine($"Идём {f} → {t}"); }
class Bus : ITransport { public void Go(string f, string t) => Console.WriteLine($"Автобус {f} → {t}"); }
```

### Observer: аналогия с подпиской на блог

Блог публикует пост — все подписчики получают уведомление. Один источник, много подписчиков. В C# это события (event).

```
class Blog
{
    public event EventHandler<string>? PostPublished;

    public void Publish(string title)
    {
        Console.WriteLine($"Опубликовано: {title}");
        PostPublished?.Invoke(this, title);
    }
}
```

Подписка:

```
Blog blog = new Blog();
blog.PostPublished += (s, title) => Console.WriteLine($"Уведомление: {title}");
blog.Publish("SOLID простыми словами");
```

## Примеры

```
// Factory: создание по выбору
var proc = PaymentProcessorFactory.Create("paypal");

// Strategy: подмена алгоритма
ITransport route = new Bus();
route.Go("A", "B");

// Observer: событие
blog.PostPublished += (s, t) => Mailer.Send(t);
```

## Сделай сам (задачи с подсказками)

1. Напиши `IPaymentProcessor` и фабрику для двух способов оплаты.
   *Подсказка:* `Create(string method)` *со switch.*
2. Сделай интерфейс `ITransport` и две стратегии (Walk, Bus).
   *Подсказка: метод* `Go(from, to)`.
3. Создай класс с событием и двух подписчиков.
   *Подсказка:* `event EventHandler<string>?` + `+=`.

## Разбор типичных ошибок

| Ошибка | Что не так | Как правильно |
|---|---|---|
| Создание объектов «размазано» по коду | сложно менять | фабрика |
| Огромный switch по способам в 10 местах | дублирование | фабрика инкапсулирует создание |
| «Наблюдатель» без события | плохо подписываться | использовать event |
| Забыл `?.Invoke` | NullReferenceException | `PostPublished?.Invoke(...)` |

## Тест — проверь себя

**Вопрос 1.** Что решает Factory?

- A) Централизованное создание объектов.
- B) Сортировку данных.
- C) Логирование.

✅ **Верно: A.**
Почему: фабрика создаёт объекты в одном месте.
Почему неверны остальные:

- B, C — Это другие задачи.

**Вопрос 2.** Что такое Strategy?

- A) Способ хранить данные.
- B) Взаозаменяемые алгоритмы за интерфейсом.
- C) Тип данных.

✅ **Верно: B.**
Почему: стратегия — сменяемое поведение через интерфейс.
Почему неверны остальные:

- A, C — Это не про хранение/типы.

**Вопрос 3.** Как в Observer источник уведомляет подписчиков?

- A) Через событие ( `event` ) и `?.Invoke`.
- B) Через прямое обращение к каждому.
- C) Через файл.

✅ **Верно: A.**
Почему: события — механизм Observer в C#.
Почему неверны остальные:

- B — Прямые ссылки — не паттерн Observer.
- C — Файлы тут ни при чём.

**Вопрос 4.** Зачем паттерны?

- A) Чтобы код был длиннее.
- B) Использовать проверенные решения и не изобретать велосипед.
- C) Чтобы запретить наследование.

✅ **Верно: B.**
Почему: паттерны — типовые проверенные решения.
Почему неверны остальные:

- A — Не для длины.
- C — Не про запреты.

## Итог урока

- Паттерны — проверенные шаблоны решений.
- Factory — создание объектов в одном месте.
- Strategy — взаимозаменяемые алгоритмы.
- Observer — события для уведомлений.

## Итог уровня 4

Ты прошёл путь до Senior-мышления:

- SOLID — пять принципов надёжного кода;
- архитектура и слои, DI;
- безопасность (JWT, секреты, валидация);
- наблюдаемость (логи, метрики, трейсы);
- паттерны проектирования.

**Поздравляю — Уровень 4 пройден!**
Теперь у тебя есть полный путь: от «что такое переменная» до проектирования систем.

# Модуль 00: Введение в C# и .NET

> **Цель:** понять, что такое C# и .NET, установить SDK, создать первую программу и разобраться, что происходит под капотом.

## Зачем это нужно

Когда ты начинаешь с C#, важно сразу понять две вещи: **язык** и **платформу**. C# — это язык программирования: правила, по которым ты пишешь код. .NET — это платформа: она превращает твой код в работающую программу и даёт огромную библиотеку готовых инструментов (работа с файлами, сетью, JSON, коллекциями и т.д.).

Многие новички путаются: «Почему мы устанавливаем .NET SDK, если учим C#?» Ответ простой — без .NET код на C# не запустится. Но и без C# платформа бесполезна. Это как автомобиль: C# — это правила дорожного движения, а .NET — сам автомобиль. Оба нужны вместе.

Индустрия любит C# за три вещи:

- **Надёжность** — строгая типизация ловит огромную долю ошибок ещё на этапе компиляции, до запуска программы.
- **Мощность** — язык развивается очень активно: каждый год выходят новые возможности, которые делают код короче и безопаснее.
- **Скорость разработки** — .NET «из коробки» даёт решение большинства рутинных задач.

## Теория

### 1. Что такое C# и .NET

C# (произносится «си-шарп») — язык программирования, созданный компанией Microsoft в 2000-х. На нём пишут:

- десктопные приложения (Windows);
- веб-сервисы и API (ASP.NET Core);
- мобильные приложения (через .NET MAUI / Xamarin);
- игры (Unity);
- облачные сервисы;
- фоновые службы и многое другое.

.NET — это **платформа**. Её ключевые части:

| Компонент | Что делает |
|---|---|
| **CLR** (Common Language Runtime) | «движок»: управляет памятью, потоками, запускает код |
| **BCL** (Base Class Library) | огромная коллекция готовых классов: `Console`, `File`, `String`, `List<T>`, `HttpClient` и тысячи других |
| **SDK** | набор инструментов для разработки: компилятор, утилиты командной строки |
| **Runtime** | среда для запуска готовых программ у пользователя |

## 2. Как код превращается в программу

C# — компилируемый язык, но не совсем так, как C++ (где сразу получается машинный код). Процесс двухступенчатый:

1. **Компилятор** (который входит в SDK) превращает твой исходный код `.cs` в промежуточный код **IL** (Intermediate Language). Он «почти машинный», но ещё не привязан к конкретному процессору.
2. **JIT-компилятор** (Just-In-Time), который живёт внутри CLR, при запуске программы на лету превращает IL в машинный код конкретного процессора.

Это даёт важное преимущество: одна и та же программа на IL может работать на Windows, Linux и macOS — платформа адаптируется под каждую ОС.

> **Аналогия.** Представь, что ты написал рецепт (исходный код). Повар (компилятор) переписывает его на универсальный язык кухонных операций (IL). А на каждой конкретной кухне (ОС) другой повар (JIT) переводит это в конкретные движения рук и приборов (машинный код).

## 3. Установка SDK

Качаем **.NET SDK** (не Runtime!) с официального сайта dotnet.microsoft.com/download.

- **SDK** нужен разработчику — он содержит компилятор и инструменты.
- **Runtime** нужен только для запуска готовых программ.

Проверяем установку:

```
dotnet --version
```

Ты увидишь номер версии, например `8.0.100`. Это значит, что SDK установлен и работает.

## 4. dotnet CLI — главный инструмент

CLI (Command Line Interface) — это команды в терминале. Вот самые важные:

| Команда | Что делает |
|---|---|
| `dotnet new console -n MyApp` | создаёт новый консольный проект `MyApp` |
| `dotnet build` | компилирует проект в папке |
| `dotnet run` | собирает и запускает программу |
| `dotnet restore` | скачивает зависимые пакеты NuGet |
| `dotnet test` | запускает тесты |

Попробуем:

```
dotnet new console -n HelloWorld
cd HelloWorld
dotnet run
```

Ты увидишь `Hello, World!` — твою первую работающую программу.

## 5. Анатомия проекта

Заглянем в папку проекта:

```
HelloWorld/
├── HelloWorld.csproj   # файл проекта: настройки и зависимости
├── Program.cs           # исходный код программы
├── bin/                 # результат сборки (генерируется)
└── obj/                 # промежуточные файлы сборки (генерируются)
```

- `Program.cs` — здесь живёт код. `bin/` и `obj/` генерируются автоматически, их **не нужно** хранить в системе контроля версий (для этого есть `.gitignore`).
- Файл `.csproj` — XML-файл с настройками проекта. В современных версиях он очень короткий:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

## 6. Top-level statements

Посмотрим на `Program.cs`:

```
Console.WriteLine("Hello, World!");
```

Это всего одна строка! Раньше нужно было писать гораздо больше:

```
using System;

namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Начиная с C# 9, компилятор позволяет писать код «вне класса» — это называется **top-level statements**. Программа упростилась в разы. Понимать старый синтаксис всё равно полезно (он встречается в чужих проектах), но писать проще.

## 7. NuGet

**NuGet** — это менеджер пакетов .NET, как pip в Python или npm в Node.js. Пакет — это готовая библиотека, которую можно подключить к проекту одной командой.

Подключить пакет:

```
dotnet add package Humanizer
```

Эта команда добавит пакет в `.csproj`, и его можно будет использовать. Сейчас в NuGet сотни тысяч пакетов — от маленьких утилит до целых фреймворков.

## 8. Первые программы

**Приветствие:**

```
Console.WriteLine("Привет, мир!");
Console.Write("Как тебя зовут? ");
```

```
string name = Console.ReadLine() ?? "";
Console.WriteLine($"Приятно познакомиться, {name}!");
```

Обрати внимание: `Console.WriteLine("Привет")` — выводит строку, а `Console.ReadLine()` — считывает ввод пользователя. Символ `$` перед строкой включает **интерполяцию** — подстановку значений переменных в строку. Полное название этого механизма — `$""` с фигурными скобками. Об этом подробнее в следующем модуле.

**Числа:**

```
 int a = 5;
int b = 7;
Console.WriteLine($"Сумма: {a + b}");
```

**Чтение чисел из консоли:**

```
 int age = int.Parse(Console.ReadLine()!);
```

`int.Parse` преобразует строку (то, что ввёл пользователь) в число. Восклицательный знак `!` говорит компилятору: «Я знаю, что тут не будет null». Про nullable-типы поговорим в модуле 04.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Копировать код без понимания | Ты не запоминаешь, что происходит, и не можешь исправить ошибку | Набирай каждую программу руками и разбирай каждую строку |
| Не игнорировать `bin/` и `obj/` | Засоряешь репозиторий мусором, который генерируется автоматически | Добавь `.gitignore` с `bin/` и `obj/` |
| «У меня работает в Visual Studio» | Другие не смогут запустить без тех же настроек | Проверяй запуск через `dotnet run` в чистом терминале |
| Жёстко зашивать значения в код | Чинить такой код больно | Используй переменные и аргументы |

## Контрольные вопросы

1. Чем отличается язык C# от платформы .NET?
2. Что делает CLR? Что делает JIT?
3. В чём разница между SDK и Runtime?
4. Какая команда создаёт новый проект?
5. Что хранится в папках `bin/` и `obj/`?
6. Что такое NuGet и зачем он нужен?

7. Что такое top-level statements?
8. Что выведет `Console.WriteLine($"Сумма: {2 + 3}");`?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 00

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Первая программа через CLI

**Сложность:** ★☆☆
**Описание:** Создай проект через терминал, запусти его, затем измени текст приветствия.

**Требования:**

- Создай проект командой `dotnet new console -n MyFirstApp`.
- Запусти его (`dotnet run`), убедись, что видишь `Hello, World!`.
- Измени текст на `Привет из C#!` и запусти снова.

**Подсказка:** После изменения `Program.cs` не забудь пересобрать — `dotnet run` это сделает автоматически.

## Задача 2: Анкета

**Сложность:** ★☆☆
**Описание:** Напиши программу-анкету: узнай имя, возраст и город, затем выведи приветствие с этими данными.

**Требования:**

- Считывай три значения через `Console.ReadLine()`.
- Выведи одно предложение с использованием интерполяции `$""`.

**Пример вывода:**

```
 Как тебя зовут? Иван
Сколько тебе лет? 25
В каком городе ты живёшь? Москва
Привет, Иван из Москвы! Тебе 25 лет.
```

## Задача 3: Подключение пакета NuGet

**Сложность:** ★★☆
**Описание:** Подключи пакет `Humanizer` и используй его для «очеловечивания» чисел.

**Требования:**

- Подключи пакет командой `dotnet add package Humanizer`.
- Попробуй `(12345).ToWords()` и `12345.Ordinalize()` — пойми, что они делают.
- Выведи результаты.

**Подсказка:** Пакет подключается в `.csproj`. После этого его можно использовать в коде — добавь `using Humanizer;` в начало файла.

## Задача 4: Изучаем структуру проекта

**Сложность:** ★☆☆
**Описание:** Исследуй, что появляется после сборки.

**Требования:**

- Собери проект (`dotnet build`).
- Загляни в папки `bin/` и `obj/` — что там лежит?
- Найди `.exe` (или бинарный файл без расширения на Linux/macOS) в `bin/Debug/net8.0/`.
- Запусти этот файл напрямую, не через `dotnet run`.

**Подсказка:** Найдите в папке `bin/Debug/net8.0/` файл с именем проекта. Запусти его двойным кликом или из терминала.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

# Разбор кода

# Разбор кода — Модуль 00

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: «Работает, и ладно»

### Исходный код

```
 Console.WriteLine("Введите число: ");
string input = Console.ReadLine() ?? "";
int number = int.Parse(input);
Console.WriteLine("Вы ввели число: " + number);
```

### Что не так

- Сложение строк через `+` (`"Вы ввели число: " + number`) — старый синтаксис. Читаемость хуже, легко ошибиться с пропущенным пробелом.
- `int.Parse` упадёт с исключением, если пользователь введёт текст, а не число.

### Как лучше

```
 Console.Write("Введите число: ");
string input = Console.ReadLine() ?? "";

if (int.TryParse(input, out int number))
{
    Console.WriteLine($"Вы ввели число: {number}");
}
else
{
    Console.WriteLine("Это не число!");
}
```

### Вывод

Используй интерполяцию `$""` вместо склейки через `+`, а для ввода от пользователя — `TryParse`, чтобы программа не падала.

## Пример 2: Копипаст без понимания

### Исходный код

```
namespace MyNamespace
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World");
        }
    }
}
```

### Что не так

- Это легальный код, но он написан по старому шаблону: понапрасну `namespace`, класс и `Main`. Кода больше, а смысла нет.
- Новичок, который копирует его не задумываясь, учится писать лишнее.

### Как лучше

```
Console.WriteLine("Hello World");
```

### Вывод

Top-level statements — современный и предпочтительный способ. Но если встретишь старый формат в чужом проекте, не пугайся: внутри всё то же самое.

## Пример 3: «У меня работает в Visual Studio»

### Исходный код

Пользователь «всегда делал так» внутри Visual Studio и не проверял запуск из терминала. Проект отказывался собираться на другой машине: пропущен пакет, неправильная целевая версия .NET, закоммиченные `bin/` и `obj/`.

### Что не так

- Окружение одного человека не является стандартом для всех.
- Закоммиченные артефакты сборки (`bin/`, `obj/`) ломают репозиторий: каждый пересобирает мусор, появляются конфликты.

### Как лучше

- Всегда проверяй чистую сборку: `dotnet build` и `dotnet run` в терминале.
- Добавь `.gitignore` с `bin/`, `obj/`, `*.user` и другими артефактами IDE.

### Вывод

Твоё окружение — не эталон. Проверяй проект так, как его соберёт CI или другой разработчик: с нуля, из терминала.

## Пример 4: Hardcoded-строка с путём к файлу

### Исходный код

```
 string path = "C:\\Users\\ivan\\Documents\\save.txt";
Console.WriteLine($"Читаю файл: {path}");
```

### Что не так

- Путь зашит в код. На другой машине (или даже у того же юзера, но в другой папке) программа не найдёт файл.
- Обратные слэши `\\` легко сломать — а экранирование в строках иногда пропускают.

### Как лучше

```
 string path = Path.Combine(Environment.SpecialFolder.MyDocuments.ToString(), "save.txt");
Console.WriteLine($"Читаю файл: {path}");
```

### Вывод

Не зашивай жёстко пути и значения. Используй `Path.Combine` и переменные окружения — программа станет переносимой.

## Выводы по модулю

Главное в начале пути — не зубрить синтаксис, а понять, **что происходит** при сборке и запуске. Пиши маленькие программы руками, запускай их из терминала и исследуй, что появляется на диске. Это заложит фундамент для всей дальнейшей работы.

# Модуль 01: Основы синтаксиса

> **Цель:** уверенно работать с типами данных, переменными, операторами, преобразованием типов и строками.

## Зачем это нужно

Данные — это сердце любой программы. Пока ты не понимаешь типы, переменные и строки, невозможно решить ни одну реальную задачу: считать зарплату, обработать ввод пользователя, прочитать файл, отправить запрос. Этот модуль — фундамент, на котором строится весь язык. Здесь мы закладываем «мускулы»: к концу модуля ты свободно будешь манипулировать числами и текстом.

## Теория

### 1. Типы данных

В C# каждая переменная имеет **тип**. Тип определяет: что можно в неё хранить, сколько памяти она занимает и какие операции с ней разрешены.

#### Типы значений (value types)

Значение хранится **напрямую** в переменной.

| Тип | Размер | Диапазон / значение | Когда использовать |
|---|---|---|---|
| `int` | 4 байта | −2 147 483 648 … 2 147 483 647 | целые числа, счётчики, индексы |
| `long` | 8 байт | ~$\pm 9.2 \times 10^{18}$ | очень большие целые |
| `short` | 2 байта | −32 768 … 32 767 | редкий, для экономии памяти |
| `byte` | 1 байт | 0 … 255 | байты, битовые маски |
| `double` | 8 байт | $\pm 1.5 \times 10^{-45}$ … $3.4 \times 10^{38}$ | обычные дробные числа (по умолчанию) |
| `float` | 4 байта | ~$\pm 3.4 \times 10^{38}$ | дробные, когда критична память |
| `decimal` | 16 байт | $\pm 7.9 \times 10^{-28}$ … $7.9 \times 10^{28}$ | деньги, точные вычисления |
| `bool` | 1 байт | `true` / `false` | флаги, условия |
| `char` | 2 байта | один символ Unicode | одиночные символы |

**Самое важное про `double` и `decimal`:**

```
double a = 0.1 + 0.2;
Console.WriteLine(a); // 0.30000000000000004 (!)
```

Дробные числа хранятся в двоичном виде, поэтому `0.1 + 0.2` даёт «хвост». Для денег и точных расчётов используй `decimal`:

```
decimal price = 19.99m;
decimal discount = 0.15m;
Console.WriteLine(price * (1 - discount));
```

**Ссылочные типы (reference types)**

Переменная хранит **ссылку** на объект в памяти. К ним относятся `string`, массивы, классы.

| Тип | Когда использовать |
|---|---|
| `string` | текст, строки |
| массивы `[]` | фиксированный набор однотипных элементов |

## 2. Переменные

Объявление и инициализация:

```
int age = 30;              // тип указан явно
var name = "Иван";         // var — тип определяется по значению
```

`var` — не «тип без типа», а просто сокращение: компилятор сам подставляет тип. Используй `var`, когда тип очевиден из правой части (например, `var list = new List<int>();`), и явный тип — когда важна читаемость.

**Константы:**

```
const double Pi = 3.14159;  // нельзя изменить после объявления
```

## 3. Правила именования

| Конвенция | Где используется | Пример |
|---|---|---|
| `PascalCase` | классы, методы, свойства | `CalculateArea`, `UserProfile` |
| `camelCase` | локальные переменные, параметры | `totalPrice`, `userAge` |
| `UPPER_CASE` / `PascalCase` | константы | `PI`, `MaxRetries` |

Имена должны быть осмысленными. `int x = 5;` — плохо. `int userAge = 5;` — хорошо.

## 4. Операторы

**Арифметика:** `+`, `-`, `*`, `/`, `%` (остаток от деления).

```
int a = 17, b = 5;
Console.WriteLine(a / b);  // 3 (целочисленное деление!)
Console.WriteLine(a % b);  // 2 (остаток)
```

Обрати внимание: `17 / 5` даёт `3`, а не `3.4`, потому что оба числа целые. Чтобы получить дробный результат, хотя бы один операнд должен быть дробным.

**Сравнение:** `==`, `!=`, `<`, `>`, `<=`, `>=`.

**Логические:** `&&` (и), `||` (или), `!` (не).

```
bool isAdult = age >= 18 && age < 65;
bool isHoliday = day == "Сб" || day == "Вс";
```

**Инкремент/декремент:** `++`, `--`.

```
int counter = 5;
counter++;        // counter == 6
Console.WriteLine(counter++); // выведет 6, потом counter станет 7
Console.WriteLine(++counter); // counter станет 8, потом выведет 8
```

**Оператор `??` (null-coalescing):** возвращает правую часть, если левая равна `null`.

```
string name = Console.ReadLine() ?? "Гость";
```

**Оператор `?.` (null-conditional):** безопасно обращается к члену, если объект не `null`.

```
string? s = null;
int? length = s?.Length;  // null, не исключение
```

## 5. Приведение типов

**Неявное** — когда данные не теряются:

```
int small = 10;
long big = small;         // неявно: int в long безопасно
```

**Явное** — когда может быть потеря данных:

```
double d = 9.7;
int i = (int)d;           // 9, дробная часть отбрасывается
```

**Проверка переполнения:**

```
checked
{
    int x = int.MaxValue;
    int y = x + 1;        // OverflowException
}
```

## 6. Преобразование строк в числа

Самый безопасный способ — `TryParse`:

```
if (int.TryParse(input, out int number))
{
    Console.WriteLine($"Число: {number}");
}
else
{
    Console.WriteLine("Это не число");
}
```

Ошибки, которые ловят `Parse` → исключения, обрабатываются конструкциями try-catch (модуль 08), но для ввода от пользователя `TryParse` удобнее.

## 7. Строки

**Интерполяция `$`** — основной способ собрать строку:

```
string name = "Мария";
int age = 28;
Console.WriteLine($"{name}, тебе {age} лет.");
```

**Вербатим `@`** — строка с буквальными переводами строк и без экранирования:

```
string path = @"C:\Users\ivan\file.txt";
string text = @"Первая строка
Вторая строка";
```

**Строки неизменяемы (immutable):** любая операция «изменения» создаёт новую строку. Это важно для памяти и производительности (детальнее — в разборе кода).

**Популярные методы:**

```
 string s = "Hello, World!";
Console.WriteLine(s.Length);          // 13
Console.WriteLine(s.ToUpper());       // "HELLO, WORLD!"
Console.WriteLine(s.Substring(7));    // "World!"
Console.WriteLine(s.IndexOf("o"));    // 4
Console.WriteLine(s.Replace("World", "C#")); // "Hello, C#!"
var parts = s.Split(", ");            // ["Hello", "World!"]
```

**Сравнение строк:**

```
 string a = "Привет";
string b = "привет";
Console.WriteLine(a == b);                      // false (регистр важен)
Console.WriteLine(a.Equals(b, StringComparison.OrdinalIgnoreCase)); // true
```

### 8. Ввод и вывод

```
 Console.Write("Введи имя: ");
string name = Console.ReadLine() ?? "";
Console.WriteLine($"Привет, {name}!");
```

Парсинг чисел с учётом культуры:

```
double d = double.Parse("3,5", System.Globalization.CultureInfo.InvariantCulture);
```

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| `double` для денег | Потеря точности из-за округлений | `decimal` |
| `17 / 5` ожидаем `3.4` | Целочисленное деление отбрасывает остаток | один из операндов — дробный: `17.0 / 5` |
| `int.Parse` без обработки | Программа падает при неверном вводе | `TryParse` |
| Склейка строк `"a" + b + "c"` | Нечитаемо; при многих операциях — медленно | интерполяция `$""` |
| Имена `x`, `y`, `temp` | Через месяц никто не поймёт код | осмысленные имена |

## Контрольные вопросы

1. Чем отличается `double` от `decimal`? Когда что использовать?
2. Что вернёт `17 / 5`? А `17.0 / 5`?
3. Что делает `%`?
4. В чём разница `counter++` и `++counter`?
5. Как безопасно превратить строку в число?
6. Что такое интерполяция строк? Приведи пример.
7. Зачем нужен оператор `??`?
8. Почему строки считаются неизменяемыми?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 01

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Калькулятор

**Сложность:** ★☆☆
**Описание:** Напиши простейший калькулятор: два числа и операция (`+`, `-`, `*`, `/`). Если операция неверная — выведи сообщение об ошибке.

**Требования:**

- Считай два числа через `double.TryParse`.
- Считай операцию через `Console.ReadLine()`.
- Используй `switch` для выбора действия.

**Пример вывода:**

```
 Введите первое число: 10
Введите второе число: 4
Введите операцию (+ - * /): /
Результат: 2,5
```

## Задача 2: Конвертер температур

**Сложность:** ★☆☆
**Описание:** Переведи температуру из Цельсия в Фаренгейты и обратно.

**Требования:**

- Формула: F = C × 9/5 + 32; C = (F − 32) × 5/9.
- Спроси направление конвертации (C→F или F→C), затем значение.
- Выведи результат с округлением до 2 знаков: `F2`.

**Подсказка:** «F2» в интерполяции — `$"{value:F2}"`.

## Задача 3: Подсчёт символов

**Сложность:** ★☆☆
**Описание:** Прими строку и выведи её длину, а также количество букв 'a' (без учёта регистра).

**Требования:**

- Используй `.Length` и, например, `.ToLower()` + `IndexOf` в цикле (или `Count`, если знаешь LINQ).
- Выведи оба числа.

## Задача 4: Шифр Цезаря

**Сложность:** ★★☆
**Описание:** Реализуй шифр Цезаря для английских строчных букв: каждая буква сдвигается на `shift` позиций в алфавите.

**Требования:**

- Прими строку и число сдвига.
- Обрабатывай `a` … `z` (строчные). Остальные символы оставляй как есть.
- Циклический сдвиг: `z` + 1 → `a`.

**Подсказка:** использовать `(char)('a' + (ch - 'a' + shift) % 26)`.

## Задача 5: Безопасный ввод числа

**Сложность:** ★★☆
**Описание:** Напиши функцию (или повторяющийся блок), которая запрашивает число, пока пользователь не введёт корректное значение.

**Требования:**

- Используй `double.TryParse` в цикле `while`.
- При неверном вводе выводи сообщение и запрашивай снова.

**Пример вывода:**

```
 Введите число: abc
Некорректное число, попробуйте ещё раз.
Введите число: 42
Спасибо, число: 42
```

## Задача 6: Разбор даты

**Сложность:** ★★☆
**Описание:** Прими строку формата `ГГГГ-ММ-ДД` (например, `2024-01-15`) и выведи день, месяц и год отдельно.

**Требования:**

- Используй `Split('-')`.
- Выведи: `Год: 2024, Месяц: 01, День: 15`.

**Подсказка:** `Split` возвращает массив строк; индексируй его `parts[0]`, `parts[1]`, `parts[2]`.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 01

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: Магические числа

### Исходный код

```
 double price = 499;
double tax = price * 0.2;
Console.WriteLine($"Цена с налогом: {price + tax}");
```

### Что не так

- `0.2` — налоговая ставка, зашитая «магическим числом». Через месяц никто не вспомнит, что значит `0.2`. При изменении налога придётся искать все вхождения.

### Как лучше

```
 const double TaxRate = 0.2;
double price = 499;
double tax = price * TaxRate;
Console.WriteLine($"Цена с налогом: {price + tax}");
```

### Вывод

Именованные константы объясняют смысл чисел. Менять `TaxRate` — одно место, а не десяток строк.

## Пример 2: `double` для денег

### Исходный код

```
 double total = 0;
for (int i = 0; i < 1000; i++)
{
    total += 0.1;
```

```
}
Console.WriteLine(total); // 99.9999999999986 — НЕ 100!
```

### Что не так

- `double` хранит числа неточно. При накоплении мелких приращений ошибка растёт.

### Как лучше

```
 decimal total = 0m;
for (int i = 0; i < 1000; i++)
{
    total += 0.1m;
}
Console.WriteLine(total); // 100.0
```

### Вывод

Для денег, процентов и всего, где важна точность, — только `decimal`. `double` — для научных расчётов, где точность до 15-17 знака не критична.

## Пример 3: Склейка строк в цикле

### Исходный код

```
 string result = "";
for (int i = 0; i < 10000; i++)
{
    result += "x"; // создаётся 10000 промежуточных строк
}
```

### Что не так

- Строки неизменяемы. Каждое `+=` создаёт **новую** строку, копируя старую. Итог — 10 000 аллокаций и квадратичная сложность.

### Как лучше

```
 var sb = new System.Text.StringBuilder();
for (int i = 0; i < 10000; i++)
{
    sb.Append('x');
```

```
}
string result = sb.ToString();
```

### Вывод

`StringBuilder` мутирует внутренний буфер без создания новых строк. Для небольшого числа операций (до ~10) интерполяция или `string.Join` — проще; для циклов — `StringBuilder`.

## Пример 4: Переполнение int

### Исходный код

```
 int secondsInYear = 60 * 60 * 24 * 365;
Console.WriteLine(secondsInYear); // -1461241 (переполнение!)
```

### Что не так

- `60 * 60 * 24 * 365` переполняет `int`: результат не влезает в 4 байта, и мы получаем отрицательное число.

### Как лучше

```
 long secondsInYear = 60L * 60 * 24 * 365; // суффикс L — long
Console.WriteLine(secondsInYear); // 31536000
```

### Вывод

Всегда думай о диапазоне типа. Суффикс `L` заставляет вычисления идти в `long`.

## Пример 5: `Parse` без обработки ошибок

### Исходный код

```
 Console.Write("Введи число: ");
int number = int.Parse(Console.ReadLine()!);
Console.WriteLine($"Квадрат: {number * number}");
```

### Что не так

- Если пользователь введёт `abc`, программа упадёт с `FormatException`. Ввод из консоли — ненадёжный источник данных.

### Как лучше

```
 Console.Write("Введи число: ");
if (int.TryParse(Console.ReadLine(), out int number))
{
    Console.WriteLine($"Квадрат: {number * number}");
}
else
{
    Console.WriteLine("Некорректное число");
}
```

### Вывод

`TryParse` — стандарт для разбора пользовательского ввода: не падает, а сигнализирует результатом `bool`.

# Модуль 02: Управление потоком

> **Цель:** свободно ветвить логику и организовывать циклы.

## Зачем это нужно

Программа без ветвлений — просто последовательность команд. Любая реальная программа принимает решения: если пользователь ввёл неверный пароль, если баланс меньше суммы — и повторяет действия: пока не введено число, пока не закончились товары в списке. Управление потоком — это «мозг» программы: здесь рождается вся её логика.

## Теория

### 1. Условные конструкции `if` / `else if` / `else`

```
 int age = 25;

if (age < 18)
{
    Console.WriteLine("Несовершеннолетний");
}
else if (age < 65)
{
    Console.WriteLine("Взрослый");
}
else
{
    Console.WriteLine("Пенсионер");
}
```

**Правило:** чем раньше выходишь из ветвлений, тем читабельнее код. Это называется **guard clauses**:

```
 void SetAge(int age)
{
    if (age <= 0 || age > 120)
    {
        Console.WriteLine("Некорректный возраст");
        return; // ранний выход
    }
```

```
    Console.WriteLine($"Возраст установлен: {age}");
}
```

**Тернарный оператор** — для простого выбора значения:

```
string status = age >= 18 ? "Взрослый" : "Несовершеннолетний";
```

## 2. `switch` и `switch`-выражения

**Классический `switch`:**

```
string day = Console.ReadLine() ?? "";

switch (day.ToLower())
{
    case "пн":
    case "вт":
    case "ср":
        Console.WriteLine("Рабочий день");
        break;
    case "сб":
    case "вс":
        Console.WriteLine("Выходной");
        break;
    default:
        Console.WriteLine("Не знаю такой день");
        break;
}
```

**`switch`-выражение** (C# 8+) — компактнее:

```
string message = day.ToLower() switch
{
    "пн" or "вт" or "ср" => "Рабочий день",
    "сб" or "вс" => "Выходной",
    _ => "Не знаю такой день"
};
```

**Паттерны** (C# 9+):

```
string Classify(int number) => number switch
{
    > 0 => "Положительное",
    < 0 => "Отрицательное",
```

```
    _ => "Ноль"
};
```

`when` — дополнительное условие:

```
string Describe(int score) => score switch
{
    >= 90 when score <= 100 => "Отлично",
    >= 70 => "Хорошо",
    >= 50 => "Удовлетворительно",
    _ => "Плохо"
};
```

### 3. Циклы

**`for`** — когда известно количество повторений:

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i); // 0 1 2 3 4
}
```

**`while`** — когда условие проверяется в начале:

```
int count = 0;
while (count < 3)
{
    Console.WriteLine("Повтор");
    count++;
}
```

**`do-while`** — когда тело должно выполниться хотя бы раз:

```
string? input;
do
{
    Console.Write("Введи 'да': ");
    input = Console.ReadLine();
}
while (input != "да");
```

**`foreach`** — перебор коллекций и строк:

```
string word = "C#";
foreach (char ch in word)
{
    Console.WriteLine(ch);
}

int[] numbers = { 1, 2, 3 };
foreach (int n in numbers)
{
    Console.WriteLine(n);
}
```

### 4. `break` и `continue`

- `break` — немедленно выйти из цикла.
- `continue` — перейти к следующей итерации.

```
for (int i = 1; i <= 10; i++)
{
    if (i == 3) continue;   // пропустить 3
    if (i == 8) break;      // остановиться на 8
    Console.WriteLine(i);    // 1 2 4 5 6 7
}
```

### 5. Вечный цикл с выходом

```
while (true)
{
    Console.Write("Команда (exit — выход): ");
    string command = Console.ReadLine() ?? "";

    if (command == "exit") break;
    Console.WriteLine($"Выполняю: {command}");
}
```

### 6. Вложенные циклы

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        Console.Write($"{i * j} ");
    }
```

```
    Console.WriteLine();
}
```

### 7. Почему не `goto`

`goto` существует, но почти всегда превращает код в «спагетти». Практически любую проблему можно решить циклом, `break` или методом. Не используй `goto` без крайней нужды.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| `else` после `return` | Лишние блоки ухудшают читаемость | ранний `return` вместо `else` |
| Сравнение `==` с `"Да"` без учета регистра | `"да"` ≠ `"Да"` | сравнивай через `.Trim().ToLower()` или `StringComparison.OrdinalIgnoreCase` |
| Бесконечный цикл без выхода | Зависание программы | контролируй `break` или условие |
| Глубокая вложенность if'ов | Код трудно читать | guard clauses, ранний выход |

## Контрольные вопросы

1. Когда используется `else if`?
2. Что такое guard clause? Приведи пример.
3. Чем классический `switch` отличается от `switch`-выражения?
4. Что делает `when` в паттерне?
5. В чём разница `while` и `do-while`?
6. Что делает `break`, а что `continue`?
7. Когда использовать `for`, а когда `foreach`?
8. Как запилить вечный цикл с выходом по команде?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 02

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: FizzBuzz

**Сложность:** ★☆☆
**Описание:** Для чисел от 1 до 100 выведи: число, если оно не делится ни на 3, ни на 5; `Fizz`, если делится на 3; `Buzz`, если на 5; `FizzBuzz`, если на оба.

**Требования:**

- Используй `for` и `%` (остаток от деления).
- Порядок проверок важен: сначала «делится на 15».

## Задача 2: Таблица умножения

**Сложность:** ★☆☆
**Описание:** Выведи таблицу умножения от 1 до 10 (10 строк по 10 столбцов).

**Требования:**

- Используй вложенные циклы.
- Форматируй вывод, чтобы столбцы были ровными: `$"{product,4}"`.

## Задача 3: Игра «Угадай число»

**Сложность:** ★★☆
**Описание:** Компьютер загадывает число от 1 до 100. Пользователь угадывает, программа подсказывает «больше»/«меньше». Даётся 7 попыток.

**Требования:**

- `Random` для загадывания: `Random.Shared.Next(1, 101)`.
- `while` с счётчиком попыток и `break` при угадывании.

**Пример вывода:**

```
 Загадано число от 1 до 100. Попробуй угадать за 7 попыток.
Попытка 1: 50
Больше.Попытка 2: 75
Меньше.
Попытка 3: 62
Поздравляю! Угадал с 3 попытки!
```

## Задача 4: Сумма цифр числа

**Сложность:** ★☆☆
**Описание:** Посчитай сумму цифр целого положительного числа.

**Требования:**

- Используй `% 10` и `/ 10`, чтобы «откусывать» цифры.
- Пример: `123` → `1 + 2 + 3 = 6`.

**Подсказка:** Строковый способ (`n.ToString().Sum(...)`) тоже допустим, но попробуй арифметический.

## Задача 5: Пирамида из символов

**Сложность:** ★★☆
**Описание:** Выведи пирамиду из `*` высотой N (вводится с клавиатуры).

**Требования:**

- На высоте 3:

```
  *
 ***
*****
```

- Используй вложенные циклы для пробелов и звёздочек.

## Задача 6: Поиск простых чисел (решето)

**Сложность:** ★★★
**Описание:** Найди все простые числа до N (вводится с клавиатуры) и выведи их.

**Требования:**

- Используй решето Эратосфена: массив `bool`, постепенное «вычёркивание».
- Ограничение: N от 2 до 1000.

## Решения

▶ Задача 1 — решение

▶ Задача 2 — решение

▶ Задача 3 — решение

▶ Задача 4 — решение

▶ Задача 5 — решение

▶ Задача 6 — решение

# Разбор кода

# Разбор кода — Модуль 02

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: Стрелочная кодировка (`else` после `return`)

### Исходный код

```
 string CheckAccess(int age)
{
    if (age >= 18)
    {
        return "Доступ разрешён";
    }
    else
    {
        return "Доступ запрещён";
    }
}
```

### Что не так

- `else` после `return` избыточен: если условие истинно, метод уже завершился. Остаётся лишний уровень вложенности.

### Как лучше

```
 string CheckAccess(int age)
{
    if (age >= 18)
        return "Доступ разрешён";

    return "Доступ запрещён";
}
```

### Вывод

Ранний выход `return` убирает `<else` и делает логику плоской. Чем меньше вложенности — тем легче читать.

## Пример 2: Дублирование условий в цепочке

### Исходный код

```
 if (score >= 90)
{
    grade = "A";
}
else if (score >= 80)
{
    grade = "B";
}
else if (score >= 70)
{
    grade = "C";
}
else if (score >= 60)
{
    grade = "D";
}
else
{
    grade = "F";
}
```

### Что не так

- Проверки дублируют границы ( `>= 90` , затем `>= 80` ). Порядок критичен, легко ошибиться, всё вручную.

### Как лучше

```
 string grade = score switch
{
    >= 90 => "A",
    >= 80 => "B",
    >= 70 => "C",
    >= 60 => "D",
    _ => "F"
};
```

### Вывод

`switch` -выражение с реляционными паттернами — выразительнее и не даёт дублировать границы.

## Пример 3: Бесконечный цикл без управляемого выхода

### Исходный код

```
while (true)
{
    Console.WriteLine("Меню: 1 – Каталог, 2 – Выход");
    string input = Console.ReadLine() ?? "";

    if (input == "1")
    {
        Console.WriteLine("Каталог...");
        // программа зависнет после выбора 1
    }
}
```

### Что не так

- Ветка «Выход» не обрабатывается вообще — пользователь не может выйти, цикл вечный.

### Как лучше

```
while (true)
{
    Console.WriteLine("Меню: 1 – Каталог, 2 – Выход");
    string input = Console.ReadLine() ?? "";

    switch (input)
    {
        case "1":
            Console.WriteLine("Каталог...");
            break;
        case "2":
            return; // выход из программы
        default:
            Console.WriteLine("Неверная команда");
            break;
    }
}
```

### Вывод

Вечный цикл обязан иметь гарантированный выход. `return` из `Main` — чистый способ завершить программу.

## Пример 4: Глубокая вложенность

### Исходный код

```
 if (isUser)
{
    if (isAdmin)
    {
        if (isActive)
        {
            Console.WriteLine("Панель администратора");
        }
    }
}
```

### Что не так

- Три уровня вложенности. Добавишь ещё условие — код утонет в скобках. «А»-логика прячется в глубине.

### Как лучше

```
 if (!isUser || !isAdmin || !isActive)
{
    return;
}

Console.WriteLine("Панель администратора");
```

### Вывод

Guard clauses инвертируют условия и выходят рано. Условие «все признаки есть» становится единственным, что остаётся после проверок.

## Пример 5: Сложное условие с `&&` / `||`

### Исходный код

```
if ((age >= 18 && age <= 60 && !isStudent) || (age >= 60 && age <= 80 && isPensioner))
```

### Что не так

- В одно выражение навалено всё. Читаемость нулевая, трудно отлаживать.

### Как лучше

```
 bool IsEligible()
{
    if (age is >= 18 and <= 60 && !isStudent)
        return true;

    if (age is >= 60 and <= 80 && isPensioner)
        return true;

    return false;
}
```

### Вывод

Разбивай сложные условия на проверяемые куски. Реляционные паттерны `is >= 18 and <= 60` читаются как «в диапазоне».

# Модуль 03: Методы

> **Цель:** разбивать код на переиспользуемые методы, правильно передавать параметры и понимать область видимости.

## Зачем это нужно

Представь, что тебе нужно посчитать площадь окружности в пяти местах программы. Без методов ты напишешь одну и ту же формулу пять раз. Напишешь метод — и вызываешь его где угодно, а исправление формулы происходит в одном месте. Методы — главный инструмент **декомпозиции**: разбиения большой задачи на маленькие понятные куски.

## Теория

### 1. Зачем нужны методы

Метод — именованный именованный блок кода, который можно вызывать многократно. Зачем это нужно:

- **Переиспользование** — не дублируем код.
- **Читаемость** — имя метода объясняет, что он делает.
- **Тестируемость** — маленький метод легко проверить отдельно.

### 2. Синтаксис

```
 double AreaOfCircle(double radius)
{
    const double Pi = 3.14159;
    return Pi * radius * radius;
}
```

Вызов:

```
 double area = AreaOfCircle(5);
Console.WriteLine(area);
```

Части сигнатуры: модификатор (по умолчанию `private`), возвращаемый тип, имя, параметры в скобках, тело.

### 3. `void` — метод, ничего не возвращающий

```
void PrintGreeting(string name)
{
    Console.WriteLine($"Привет, {name}!");
}
```

### 4. Параметры по значению (по умолчанию)

```
void Change(int x)
{
    x = 100; // меняем копию
}

int a = 5;
Change(a);
Console.WriteLine(a); // 5 — не изменилось!
```

Для ссылочных типов (строка, объект) передаётся ссылка — но присвоение параметру новой ссылки не влияет на вызывающего.

### 5. `ref`, `out`, `in`

**`out`** — метод обязан присвоить значение параметру:

```
bool TryParsePositive(string s, out int result)
{
    if (int.TryParse(s, out int value) && value > 0)
    {
        result = value;
        return true;
    }

    result = 0;
    return false;
}
```

```
if (TryParsePositive("42", out int n))
{
    Console.WriteLine(n); // 42
}
```

**`ref`** — передаём по ссылке, метод может изменить оригинал:

```
void Swap(ref int a, ref int b)
{
    (a, b) = (b, a);
}
```

```
int x = 1, y = 2;
Swap(ref x, ref y);
Console.WriteLine($"{x} {y}"); // 2 1
```

**`in`** — ссылка только для чтения (эффективность для крупных структур), редко используется новичками.

### 6. `params` — переменное число аргументов

```
int Sum(params int[] numbers)
{
    int total = 0;
    foreach (int n in numbers) total += n;
    return total;
}
```

```
Console.WriteLine(Sum(1, 2, 3));     // 6
Console.WriteLine(Sum(1, 2, 3, 4, 5)); // 15
```

### 7. Именованные и необязательные параметры

```
void Order(string item, int quantity = 1, string? note = null)
{
    Console.WriteLine($"{item}: x{quantity}");
    if (note is not null) Console.WriteLine($"Примечание: {note}");
}
```

```
Order("Пицца");                      // quantity=1, note=null
Order("Пицца", quantity: 3, note: "Без лука"); // именованные аргументы
```

### 8. Перегрузка методов (overloading)

Одинаковое имя, разные параметры:

```
int Add(int a, int b) => a + b;
double Add(double a, double b) => a + b;
int Add(int a, int b, int c) => a + b + c;
```

Компилятор выбирает подходящую перегрузку по типам аргументов.

## 9. Область видимости

```
 int y = 10; // видима во всём методе

void LocalScope()
{
    int x = 5; // видима только внутри LocalScope
    Console.WriteLine(x);
}
// здесь x недоступен
```

Переменные из внешних блоков (например, объявленные в `Main` до вызова) в методах не видны — в метод нужно явно передать их параметрами.

## 10. Статические методы

`static` означает: метод принадлежит типу, а не конкретному объекту. Вызывается через имя класса:

```
 Console.WriteLine(Math.Max(5, 10)); // Math — класс, Max — статический метод
```

В `Program` на верхнем уровне все наши методы — статические (или локальные функции, см. ниже).

**Локальные функции** — методы внутри методов, полезны для вспомогательной логики:

```
 int FactorialLocal(int n)
{
    int Mul(int a, int b) => a * b; // локальная функция

    int result = 1;
    for (int i = 2; i <= n; i++) result = Mul(result, i);
    return result;
}
```

## 11. Возврат нескольких значений через tuple

```
 (int Min, int Max) FindMinMax(int[] numbers)
{
    int min = numbers[0], max = numbers[0];
    foreach (int n in numbers)
    {
```

```
        if (n < min) min = n;
        if (n > max) max = n;
    }
    return (min, max);
}
```

```
var (min, max) = FindMinMax(new[] { 3, 1, 4, 1, 5 });
Console.WriteLine($"{min}..{max}"); // 1..5
```

### 12. Рекурсия

Метод вызывает сам себя. Обязателен **базовый случай** — условие выхода.

```
long Factorial(int n)
{
    if (n <= 1) return 1;        // базовый случай
    return n * Factorial(n - 1); // рекурсивный шаг
}
```

Каждый рекурсивный вызов кладётся на стек. Слишком глубокая рекурсия → `StackOverflowException`. Для факториала/Фибоначчи рекурсия красива, но, например, наивный Фибоначчи экспоненциально медленный.

### 13. Методы расширения (анонс)

Методы расширения позволяют «добавить» метод к существующему типу. Подробно разберём в модуле 10 (LINQ), а пока просто знай, что это мощный механизм C#.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Метод-«монстр» на 100 строк | Невозможно тестировать и понять | разбить на несколько маленьких методов |
| Дублирование логики в нескольких методах | Правка в одном месте забывается в другом | вынести в один общий метод |
| Нет guard clauses | Глубокая вложенность `if` | ранний `return` при неверных данных |
| `out`-параметр там, где читабельнее tuple | Код выглядит как форточка | вернуть кортеж |
| Магические значения в теле | Менять больно | параметры или именованные константы |

## Контрольные вопросы

1. Как объявить метод, ничего не возвращающий?
2. В чём отличие `ref` от `out`?
3. Что делает `params`?
4. Что такое перегрузка методов? Приведи пример.
5. Что такое область видимости?
6. Что такое локальная функция?
7. Как вернуть два значения из метода?
8. Что такое базовый случай рекурсии?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 03

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Утилиты для строк

**Сложность:** ★☆☆
**Описание:** Напиши методы: `IsPalindrome(string)` (палиндром — читается одинаково в обе стороны), `ReverseString(string)`.

**Требования:**

- `IsPalindrome` без учёта регистра и пробелов.
- Проверь на: `"Аргентина манит негра"`.

**Подсказка:** `string.Join("", s.Split(' '))` убирает пробелы; сравнение без учёта регистра — через `ToLower()`.

## Задача 2: GCD (наибольший общий делитель)

**Сложность:** ★☆☆
**Описание:** Реализуй алгоритм Евклида через цикл (не рекурсию).

**Требования:**

- Метод `int GCD(int a, int b)`.
- `GCD(48, 18) == 6`.

**Подсказка:** `while (b != 0) { int t = b; b = a % b; a = t; }`.

## Задача 3: `params`-сумма

**Сложность:** ★☆☆
**Описание:** Напиши метод `Sum(params int[] numbers)` и вызови его с разным числом аргументов.

**Требования:**

- `Sum(1, 2, 3)` → `6`, `Sum(10, 20)` → `30`.

- Покажи, что метод работает с любым числом аргументов.

## Задача 4: Обмен через `ref`

**Сложность:** ★★☆
**Описание:** Напиши метод `Swap(ref int a, ref int b)` и докажи, что он меняет оригиналы.

**Требования:**

- После `Swap(ref x, ref y)` значения `x` и `y` меняются местами.
- Объясни в комментарии, почему без `ref` это не сработает.

## Задача 5: Рекурсивная сумма цифр

**Сложность:** ★★☆
**Описание:** Реализуй вычисление суммы цифр числа рекурсией.

**Требования:**

- Метод `int SumOfDigits(int n)`.
- Базовый случай: `n < 10` → `n`.
- Пример: `SumOfDigits(1234) == 10`.

**Подсказка:** `n % 10 + SumOfDigits(n / 10)`.

## Задача 6: Бинарный поиск

**Сложность:** ★★★
**Описание:** Реализуй бинарный поиск по отсортированному массиву. Верни индекс элемента или `-1`, если его нет.

**Требования:**

- Метод `int BinarySearch(int[] sorted, int target)`.
- Массив отсортирован по возрастанию.
- Пустой массив → `-1`.

**Подсказка:** Поддерживай `left` и `right`; средина `mid = (left + right) / 2`; сужай диапазон в два раза на каждом шаге.

## Задача 7: Перегрузка `Area`

**Сложность:** ★★☆
**Описание:** Реализуй три перегрузки метода `Area` для круга, прямоугольника и треугольника.

**Требования:**

- `Area(double radius)` — круг ($\pi r^2$).
- `Area(double width, double height)` — прямоугольник.
- `Area(double a, double b, double c)` — треугольник по формуле Герона.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

▶ **Задача 7 — решение**

# Разбор кода

# Разбор кода — Модуль 03

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: God-метод

### Исходный код

```
 void ProcessOrder(string orderId)
{
    // 1. Валидация
    // 2. Расчёт скидок
    // 3. Проверка остатков
    // 4. Оформление
    // 5. Отправка e-mail
    // ... 80 строк логики в одном методе
}
```

### Что не так

- Метод делает пять разных вещей. Его нельзя протестировать по частям; одну ошибку сложно найти; команда конфликтует при правках.

### Как лучше

```
 void ProcessOrder(string orderId)
{
    if (!ValidateOrder(orderId)) return;
    decimal total = CalcTotal(orderId);
    if (!CheckStock(orderId)) return;
    PlaceOrder(orderId, total);
    NotifyCustomer(orderId);
}
```

### Вывод

Один метод — одно действие. Имя метода должно полностью описывать то, что он делает.

## Пример 2: Дублирование логики

### Исходный код

```
 void SaveUser(string name)
{
    if (string.IsNullOrWhiteSpace(name))
    {
        Console.WriteLine("Имя не должно быть пустым");
        return;
    }
    // сохраняем
}

void UpdateUser(string name)
{
    if (string.IsNullOrWhiteSpace(name))
    {
        Console.WriteLine("Имя не должно быть пустым");
        return;
    }
    // обновляем
}
```

### Что не так

- Проверка продублирована. Захочешь поменять сообщение или порог — придётся менять в двух местах (и забыть в одном).

### Как лучше

```
 bool IsValidName(string name) => !string.IsNullOrWhiteSpace(name);

void SaveUser(string name)
{
    if (!IsValidName(name)) { Console.WriteLine("Имя не должно быть пустым"); return; }
    // сохраняем
}

void UpdateUser(string name)
{
    if (!IsValidName(name)) { Console.WriteLine("Имя не должно быть пустым"); return; }
    // обновляем
}
```

### Вывод

Общее правило — в общий метод. Дублирование — источник рассинхронизации поведения.

## Пример 3: Отсутствие guard clauses

### Исходный код

```
 double Divide(double a, double b)
{
    if (b != 0)
    {
        return a / b;
    }
    else
    {
        throw new DivideByZeroException();
    }
}
```

### Что не так

- Основной путь спрятан внутри `if`, а обработка ошибки — в `else`. Читается тяжело.

### Как лучше

```
 double Divide(double a, double b)
{
    if (b == 0) throw new DivideByZeroException();
    return a / b;
}
```

### Вывод

Исключительные случаи обрабатывай первыми и выходи рано. Основная логика всегда остаётся плоской.

## Пример 4: `out`-параметр для понятного результата

### Исходный код

```
 void SplitFullName(string full, out string first, out string last)
{
    var parts = full.Split(' ', 2);
    first = parts.Length > 0 ? parts[0] : "";
```

```
    last = parts.Length > 1 ? parts[1] : "";
}

SplitFullName("Иван Иванов", out string f, out string l);
Console.WriteLine($"{f} / {l}");
```

### Что не так

- Кортеж читается яснее: метод возвращает одно значение, а не «протыкает дыры» через `out`. Плюс без `out` проще использовать в выражениях.

### Как лучше

```
 (string First, string Last) SplitFullName(string full)
{
    var parts = full.Split(' ', 2);
    return (parts.Length > 0 ? parts[0] : "", parts.Length > 1 ? parts[1] : "");
}

var (first, last) = SplitFullName("Иван Иванов");
Console.WriteLine($"{first} / {last}");
```

### Вывод

Предпочитай возврат кортежа `out`-параметрам — код становится декларативнее.

## Пример 5: Магические числа в теле

### Исходный код

```
 double BikePrice(int typeCode)
{
    if (typeCode == 1) return 25000;
    if (typeCode == 2) return 35000;
    return 15000;
}
```

### Что не так

- `25000`, `35000`, `15000` — непонятные цены. `1`, `2` — непонятные коды. Изменение цены заставляет искать по всему проекту.

### Как лучше

```
 const decimal KidsBikePrice = 15000m;
const decimal CityBikePrice = 25000m;
const decimal MountainBikePrice = 35000m;

decimal BikePrice(int typeCode) => typeCode switch
{
    1 => CityBikePrice,
    2 => MountainBikePrice,
    _ => KidsBikePrice
};
```

### Вывод

Именованные константы и `switch`-выражение делают назначение чисел явным.

# Модуль 04: ООП — классы и инкапсуляция

> **Цель:** проектировать собственные типы данных через классы, скрывать внутренности (инкапсуляция) и грамотно управлять состоянием.

## Зачем это нужно

До сих пор мы работали с примитивами: `int`, `string`. Но в реальном мире данные имеют структуру: у человека есть имя, возраст и почта; у счёта — номер и баланс; у заказа — список позиций и сумма. Если хранить всё в отдельных переменных, код быстро превращается в кашу. Классы позволяют сгруппировать данные и поведение в один тип — это ООП (объектно-ориентированное программирование).

## Теория

### 1. Четыре столпа ООП

| Столп | Идея | Где разберём |
|---|---|---|
| **Инкапсуляция** | скрыть внутреннее состояние, дать доступ через методы | этот модуль |
| **Наследование** | общие свойства вынести в базовый класс | модуль 05 |
| **Полиморфизм** | один интерфейс — разные реализации | модуль 05 |
| **Абстракция** | скрыть сложность за понятным интерфейсом | модуль 05/14 |

### 2. Класс и объект

**Класс** — чертёж (тип). **Объект** — конкретный экземпляр по этому чертёжу.

```
class Person
{
    public string Name;   // поле
    public int Age;
}
```

Создание объектов:

```
Person p1 = new Person { Name = "Иван", Age = 30 };
Person p2 = new Person { Name = "Мария", Age = 28 };
```

## 3. Поля и свойства

**Поля** — обычные переменные в классе. Но поля напрямую раскрывать наружу не принято — для этого есть свойства.

**Свойства** — пара методов-аксессоров:

```
 class BankAccount
{
    private decimal _balance; // поле: скрыто

    public decimal Balance       // свойство: публичный доступ
    {
        get => _balance;
        private set => _balance = value;
    }

    public decimal Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        _balance += amount;
        return _balance;
    }
}
```

**Auto-property** — свойство без явно объявленного поля:

```
 public string Name { get; set; }   // читать и записывать
public string Email { get; init; } // можно задать только при создании
```

## 4. Модификаторы доступа

| Модификатор | Где виден |
|---|---|
| `public` | везде |
| `private` | только внутри класса |
| `protected` | внутри класса и наследников (модуль 05) |
| `internal` | внутри сборки |

**Золотое правило:** поле/состояние должно быть `private`; наружу отдавай свойства и методы. Это и есть инкапсуляция — никто не может испортить баланс, минуя `Deposit`.

## 5. Конструкторы

Конструктор — метод, вызываемый при создании объекта. Имя совпадает с именем класса, возвращаемого типа нет.

```
class Person
{
    public string Name { get; }

    public Person(string name)
    {
        if (string.IsNullOrWhiteSpace(name))
            throw new ArgumentNullException(nameof(name));

        Name = name;
    }
}
```

**Перегрузка конструкторов и `this()`:**

```
class Point
{
    public int X { get; }
    public int Y { get; }

    public Point(int x, int y) { X = x; Y = y; }
    public Point() : this(0, 0) { } // делегируем «центральному» конструктору
}
```

**Object initializer** — задание свойств при создании:

```
var p = new Person("Иван") { Age = 30 };
```

## 6. `readonly`, `const`, `static readonly`

- `const` — константа времени компиляции, часть типа.
- `static readonly` — значение создаётся один раз при первом обращении к типу.

```
class Config
{
    public const int MaxRetries = 3;
    public static readonly DateTime Epoch = new DateTime(1970, 1, 1);
}
```

## 7. Статические члены

Статические члены принадлежат типу, а не объекту. Общие для всех экземпляров.

```
class Counter
{
    public static int Created;       // сколько всего объектов создано
    public Counter() => Created++;
}
```

```
var a = new Counter();
var b = new Counter();
Console.WriteLine(Counter.Created); // 2
```

**Статические классы** — нельзя создавать объекты; только статические члены. Примеры в BCL: `Math`, `File`, `Console`.

```
static class TemperatureConverter
{
    public static double CelsiusToFahrenheit(double c) => c * 9 / 5 + 32;
}
```

### 8. `record` — введение

`record` — ссылочный тип с value-равенством (модуль 06). Компилятор сам генерирует `Equals`, `GetHashCode`, `ToString`.

```
record Person(string Name, int Age);
```

```
var p1 = new Person("Иван", 30);
var p2 = new Person("Иван", 30);
Console.WriteLine(p1 == p2); // True — по значениям, а не по ссылке
```

### 9. Nullable reference types

Модификатор `?` сообщает: «может быть null».

```
string? maybe = null;       // может быть null
string required = "текст";   // не должен быть null
```

Компилятор предупредит, если ты разыменуешь `maybe` без проверки. Операторы `??` и `?.` уже знакомы из модуля 01.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| публичные поля `public int Balance` | состояние меняют напрямую, валидация обходится | свойства с `private set` или методы |
| «проваливание» инкапсуляции | наружу отдаётся внутренняя коллекция, которую меняют | `IReadOnlyList<T>` (модуль 07) |
| вся логика в `Program` | God-класс, нетестируемо | разбить на классы по смыслу |
| общий изменяемый `static`-стат | гонки, неожиданные изменения из любого места | использовать осторожно, только по делу |
| дублирование кода в конструкторах | лишние повторы | делегирование через `this()` |

## Контрольные вопросы

1. В чём разница между классом и объектом?
2. Чем свойство отличается от поля?
3. Что такое инкапсуляция и зачем она?
4. Какие бывают модификаторы доступа?
5. Что такое конструктор? Зачем `this()`?
6. Чем `const` отличается от `static readonly`?
7. Что такое статический член?
8. Что такое record в двух словах?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 04

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: `BankAccount`

**Сложность:** ★☆☆
**Описание:** Класс банковского счёта с защищённым балансом.

**Требования:**

- Поле баланс — `private`.
- `Deposit(decimal amount)` — пополнение; отрицательное/нулевое — `ArgumentOutOfRangeException`.
- `Withdraw(decimal amount)` — снятие; нельзя уйти в минус.
- Свойство `Balance` — только чтение.

**Подсказка:** `private set` в свойстве или просто `get`-свойство с приватным полем.

## Задача 2: `Point`

**Сложность:** ★☆☆
**Описание:** Класс точки на плоскости с методом `DistanceTo`.

**Требования:**

- `X`, `Y` — свойства только для чтения (`init` или `private set`).
- `double DistanceTo(Point other)` — евклидово расстояние.

**Подсказка:** `Math.Sqrt((X-other.X)^2 + (Y-other.Y)^2)`.

## Задача 3: `Student`

**Сложность:** ★★☆
**Описание:** Класс студента с валидацией в свойствах.

**Требования:**

- `Name` — непустая строка (иначе `ArgumentException`).

- `Age` — от 14 до 100 (иначе `ArgumentOutOfRangeException`).
- `Gpa` (средний балл) — от 0 до 5.

## Задача 4: `record` + `with`

**Сложность:** ★☆☆
**Описание:** Используй `record Book(string Title, string Author, int Year)`.

**Требования:**

- Создай экземпляр, склонируй через `with`, изменив год.
- Покажи, что `==` сравнивает по значениям.

## Задача 5: Статический класс `TemperatureConverter`

**Сложность:** ★☆☆
**Описание:** Статический класс с двумя методами для температур.

**Требования:**

- `CelsiusToFahrenheit(double)` и `FahrenheitToCelsius(double)`.
- Вызови из `Program` без создания объекта.

## Задача 6: `Countdown`-таймер

**Сложность:** ★★★
**Описание:** Класс обратного отсчёта с инкапсуляцией.

**Требования:**

- `Remaining` — свойство (чтение), приватный сеттер.
- `Tick()` — уменьшает на 1; при 0 бросает `InvalidOperationException`.
- `Reset(int seconds)` — устанавливает новое значение (только положительное).

## Решения

▶ Задача 1 — решение

▶ Задача 2 — решение

▶ Задача 3 — решение

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 04

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: Публичные поля вместо свойств

### Исходный код

```
class BankAccount
{
    public decimal Balance;
}
```

И тут начинается хаос:

```
var acc = new BankAccount { Balance = -500 }; // отрицательный баланс!
acc.Balance = 999999;                         // кто угодно меняет
```

### Что не так

- Инкапсуляция отсутствует: поле можно менять напрямую, валидацию обойти.

### Как лучше

```
class BankAccount
{
    private decimal _balance;
    public decimal Balance => _balance;
    // Deposit / Withdraw с проверками
}
```

### Вывод

Внутреннее состояние — `private`. Наружу — только контролируемые методы и свойства.

## Пример 2: «Проваливание» инкапсуляции

### Исходный код

```
class Cart
{
    public List<string> Items { get; } = new();
}

var cart = new Cart();
cart.Items.Add("Pizza"); // модификация извне без контроля
cart.Items.Clear();      // всё стёрто
```

### Что не так

- Публичное `List<T>` отдаёт доступ на изменение внутренней коллекции. «Защита» ничего не защищает.

### Как лучше

```
class Cart
{
    private readonly List<string> _items = new();
    public IReadOnlyList<string> Items => _items;

    public void Add(string item)
    {
        if (string.IsNullOrWhiteSpace(item)) return;
        _items.Add(item);
    }
}
```

### Вывод

Отдавай наружу `IReadOnlyList<T>` (подробнее — модуль 07). Мутация — только через методы.

## Пример 3: God-класс `Program`

### Исходный код

```
// Program.cs — 500 строк: и меню, и расчёты, и сохранение
var action = Console.ReadLine();
// ... если action == "1" то считаем налог, если "2" сохраняем, ...
```

### Что не так

- Один класс делает всё: ввод, бизнес-логику, хранение. Тестировать и поддерживать невозможно.

### Как лучше

```
// Меню — в ConsoleUI
// Расчёты — в TaxCalculator
// Сохранение — в FileStorage
// Program.cs только связывает их
```

### Вывод

Каждый класс — одна ответственность (подробнее — модуль 14, SRP).

## Пример 4: Общий изменяемый `static`

### Исходный код

```
static class Global
{
    public static int Counter = 0;
}

// в 10 местах: Global.Counter++;
// в 3 местах: Global.Counter = 0;
```

### Что не так

- Состояние общее для всей программы; его меняют непредсказуемо. Трудно отлаживать.

### Как лучше

```
class Counter
{
    private int _value;
    public void Increment() => _value++;
    public void Reset() => _value = 0;
    public int Value => _value;
}
```

### Вывод

Изменяемое состояние — в конкретных объектах, а не в глобальных статических полях.

## Пример 5: Дублирование в конструкторах

### Исходный код

```
 class Rectangle
{
    public int Width { get; }
    public int Height { get; }

    public Rectangle(int width, int height)
    {
        if (width <= 0 || height <= 0) throw new ArgumentOutOfRangeException();
        Width = width;
        Height = height;
    }

    public Rectangle() // копия логики валидации
    {
        Width = 1;
        Height = 1;
    }
}
```

### Что не так

- Валидация продублирована, значения по умолчанию зашиты в двух местах.

### Как лучше

```
 public Rectangle(int width = 1, int height = 1)
{
    if (width <= 0 || height <= 0) throw new ArgumentOutOfRangeException();
    Width = width;
    Height = height;
}
```

### Вывод

Необязательные параметры с дефолтами вытесняют вспомогательные конструкторы.

# Модуль 05: Наследование и полиморфизм

> **Цель:** строить иерархии типов, переопределять поведение и правильно выбирать между наследованием и интерфейсами.

## Зачем это нужно

Когда в программе появляются «родственные» типы — круг и квадрат, кошка и собака, сберегательный и кредитный счёт, — хочется переиспользовать общий код и обращаться с ними единообразно. Это и дают **наследование** и **полиморфизм**. Без них приходится писать кучу `if`-ов по типу, что хрупко и разрастётся.

## Теория

### 1. Наследование

Класс-наследник получает члены базового класса.

```
class Animal
{
    public string Name { get; set; }

    public void Eat() => Console.WriteLine($"{Name} ест");
}

class Dog : Animal
{
    public void Bark() => Console.WriteLine("Гав!");
}
```

```
var dog = new Dog { Name = "Рекс" };
dog.Eat();   // из базового класса
dog.Bark();  // своё
```

**Ограничение:** в C# только **одиночное** наследование — один базовый класс.

#### Конструкторы и `base`

```
class Animal
{
```

```
    public string Name { get; }
    public Animal(string name) => Name = name;
}

class Dog : Animal
{
    public Dog(string name) : base(name) { }
}
```

Наследник обязан вызвать базовый конструктор через `base(...)`.

## 2. Виртуальные методы: `virtual` / `override`

```
 class Animal
{
    public virtual string Speak() => ".....";
}

class Dog : Animal
{
    public override string Speak() => "Гав!";
}
```

- `virtual` — метод можно переопределить.
- `override` — наследник меняет поведение.

## 3. Полиморфизм

Ссылка базового типа может указывать на объект наследника:

```
 Animal a = new Dog();
Console.WriteLine(a.Speak()); // "Гав!" — вызывается переопределённый метод
```

Что важно: **какой метод вызовется, решается в рантайме** по фактическому типу объекта. Это называется виртуальная диспетчеризация.

```
 Animal[] animals = { new Dog(), new Cat(), new Cow() };
foreach (Animal animal in animals)
{
    Console.WriteLine(animal.Speak());
}
```

## 4. Абстрактные классы и методы

`abstract` — класс, который нельзя инстанцировать. Может содержать абстрактные методы — без тела, их обязаны реализовать наследники.

```
abstract class Shape
{
    public abstract double Area();
}

class Circle : Shape
{
    public double Radius { get; }
    public Circle(double radius) => Radius = radius;
    public override double Area() => Math.PI * Radius * Radius;
}
```

```
// var s = new Shape(); // Ошибка: абстрактный класс нельзя создать
Shape s = new Circle(2);
Console.WriteLine(s.Area());
```

## 5. `sealed`

`sealed` — запрещает наследование (или переопределение метода):

```
sealed class Point2D { }
// class Derived : Point2D { } // Ошибка
```

## 6. Переопределение `ToString` и `Equals`

`object` — корень всех типов. Его виртуальные методы можно переопределять:

```
class Person
{
    public string Name { get; }
    public Person(string name) => Name = name;

    public override string ToString() => $"Человек: {Name}";
}
```

```
Console.WriteLine(new Person("Иван")); // "Человек: Иван"
```

Про `Equals` / `GetHashCode` подробнее поговорим в модуле 06 (там же — `record`).

## 7. Интерфейсы

Интерфейс — «контракт»: набор членов, которые обязана реализовать реализация. Члены интерфейса не имеют тела.

```
interface IShape
{
    double Area();
}
```

Класс реализует интерфейс:

```
class Square : IShape
{
    public double Side { get; }
    public Square(double side) => Side = side;
    public double Area() => Side * Side;
}
```

Интерфейсов можно реализовывать **много**:

```
class Bird : IFlyable, IComparable<Bird> { ... }
```

**Явная реализация**

```
interface ILogger
{
    void Log(string message);
}

class ConsoleLogger : ILogger
{
    void ILogger.Log(string message) => Console.WriteLine(message);
}
```

Явная реализация видна только через ссылку на интерфейс. Используется, когда метод не должен быть «на виду» или конфликтует с другим.

## 8. Интерфейс vs абстрактный класс

| Признак | Абстрактный класс | Интерфейс |
|---|---|---|
| Наследование | один | много |
| Состояние (поля) | да | нет (только члены-контракты) |
| Конструкторы | да | нет |

| Признак | Абстрактный класс | Интерфейс |
|---|---|---|
| Когда выбирать | общая реализация + данные | только «что умеет» (контракт) |

**Практическое правило:** если связи «является» (IS-A) и есть общая логика — абстрактный класс. Если нужен только контракт «умеет» (CAN-DO) — интерфейс.

### 9. `is`, `as`, приведение

```
object obj = new Dog();

if (obj is Dog dog)          // pattern matching
{
    dog.Bark();
}

var d = obj as Dog;          // as: null, если тип не подходит
if (d is not null) d.Bark();
```

### 10. IDisposable (анонс)

`IDisposable` — интерфейс с методом `Dispose()`. Его реализуют типы, владеющие ресурсами (файлы, соединения). Подробно разберём в модуле 08.

### 11. LSP — принцип Лисков (интуиция)

Наследник не должен ломать поведение базового класса. Если переопределение бросает исключение там, где базовый класс всегда работает, — иерархия спроектирована плохо. Полноценный разбор — модуль 14.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Наследование ради переиспользования кода | Хрупкая «родственная» связь ради утилит | композиция (поле вместо наследования) |
| `switch` по типу вместо виртуального метода | Новый тип → правка всех switch | виртуальный метод в базовом классе |
| Забыл `override` / не поставил `virtual` | Метод скрывается, а не переопределяется | планируй иерархию заранее |
| Интерфейс с 15 методами | много реализаций тянет лишнее | дробить интерфейсы (модуль 14) |
| Переопределил `Equals`, забыл `GetHashCode` | коллекции будут вести себя странно | переопределять пару вместе |

## Контрольные вопросы

1. Что даёт наследование?
2. Почему в C# нельзя множественное наследование классов?
3. В чём разница `virtual` / `override` и просто нового метода?
4. Что такое полиморфизм?
5. Чем абстрактный класс отличается от интерфейса?
6. Что делает `sealed`?
7. Что такое явная реализация интерфейса?
8. Что даёт pattern matching `is`?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 05

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Иерархия `Shape`

**Сложность:** ★★☆
**Описание:** Абстрактный класс `Shape` с абстрактным `Area()`. Наследники `Circle`, `Rectangle`, `Triangle`.

**Требования:**

- `Shape` — `abstract`, метод `Area()` — `abstract`.
- `Circle` — по радиусу; `Rectangle` — по сторонам; `Triangle` — по Герону.
- Создай массив `Shape[]` и вычисли сумму площадей через полиморфизм.

## Задача 2: Интерфейс `IPayable`

**Сложность:** ★★☆
**Описание:** Интерфейс для платежей, реализуют несколько типов.

**Требования:**

- `IPayable` с `decimal CalculateAmount()`.
- `Salary` (фикс. оклад) и `Contract` (ставка × часы).

**Подсказка:** `decimal` для денег.

## Задача 3: Животные

**Сложность:** ★☆☆
**Описание:** `Animal` с `virtual Speak()`, наследники `Dog`, `Cat`, `Cow` переопределяют.

**Требования:**

- Список `List<Animal>` с разными животными; выведи `Speak()` по каждому.

## Задача 4: Явная реализация `IComparable<Person>`

**Сложность:** ★★★
**Описание:** Сортировка людей по возрасту.

**Требования:**

- Класс `Person` + реализация `IComparable<Person>` (по возрасту).
- Массив людей, `Array.Sort`, вывод по возрастанию.

**Подсказка:** `CompareTo` возвращает отрицательное, ноль или положительное.

## Задача 5: Банк

**Сложность:** ★★☆
**Описание:** Интерфейс `IBankAccount` + абстрактный базовый класс `BankAccount` и `SavingsAccount` с процентами.

**Требования:**

- `IBankAccount`: `Deposit`, `Withdraw`, `Balance`.
- `BankAccount` — базовый, `SavingsAccount` добавляет `ApplyInterest(double rate)`.

## Задача 6: `sealed` vs наследование

**Сложность:** ★☆☆
**Описание:** Покажи разницу между `sealed` и обычным классом.

**Требования:**

- Создай `sealed class Point2D { }`.
- Покажи в комментарии, почему он не может иметь наследников и когда это надо.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 05

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: Наследование ради переиспользования

### Исходный код

```
 class FileLogger
{
    public void Log(string message) => Console.WriteLine($"LOG: {message}");
}

class ReportGenerator : FileLogger
{
    public void Generate() => Log("Отчёт сгенерирован");
}
```

### Что не так

- `ReportGenerator` не «является» логгером (IS-A). Связь слабая, но наследование делает её намертво: любой наследник теперь «будет» логгером. Лучше — композиция.

### Как лучше

```
 class ReportGenerator
{
    private readonly FileLogger _logger = new();

    public void Generate() => _logger.Log("Отчёт сгенерирован");
}
```

### Вывод

Используй наследование только там, где есть настоящая связь «является». Для инструментов — композиция.

## Пример 2: `switch` по типу вместо виртуального метода

### Исходный код

```
 using PrintArea(Shape shape)
{
    if (shape is Circle c) return Math.PI * c.Radius * c.Radius;
    if (shape is Rectangle r) return r.Width * r.Height;
    if (shape is Triangle t) { /* формула Герона */ }
    return 0;
}
```

### Что не так

- Каждая новая фигура → правка `PrintArea`. Логика «размазана» по потребителям.

### Как лучше

```
 abstract class Shape
{
    public abstract double Area();
}
```

### Вывод

Классический случай OCP/полиморфизма: новое поведение добавляется новым классом, а не правкой старых.

## Пример 3: Забыт `virtual` / `override`

### Исходный код

```
 class Animal { public string Speak() => "..."; }
class Dog : Animal { public new string Speak() => "Гав!"; }

Animal a = new Dog();
Console.WriteLine(a.Speak()); // "..." — метод скрыт, а не переопределён
```

### Что не так

- `new` скрывает метод, но через базовую ссылку вызывает версия базового класса. Полиморфизм не работает.

### Как лучше

```
class Animal { public virtual string Speak() => "..."; }
class Dog : Animal { public override string Speak() => "Гав!"; }
```

### Вывод

Используй `virtual` / `override` — тогда вызов пойдёт к фактической реализации.

## Пример 4: Жирный интерфейс

### Исходный код

```
interface IWorker
{
    void Work();
    void Eat();
    void Sleep();
    void PrintReport();
    void SendEmail();
}
```

### Что не так

- Не все работники отправляют email или спят. Придётся реализовывать «вхолостую» все методы. Нарушение принципа разделения интерфейсов.

### Как лучше

```
interface IWorker { void Work(); }
interface IReportable { void PrintReport(); }
interface INotifier { void SendEmail(); }
```

### Вывод

Интерфейсы дробятся по ролям (подробнее — модуль 14, ISP).

## Пример 5: `Equals` без `GetHashCode`

### Исходный код

```
class Person
{
    public string Name { get; }
```

```
    public Person(string name) => Name = name;

    public override bool Equals(object? obj) =>
        obj is Person other && Name == other.Name;
    // GetHashCode не переопределён
}
```

### Что не так

- Объекты, равные по `Equals`, обязаны иметь одинаковый `GetHashCode`. Иначе поведение в `Dictionary` / `HashSet` сломается.

### Как лучше

```
public override int GetHashCode() => Name.GetHashCode();
```

### Вывод

`Equals` и `GetHashCode` переопределяются парой (или просто используй `record` — он сделает это сам).

# Модуль 06: Продвинутые типы данных

> **Цель:** выбирать подходящий тип (struct/enum/record/tuple), использовать паттерн-матчинг и безопасно работать с nullable.

## Зачем это нужно

К этому моменту ты знаешь классы и интерфейсы. Но C# даёт и другие типы, каждый — под свою задачу: `enum` вместо «магических чисел», `record` для неизменяемых данных, `struct` для мелких value-сущностей, туплы для быстрых многосоставных значений. Плюс паттерн-матчинг — современный способ красиво разбирать данные. Это уровень «сильного среднего» разработчика.

## Теория

### 1. `enum` — перечисления

```
enum OrderStatus
{
    New,
    Paid,
    Shipped,
    Delivered
}
```

Использование:

```
OrderStatus status = OrderStatus.Paid;
switch (status)
{
    case OrderStatus.New: Console.WriteLine("Новый"); break;
    case OrderStatus.Paid: Console.WriteLine("Оплачен"); break;
    default: Console.WriteLine("Другое"); break;
}
```

- По умолчанию базовый тип — `int` (можно сменить).
- `[Flags]` позволяет комбинировать значения (побитово).
- `Enum.TryParse("Paid", out OrderStatus s)` — безопасный разбор строки.

**Зачем:** вместо «магических чисел» — понятные имена, автодополнение, проверки на этапе компиляции.

## 2. `struct` — тип значения

```
struct Point
{
    public int X { get; init; }
    public int Y { get; init; }
}
```

- Значение хранится в переменной напрямую; копируется при присваивании.
- Не может наследоваться (кроме `interface`).
- Идеален для **мелких неизменяемых** сущностей: координаты, RGB-цвет, дата.
- Большие и мутабельные — лучше класс.

**`readonly struct`** — запрет мутации:

```
readonly struct Point
{
    public int X { get; }
    public int Y { get; }
    public Point(int x, int y) { X = x; Y = y; }
}
```

## 3. `record` — value-равенство

`record` — класс с автоматическим `Equals` / `GetHashCode` / `ToString` по значениям.

```
record Person(string Name, int Age);
```

```
var a = new Person("Иван", 30);
var b = new Person("Иван", 30);
Console.WriteLine(a == b);       // True — по значениям
Console.WriteLine(a with { Age = 31 }); // копия с изменением
```

**`record struct`** — record на базе struct:

```
record struct Point(int X, int Y);
```

**Для чего:** DTO, конфигурации, данные, которые сравнивают по содержимому.

## 4. Туплы

```
var t = (X: 10, Y: 20);
```

```
Console.WriteLine(t.X); // 10
```

```
 (int Min, int Max) FindRange(int[] numbers) => (numbers.Min(), numbers.Max());
var (min, max) = FindRange(new[] { 3, 1, 4 });
```

- Удобны для возврата нескольких значений без отдельного класса.

## 5. Nullable value types

```
 int? maybe = null; // System.Nullable<int>
```

```
 int? a = null;
int b = a ?? 42;   // b == 42
Console.WriteLine(a.HasValue); // false
```

Операции с nullable ограничены — проверяй через `??`, `?.` или `hasValue`.

## 6. Паттерн-матчинг (C# 9–12)

**Type pattern:**

```
 if (obj is string s)
{
    Console.WriteLine(s.Length);
}
```

**Property pattern:**

```
 bool IsAdult(Person p) => p is { Age: >= 18 };
```

**Relational + logical:**

```
 string Classify(int n) => n switch
{
    < 0 => "отрицательное",
    > 0 => "положительное",
    _ => "ноль"
};
```

**Combined (and/or/not):**

```
bool IsWorkTime(DayOfWeek day) => day is
    DayOfWeek.Monday or DayOfWeek.Tuesday or DayOfWeek.Wednesday or
```

```
    DayOfWeek.Thursday or DayOfWeek.Friday;
```

**Discards ( `_` ):**

```
_ => "default"
```

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| `class` для мелких value-сущностей | боксинг/аллокации, нет value-семантики | `struct` или `record struct` |
| `struct` для большой мутабельной сущности | копирование дорого, состояние «плывёт» | класс |
| Магические числа вместо enum | непонятно, легко перепутать | enum |
| Мутабельный record ( `set` вместо `init` ) | теряется смысл «неизменяемого снимка» | `init` или позиционный record |
| `dict.GetValueOrDefault` без обработки отсутствия | может вернуть `null` /дефолт | проверяй `TryGetValue` |

## Контрольные вопросы

1. Когда выбрать `struct` , а когда `class` ?
2. Что автоматически генерирует `record` ?
3. Что такое `record struct` ?
4. Как создать тупл с именованными элементами?
5. Что такое nullable-тип и как безопасно работать с ним?
6. Какие бывают виды паттернов?
7. Что делает дискард `_` ?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 06

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: enum-меню

**Сложность:** ★☆☆
**Описание:** Создай `enum MenuAction { Add, List, Remove, Exit }` и прочитай действие с клавиатуры.

**Требования:**

- `Enum.TryParse` для разбора.
- `switch`-выражение для вывода сообщения.

## Задача 2: struct Money

**Сложность:** ★★☆
**Описание:** `readonly struct Money` с валютами и методом сложения.

**Требования:**

- Поля: `decimal Amount`, `string Currency`.
- `Money Add(Money other)` — складывает только если валюта совпадает (иначе `InvalidOperationException`).
- Покажи, что присваивание создаёт копию.

## Задача 3: record с `with`

**Сложность:** ★☆☆
**Описание:** `record Person(string Name, int Age)`.

**Требования:**

- Создай два равных экземпляра, убедись, что `==` вернёт `true`.
- Через `with` создай копию с другим возрастом.

## Задача 4: Деструктуризация туплов

**Сложность:** ★★☆
**Описание:** Метод возвращает минимум, максимум и среднее массива.

**Требования:**

- Верни `(int Min, int Max, double Avg)`.
- Разбери результат через `var (min, max, avg) = ...`.

**Подсказка:** Пустой массив — брось `ArgumentException`.

## Задача 5: Калькулятор на паттернах

**Сложность:** ★★☆
**Описание:** Калькулятор с операторами `+ - * / %` через `switch`-выражение с реляционными паттернами для проверки деления на ноль.

**Требования:**

- Обработай деление на ноль — верни `double.NaN` или брось исключение.

## Задача 6: Обработка nullable

**Сложность:** ★★★
**Описание:** Напиши метод, который ищет максимальное значение в массиве `int?` и безопасно возвращает `int?`.

**Требования:**

- `int? FindMax(int?[] numbers)`.
- Учитывай `null`-элементы: они игнорируются.
- Если все элементы `null` — верни `null`.

## Решения

▶ Задача 1 — решение

▶ Задача 2 — решение

▶ Задача 3 — решение

▶ Задача 4 — решение

▶ Задача 5 — решение

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 06

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: `class` вместо `struct` для мелкой value-сущности

### Исходный код

```
class Point
{
    public int X { get; set; }
    public int Y { get; set; }
}
```

### Что не так

- `Point` — мелкая value-сущность. Как `class`, она ссылочная, копирования по значению нет, добавляет аллокации.

### Как лучше

```
readonly struct Point
{
    public int X { get; }
    public int Y { get; }
    public Point(int x, int y) { X = x; Y = y; }
}
```

### Вывод

Мелкие неизменяемые сущности — `readonly struct` (или `record struct`).

## Пример 2: Магические числа вместо enum

### Исходный код

```
if (orderStatus == 2) { /* оплачен */ }
if (orderStatus == 5) { /* доставлен */ }
```

### Что не так

- `2` и `5` ничего не говорят. Перепутать легко, поддерживать больно.

### Как лучше

```
 enum OrderStatus { New, Paid, Shipped, Delivered }
if (orderStatus == OrderStatus.Paid) { }
```

### Вывод

Enum — типобезопасная замена магических чисел.

## Пример 3: Строка-«статус» вместо enum

### Исходный код

```
 string status = "done";
if (status == "done" || status == "DONE" || status == "Done") { }
```

### Что не так

- Строки непостоянны, сравнение с учётом регистра — источник багов. Нет автодополнения.

### Как лучше

```
 enum OrderStatus { None, Done }
if (status == OrderStatus.Done) { }
```

### Вывод

Enum исключает опечатки и регистр.

## Пример 4: Мутабельный record

### Исходный код

```
 record User
{
    public string Name { get; set; } // set — запись можно менять
}
```

### Что не так

- Смысл `record` — неизменяемый снимок значения. `set` позволяет менять поля извне, ломая value-равенство и `with`-семантику.

### Как лучше

```
 record User(string Name);
// или с init:
record User { public string Name { get; init; } }
```

### Вывод

`record` задуман как неизменяемый; используй `init`.

## Пример 5: `Dictionary.GetValueOrDefault` без обработки отсутствия

### Исходный код

```
 var dict = new Dictionary<string, int> { ["a"] = 1 };
int value = dict.GetValueOrDefault("missing"); // 0, но 0 — легитимное значение
if (value == 0) Console.WriteLine("Не найдено"); // ложная тревога
```

### Что не так

- Нельзя отличить «нет ключа» от «значение 0».

### Как лучше

```
 if (dict.TryGetValue("missing", out int actual))
{
    Console.WriteLine(actual);
}
else
{
    Console.WriteLine("Не найдено");
}
```

### Вывод

`TryGetValue` различает отсутствие ключа и реальное значение.

# Модуль 07: Обобщения и коллекции

> **Цель:** уверенно выбирать и использовать коллекции, понимать обобщения и их ограничения.

## Зачем это нужно

Программы постоянно работают с наборами данных: списки товаров, словари настроек, очереди задач, уникальные множества. В C# коллекции параметризованы через обобщения. Понимание их внутреннего устройства и Big O — то, что отличает «набирающего» от среднего разработчика: это влияет на скорость твоих программ при больших объёмах данных.

## Теория

### 1. Зачем обобщения

Сначала — проблема. «Старые» необобщённые коллекции хранили `object`:

```
// Так было раньше (и так не надо):
var list = new System.Collections.ArrayList();
list.Add(1);
list.Add("текст");   // положили строку в список чисел
int x = (int)list[0]; // ручное приведение + риск исключения
```

Обобщения убирают оба недостатка — типобезопасность и приведение:

```
List<int> numbers = new() { 1, 2, 3 };
numbers.Add(4);
// numbers.Add("текст"); // ошибка компиляции — тип уже известен
```

### 2. Обобщённые методы и классы

```
T Max<T>(T a, T b) where T : IComparable<T> =>
    a.CompareTo(b) >= 0 ? a : b;
```

```
Console.WriteLine(Max(3, 7));       // T = int
Console.WriteLine(Max("a", "z"));   // T = string
```

### 3. Ограничения `where`

| Ограничение | Смысл |
|---|---|
| `where T : class` | только ссылочные типы |
| `where T : struct` | только типы значения |
| `where T : new()` | обязателен конструктор без параметров |
| `where T : ISomeInterface` | реализует интерфейс |
| `where T : BaseClass` | наследник базового класса |

```
void PrintAll<T>(IEnumerable<T> items) where T : class =>
    foreach (var item in items) Console.WriteLine(item);
```

## 4. Основные коллекции

**`List<T>`** — динамический массив.

```
 var users = new List<string> { "Иван", "Мария" };
users.Add("Пётр");
users.Remove("Иван");
Console.WriteLine(users[0]); // Мария
users.Sort();
```

**`Dictionary<TKey, TValue>`** — пары «ключ-значение», быстрый доступ по ключу.

```
 var ages = new Dictionary<string, int>
{
    ["Иван"] = 30
};
ages["Мария"] = 28;

if (ages.TryGetValue("Иван", out int age))
{
    Console.WriteLine(age);
}
```

**`HashSet<T>`** — уникальные элементы, мгновенная проверка принадлежности.

```
 var tags = new HashSet<string> { "csharp", "dotnet" };
tags.Add("csharp");      // уже есть — не добавится
Console.WriteLine(tags.Contains("dotnet")); // True
```

**`Queue<T>`** / **`Stack<T>`** — FIFO/LIFO.

```
var tasks = new Queue<string>();
tasks.Enqueue("Первая");
tasks.Enqueue("Вторая");
var next = tasks.Dequeue(); // "Первая"

var undo = new Stack<string>();
undo.Push("A");
undo.Push("B");
var top = undo.Pop();       // "B"
```

## 5. `IEnumerable<T>` и `foreach`

`IEnumerable<T>` — минимальный контракт перебора. `foreach` работает с любым, кто его реализует.

```
void PrintAll(IEnumerable<int> items)
{
    foreach (int n in items) Console.WriteLine(n);
}

PrintAll(new List<int> { 1, 2, 3 });
PrintAll(new[] { 4, 5 });
```

## 6. Интерфейсы только для чтения

На границах API не отдавай мутабельные коллекции. Отдавай `IReadOnlyList<T>`, `IReadOnlyCollection<T>`, `IReadOnlyDictionary<TKey, TValue>`:

```
class Order
{
    private readonly List<string> _items = new();
    public IReadOnlyList<string> Items => _items; // снаружи не изменить
}
```

## 7. Big O коллекций

| Коллекция | Доступ по индексу/ключу | Поиск | Добавление |
|---|---|---|---|
| `List<T>` | O(1) | O(n) | O(1) в конец, O(n) в середину |
| `Dictionary<TKey,TValue>` | O(1) | O(1) | O(1) |
| `HashSet<T>` | нет | O(1) | O(1) |

| Коллекция | Доступ по индексу/ключу | Поиск | Добавление |
|---|---|---|---|
| `Queue<T>` / `Stack<T>` | нет | O(n) | O(1) |
| отсортированные `SortedSet` / `SortedDictionary` | O(log n) | O(log n) | O(log n) |

### 8. Коллекционные выражения (C# 12)

```
int[] a = [1, 2, 3];
List<int> b = [1, 2, 3];
int[] c = [.. a, .. b, 4]; // оператор spread
```

### 9. `yield return` (анонс)

Позволяет создавать `IEnumerable<T>` «лениво», без материализации. Подробно разберём в модуле 10.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| `List<T>` там, где нужен `HashSet<T>` и наоборот | разная сложность поиска | выбирай по операции |
| Публичное `List<T>` в API | мутация извне | `IReadOnlyList<T>` |
| `Dictionary` перебор по ключу вместо `TryGetValue` | O(n) вместо O(1), некрасиво | `TryGetValue` |
| Изменение коллекции в `foreach` | `InvalidOperationException` | собрать список изменений, затем применить |
| `Search` -перебор `List.Contains` в цикле вложенном | $O(n^2)$ | `HashSet` |

## Контрольные вопросы

1. Что решают обобщения?
2. Какие ограничения `where` бывают?
3. Чем `List<T>` отличается от `Dictionary<TKey,TValue>`?
4. Зачем `HashSet<T>`?
5. Что такое `IEnumerable<T>`?
6. Почему не отдавать `List<T>` наружу?
7. Сколько стоит поиск в `Dictionary`? а в `List`?
8. Что такое коллекционные выражения?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 07

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Телефонная книга

**Сложность:** ★☆☆
**Описание:** Функции-«книги» на `Dictionary<string, string>` (имя → телефон).

**Требования:**

- Добавить, найти, удалить контакт.
- `TryGetValue` для поиска.

## Задача 2: Частотный анализ слов

**Сложность:** ★★☆
**Описание:** Разбей текст на слова, посчитай частоту каждого.

**Требования:**

- `Dictionary<string, int>`.
- Игнорируй знаки препинания и регистр.
- Выведи топ-5 слов.

**Подсказка:** `Split` с разделителями, `ToLower()`, сортировка пары по значению.

## Задача 3: Уникальные посетители

**Сложность:** ★★☆
**Описание:** Два списка визитов за день; найди общих и уникальных посетителей.

**Требования:**

- `HashSet<string>`.
- Выведи: только в день 1, только в день 2, общие.

## Задача 4: Обобщённый стек с максимумом

**Сложность:** ★★★
**Описание:** Класс `MaxStack<T>` с ограничением `where T : IComparable<T>`.

**Требования:**

- `Push`, `Pop`, `Max` (максимум среди элементов).
- Поле хранит стек; `Max()` — проход по всем.

**Подсказка:** `IComparable<T>`, сравнение `CompareTo`.

## Задача 5: Очередь печати

**Сложность:** ★☆☆
**Описание:** Симуляция очереди печати через `Queue<int>`.

**Требования:**

- Добавляй задачи (`Enqueue`), обрабатывай (`Dequeue`), пока не пусто.

## Задача 6: Рефакторинг на `List<T>`

**Сложность:** ★★☆
**Описание:** Массив участников соревнования → список с возможностью добавить/удалить.

**Требования:**

- Замени `int[]` на `List<int>`.
- Реализуй «добавить участника», «удалить по номеру», «вывести средний балл».

## Решения

▶ Задача 1 — решение

▶ Задача 2 — решение

▶ Задача 3 — решение

▶ Задача 4 — решение

▶ Задача 5 — решение

▶ Задача 6 — решение

# Разбор кода

# Разбор кода — Модуль 07

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: `List.Contains` в цикле — O($n^2$)

### Исходный код

```
 var listA = ...; // 10 000 элементов
var listB = ...; // 10 000 элементов
var duplicates = new List<int>();

foreach (var x in listA)
{
    if (listB.Contains(x)) // O(n) на каждую проверку
    {
        duplicates.Add(x);
    }
}
```

### Что не так

- `List.Contains` — O(n); в цикле по 10 000 элементов это ~100 млн операций.

### Как лучше

```
 var setB = listB.ToHashSet();
var duplicates = listA.Where(setB.Contains).ToList(); // O(n)
```

### Вывод

Для проверок существования используй `HashSet` / `Dictionary` — O(1) вместо O(n).

## Пример 2: Публичный `List<T>` в API

### Исходный код

```
 class Cart
{
```

```
    public List<string> Items { get; } = new();
}

var cart = new Cart();
cart.Items.Clear(); // кто угодно стирает
```

### Что не так

- `List` раскрывает внутреннее состояние на изменение. Инкапсуляция нарушена.

### Как лучше

```
class Cart
{
    private readonly List<string> _items = new();
    public IReadOnlyList<string> Items => _items;
}
```

### Вывод

Отдавай наружу `IReadOnlyList<T>` — потребители читают, но не меняют.

## Пример 3: Изменение коллекции в `foreach`

### Исходный код

```
var names = new List<string> { "Иван", "Мария", "Пётр" };

foreach (var name in names)
{
    if (name.StartsWith("M")) names.Remove(name); // InvalidOperationException!
}
```

### Что не так

- Модификация коллекции во время перебора бросает `InvalidOperationException`.

### Как лучше

```
var toRemove = names.Where(n => n.StartsWith("M")).ToList();
foreach (var name in toRemove) names.Remove(name);
```

### Вывод

Сначала собери список того, что менять, затем применяй изменения.

## Пример 4: Перебор ключей ради поиска

### Исходный код

```
 if (dict.Keys.Contains(key))
{
    return dict[key];
}
```

### Что не так

- Двойной проход (поиск ключа, затем доступ) и лишний разбор.

### Как лучше

```
 if (dict.TryGetValue(key, out var value))
{
    return value;
}
```

### Вывод

`TryGetValue` — один проход, один разбор, O(1).

## Пример 5: Необобщённые коллекции с боксингом

### Исходный код

```
 var items = new System.Collections.ArrayList();
items.Add(1);
items.Add(2);
int sum = 0;
foreach (int item in items) sum += item;
```

### Что не так

- `ArrayList` хранит `object`, вызывает боксинг/анбоксинг и требует приведение типов.

### Как лучше

```
 var items = new List<int> { 1, 2 };
int sum = items.Sum();
```

### Вывод

Всегда обобщённые коллекции: безопасность и производительность.

# Модуль 08: Исключения и обработка ошибок

> **Цель:** грамотно бросать, ловить и обрабатывать ошибки; проектировать собственные исключения.

## Зачем это нужно

Программа работает с ненадёжным внешним миром: пользователь вводит мусор, файл отсутствует, сеть недоступна. Исключения — главный механизм сообщения об ошибках в C#. Умение писать код, который не падает молча и **диагностирует** проблему, — необходимый навык.

## Теория

### 1. Зачем исключения

Можно вернуть «код ошибки» (например, `-1`). Но тогда каждый вызов нужно проверять. Исключения — единый механизм: метод бросает `throw`, а вызывающий может перехватить `catch`.

Исключение — это объект, наследуемый от `Exception`.

### 2. `try` / `catch` / `finally`

```
try
{
    int x = int.Parse(input);
    Console.WriteLine(x);
}
catch (FormatException)
{
    Console.WriteLine("Это не число");
}
catch (Exception ex)
{
    Console.WriteLine($"Неизвестная ошибка: {ex.Message}");
}
finally
{
    Console.WriteLine("Выполнится в любом случае");
}
```

## 3. Стандартные исключения

| Исключение | Когда |
|---|---|
| `ArgumentException` | неверный аргумент |
| `ArgumentNullException` | `null` на входе |
| `ArgumentOutOfRangeException` | значение вне диапазона |
| `InvalidOperationException` | операция недопустима в текущем состоянии |
| `FormatException` | неверный формат строки |
| `DivideByZeroException` | деление на ноль |
| `NullReferenceException` | разыменование `null` |
| `NotImplementedException` | метод не реализован |

Валидация аргументов — guard clauses:

```
 void SetAge(int age)
{
    if (age <= 0) throw new ArgumentOutOfRangeException(nameof(age), "Возраст должен быть положительным");
    ...
}
```

## 4. `throw` vs `rethrow`

Никогда не теряй стек вызовов — бросай «как есть»:

```
 catch (Exception ex)
{
    Log(ex);
    throw; // сохраняет стек — правильно
    // throw ex; // ПЛОХО: обнуляет стек
}
```

## 5. `throw`-выражения

В тернарном операторе и инициализаторах:

```
static string EnsureNotEmpty(string? value) =>
    string.IsNullOrWhiteSpace(value)
        ? throw new ArgumentException("Пустое значение")
        : value;
```

## 6. Фильтры `when`

```
 catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.Unauthorized)
{
    // перелогиниться
}
catch (HttpRequestException)
{
    // другая ошибка сети
}
```

## 7. Собственные исключения

```
 class InvalidEmailException : Exception
{
    public InvalidEmailException(string email)
        : base($"Некорректный email: {email}")
    {
        Email = email;
    }

    public string Email { get; }
}
```

```
throw new InvalidEmailException("not-an-email");
```

- Имя заканчивается на `Exception`.
- Наследуемся от `Exception` (или от конкретного типа).
- Сообщения — понятные.

## 8. `using` и `IDisposable`

`IDisposable` — интерфейс с `Dispose()`. Типы, владеющие ресурсами (файлы, соединения), его реализуют. `using` гарантирует освобождение даже при исключении.

```
 using (var file = File.CreateText("log.txt"))
{
    file.WriteLine("запись");
} // Dispose() вызывается автоматически
```

**Using declaration** (C# 8) — проще:

```
using var file = File.CreateText("log.txt");
file.WriteLine("запись");
```

Освобождение происходит в конце области видимости.

### 9. Чего не делать

- Пустой `catch { }` — проглатывание ошибок.
- Использовать исключения для управления потоком (как `goto`).
- Ловить `catch (Exception)` без нужды.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Пустой `catch { }` | молча теряется причина | логировать и/или перебросить |
| `throw ex;` | теряется стек вызовов | `throw;` |
| `catch (Exception)` везде | ловит и «корявые» баги | конкретные типы |
| `new Exception("...")` | непонятный тип | свой класс или стандартный тип |
| Ресурсы без `using` | утечка | `using` |
| Исключения как поток управления | нечитаемо | нормальный `if`/возврат |

## Контрольные вопросы

1. Зачем исключения, а не коды ошибок?
2. Что делает `finally`?
3. Когда `throw;` отличает от `throw ex;`?
4. Что даёт `catch (Exception ex) when`?
5. Как создать своё исключение?
6. Что реализует `IDisposable` и зачем `using`?
7. Почему пустой `catch` вреден?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 08

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Собственное исключение

**Сложность:** ★★☆
**Описание:** Создай `InvalidEmailException : Exception` и используй его при валидации email.

**Требования:**

- Проверка: содержит `@` и `.`.
- При некорректном — `throw new InvalidEmailException(email)`.

## Задача 2: Guard-валидация

**Сложность:** ★☆☆
**Описание:** Метод `Withdraw(decimal amount, decimal balance)`.

**Требования:**

- `amount <= 0` → `ArgumentOutOfRangeException`.
- `amount > balance` → `InvalidOperationException`.
- Используй `nameof`.

## Задача 3: TryParse-паттерн

**Сложность:** ★★☆
**Описание:** Напиши метод, возвращающий в `out` разобранную дату и `bool` успешности.

**Требования:**

- `bool TryParseDate(string s, out DateTime date)`.
- Внутри — блок try-catch с `FormatException`.

## Задача 4: `when`-фильтр

**Сложность:** ★★★
**Описание:** Обработай исключения пользователя и остальные- по-разному.

**Требования:**

- `MyAppException` — «пользователь ошибся» → понятное сообщение.
- Другое `Exception` — лог + переброс (`throw;`).

## Задача 5: `IDisposable` + `using`

**Сложность:** ★★☆
**Описание:** Класс `FakeConnection` с имитацией ресурса.

**Требования:**

- Реализуй `IDisposable` (пиши в консоль, что соединение закрыто).
- Используй `using`-декларацию. Покажи, что `Dispose` вызовется при исключении.

## Задача 6: Рефакторинг null-возвратов

**Сложность:** ★★☆
**Описание:** Замени возврат `null` на исключение в методе, который не должен возвращать `null`.

**Требования:**

- Например, `FindUser` в списке, обязанный найти.
- Объясни, когда `null` уместен, а когда — `throw`.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 08

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: Пустой `catch`

### Исходный код

```
try
{
    ProcessFile(path);
}
catch
{
    // ничего не делаем
}
```

### Что не так

- Ошибка проглатывается молча. Приложение работает «как ни в чём не бывало», но данные теряются. Диагностика невозможна.

### Как лучше

```
try
{
    ProcessFile(path);
}
catch (IOException ex)
{
    Console.WriteLine($"Не могу прочитать файл: {ex.Message}");
    throw; // или обработать осмысленно
}
```

### Вывод

Пустой `catch` — зло. Либо обрабатывай, либо перебрасывай, либо хотя бы логируй.

## Пример 2: `throw ex;`

### Исходный код

```
 catch (Exception ex)
{
    Log(ex);
    throw ex; // ПЛОХО
}
```

### Что не так

- `throw ex;` **сбрасывает** стек вызовов: настоящая точка отказа скрывается, отладка превращается в гадание.

### Как лучше

```
 catch (Exception ex)
{
    Log(ex);
    throw; // сохраняет исходный стек
}
```

### Вывод

Для переброса используй голый `throw;`, не `throw ex;`.

## Пример 3: Один `catch (Exception)` на всё

### Исходный код

```
 try
{
    var result = ProcessOrder(order);
    Console.WriteLine(result);
}
catch (Exception ex)
{
    Console.WriteLine("Что-то пошло не так: " + ex.Message);
}
```

### Что не так

- Ловится всё подряд: и ожидаемая ошибка бизнес-логики, и `NullReferenceException` (баг), и `OutOfMemoryException`. Все обрабатываются одинаково.

### Как лучше

```
try
{
    var result = ProcessOrder(order);
    Console.WriteLine(result);
}
catch (OrderValidationException ex)
{
    Console.WriteLine($"Некорректный заказ: {ex.Message}");
}
catch (NullReferenceException)
{
    Console.WriteLine("Внутренний баг: обращение к null");
}
```

### Вывод

Лови конкретные типы — и только то, что можешь осмысленно обработать.

## Пример 4: `new Exception("...")`

### Исходный код

```
if (account.Balance < amount)
    throw new Exception("Недостаточно средств");
```

### Что не так

- `Exception` слишком общий: невозможно отловить именно эту ситуацию, не поймав остальное. Сообщение — единственное отличие.

### Как лучше

```
if (account.Balance < amount)
    throw new InvalidOperationException("Недостаточно средств");
```

### Вывод

Используй семантически точный тип исключения. Если стандартных не хватает — создай свой.

## Пример 5: Ресурсы без `using`

### Исходный код

```
 var reader = new StreamReader("data.txt");
var line = reader.ReadLine();
Console.WriteLine(line);
// reader.Dispose() так и не вызовется, если будет исключение
```

### Что не так

- Если `ReadLine` бросит исключение, файл останется открытым (утечка хэндла). При повторных запусках — «файл занят другим процессом».

### Как лучше

```
 using var reader = new StreamReader("data.txt");
var line = reader.ReadLine();
Console.WriteLine(line);
```

### Вывод

Все `IDisposable` — под `using` (или в `finally` с `Dispose`).

## Пример 6: Исключения как поток управления

### Исходный код

```
 try
{
    if (string.IsNullOrWhiteSpace(name)) throw new ArgumentException();
    ...
}
catch (ArgumentException)
{
    // «поймали» ожидаемую ситуацию, чтобы что-то сделать
}
```

### Что не так

- Исключения используются вместо обычных условий. Это медленно, нечитаемо и запутывает.

### Как лучше

```
 if (string.IsNullOrWhiteSpace(name))
{
    Console.WriteLine("Имя не задано");
    return;
}
```

### Вывод

Исключения — для исключительных ситуаций, а не для рутины.

# Модуль 09: Делегаты, лямбды и события

> **Цель:** передавать поведение как параметр, писать компактные лямбды и строить системы уведомлений на событиях.

## Зачем это нужно

Функции в C# — это значения. Их можно передавать в методы, хранить и вызывать позже. На этой идее построены колбэки, события и LINQ. Понимание делегатов и лямбд открывает дверь к паттернам Observer, Strategy и большей части современного C# API.

## Теория

### 1. Делегаты

Делегат — тип, который описывает сигнатуру метода.

```
delegate void Notify(string message);
```

Мы можем передать метод в переменную-делегат и вызвать его:

```
void PrintToConsole(string msg) => Console.WriteLine(msg);

Notify notifier = PrintToConsole;
notifier("Привет");
```

Делегат можно передать как параметр другого метода — это и есть **колбэк**:

```
void ExecuteAction(Notify callback)
{
    callback("Действие выполняется");
}
```

### 2. `Action`, `Func`, `Predicate`

Необязательно объявлять свой делегат — есть готовые:

| Тип | Возвращает | Пример |
|---|---|---|
| `Action` | ничего | `Action log = () => Console.WriteLine("!");` |

| Тип | Возвращает | Пример |
|---|---|---|
| `Action<T>` | ничего | `Action<string> show = s => Console.WriteLine(s);` |
| `Func<TR>` | `TR` | `Func<int> f = () => 42;` |
| `Func<T1,T2,TR>` | `TR` | `Func<int,int,int> add = (a,b) => a+b;` |
| `Predicate<T>` | `bool` | `Predicate<int> >Zero = n => n > 0;` |

## 3. Лямбда-выражения

Лямбда — анонимная функция:

```
 Func<int, int> square = x => x * x;
Console.WriteLine(square(5)); // 25
```

С телом из нескольких операторов:

```
 Action<int> showTable = n =>
{
    for (int i = 1; i <= n; i++) Console.WriteLine(i);
};
```

Лямбды повсюду в BCL:

```
 var numbers = new List<int> { 3, 1, 4, 1, 5 };
numbers.Sort((a, b) => b.CompareTo(a)); // по убыванию
var firstEven = numbers.Find(n => n % 2 == 0);
```

## 4. Замыкания (closures)

Лямбда «захватывает» переменные из внешнего контекста:

```
 int factor = 10;
Func<int, int> multiply = x => x * factor;
Console.WriteLine(multiply(5)); // 50
```

> **Важно:** захваченная переменная — по ссылке. Если она изменится позже, лямбда увидит новое значение. Это причина классической ловушки с циклом (см. разбор кода).

## 5. События

`event` — «публикация» некоего события, на которое подписываются обработчики.

```
class OrderService
{
    public event EventHandler? OrderPlaced;

    public void PlaceOrder(string id)
    {
        Console.WriteLine($"Заказ {id} создан");
        OrderPlaced?.Invoke(this, EventArgs.Empty);
    }
}
```

Подписка и отписка:

```
var service = new OrderService();
service.OrderPlaced += OnOrderPlaced;   // подписаться
service.OrderPlaced -= OnOrderPlaced;   // отписаться

void OnOrderPlaced(object? sender, EventArgs e)
{
    Console.WriteLine("Уведомление клиенту отправлено");
}
```

- `EventHandler` / `EventHandler<T>` — стандартные типы для событий.
- Публикация через `?.Invoke(...)` — безопасная проверка на null.

### 6. Ковариантность/контравариантность (кратко)

Делегаты позволяют подставлять более специфичные типы там, где ожидается базовый (ковариантность) и наоборот (контравариантность). Это редко нужно новичку — просто знай, что такое существует.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Делегат-поле вместо события | любой может присвоить и затереть подписчиков | `event` |
| Публикация без null-проверки | `NullReferenceException` | `Event?.Invoke(...)` |
| Лямбда вместо именованного метода, если он переиспользуется | дублирование и сложность отладки | отдельный метод |
| Замыкание на переменную цикла | неожиданные значения | запоминать значение в локальную переменную |
| `+=` одного и того же метода дважды | двойное выполнение | контролировать подписку |

## Контрольные вопросы

1. Что такое делегат?
2. Когда `Action`, а когда `Func`?
3. Что такое лямбда?
4. Что такое замыкание?
5. Что даёт `event` поверх делегата?
6. Что такое `EventHandler`?
7. Зачем `?.Invoke(...)` при публикации?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 09

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Калькулятор на `Func`

**Сложность:** ★☆☆
**Описание:** Принимай операцию как `Func<double, double, double>`.

**Требования:**

- `double Apply(double a, double b, Func<double, double, double> op)`.
- Вызови для `+`, `*`, `%`.

## Задача 2: Уведомления о заказе

**Сложность:** ★★☆
**Описание:** Класс `OrderService` публикует `OrderPlaced`.

**Требования:**

- Событие `EventHandler<string>` с ID заказа.
- Два подписчика: логгер и отправщик почты.
- Проверь отписку.

## Задача 3: Кастомный делегат

**Сложность:** ★★☆
**Описание:** Объяви `delegate double Operation(double a, double b);` и используй вместо `Func`.

**Требования:**

- Метод `double Calc(double a, double b, Operation op)`.
- Передай `Add`, `Multiply`, лямбду.

## Задача 4: Счётчик вызовов

**Сложность:** ★★☆
**Описание:** Лямбда-замыкание, которое накапливает вызовы.

**Требования:**

- `Func<int>` возрастающий счётчик.
- Вызови 5 раз, выведи последнее значение.

## Задача 5: Сортировка объектов

**Сложность:** ★★☆
**Описание:** Список `Person` (имя, возраст); сортируй разными способами.

**Требования:**

- Сортировка по имени, по возрасту, по возрасту по убыванию.
- Сравни с `IComparable` из модуля 05.

## Задача 6: Защита от утечки подписки

**Сложность:** ★★★
**Описание:** Класс-подписчик, который корректно отписывается в `Dispose()`.

**Требования:**

- Каждый `Subscribe` создаёт объект, который подписывается.
- При `Dispose()` — `-=`.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 09

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: Делегат-поле вместо `event`

### Исходный код

```
 class OrderService
{
    public Action<string>? OrderPlaced; // не event!
}
```

### Что не так

- Любой код может **присвоить**: `service.OrderPlaced = DoSomething;` — стирая всех остальных подписчиков. Отписка и множественная подписка работают некорректно.

### Как лучше

```
 class OrderService
{
    public event Action<string>? OrderPlaced;
}
```

### Вывод

`event` ограничивает: снаружи доступны только `+=` и `-=`, нельзя присвоить и затереть.

## Пример 2: Публикация без null-проверки

### Исходный код

```
 OrderPlaced?.Invoke(this, id); // ок
// а вот так — НЕЛЬЗЯ:
OrderPlaced.Invoke(this, id);  // NullReferenceException, если нет подписчиков
```

### Что не так

- Если подписчиков нет, `Invoke` падает. Нужно проверять на `null`.

### Как лучше

```
OrderPlaced?.Invoke(this, id);
```

### Вывод

Всегда публикуй через `?.Invoke(...)`.

## Пример 3: Лямбда вместо именованного метода

### Исходный код

```
 service.Moved += (s, e) => EmailClient.Send(GetEmail());
service.Moved += (s, e) => EmailClient.Send(GetEmail());
```

### Что не так

- Одна и та же лямбда, подписанная дважды, выполнится дважды. Отписаться от неё невозможно (нет ссылки на метод).

### Как лучше

```
 void OnMoved(object? s, EventArgs e) => EmailClient.Send(GetEmail());

service.Moved += OnMoved;
// ... контроль подписки и отписка:
service.Moved -= OnMoved;
```

### Вывод

Переиспользуемые обработчики — отдельные методы, а не анонимные лямбды.

## Пример 4: Замыкание на переменную цикла

### Исходный код

```
 var actions = new List<Action>();
for (int i = 0; i < 3; i++)
{
    actions.Add(() => Console.WriteLine(i));
```

```
}

foreach (var a in actions) a(); // 3 3 3 — а НЕ 0 1 2
```

### Что не так

- Лямбда захватывает `i` по ссылке. К моменту вызова цикл уже закончился, `i == 3`.

### Как лучше

```
 for (int i = 0; i < 3; i++)
{
    int copy = i; // локальная копия на каждой итерации
    actions.Add(() => Console.WriteLine(copy));
}
// выведет 0 1 2
```

### Вывод

Если нужно зафиксировать значение в момент создания лямбды — скопируй его в локальную переменную.

## Пример 5: Подписчик живёт дольше источника

### Исходный код

```
 class Logger
{
    public Logger(Timer timer) => timer.Elapsed += OnElapsed;
    private void OnElapsed(object? s, ElapsedEventArgs e) => ...
}
```

### Что не так

- Подписчик `Logger` удерживается источником; если `Logger` больше не нужен, он не освободится (утечка памяти).

### Как лучше

```
 class Logger : IDisposable
{
    private readonly Timer _timer;
    public Logger(Timer timer)
```

```
    {
        _timer = timer;
        _timer.Elapsed += OnElapsed;
    }
    public void Dispose() => _timer.Elapsed -= OnElapsed;
}
```

### Вывод

Кто подписался, тот и отписывается (обычно в `Dispose`).

# Модуль 10: LINQ

> **Цель:** писать декларативные запросы к коллекциям и понимать «ленивое» выполнение.

## Зачем это нужно

LINQ (Language Integrated Query) — встроенный язык запросов к коллекциям. Вместо десятков строк циклов и временных списков ты пишешь выразительный пайплайн: «отфильтруй, спроецируй, отсортируй, возьми топ-5». Это один из самых ценных навыков для .NET-разработчика — с LINQ вы сталкиваетесь практически в каждом реальном проекте.

## Теория

### 1. Два синтаксиса

**Методный (method syntax):**

```
var adults = users.Where(u => u.Age >= 18).OrderBy(u => u.Name).ToList();
```

**Query-синтаксис:**

```
var adults = (from u in users
              where u.Age >= 18
              orderby u.Name
              select u).ToList();
```

Оба эквивалентны. Методный встречается чаще — мы будем использовать его.

### 2. Отложенное выполнение (deferred execution)

Главная концепция! Запрос **не выполняется** в момент вызова `Where` / `Select`. Он выполняется при переборе (материализации).

```
 var query = numbers.Where(n => n > 2);       // пока ничего не посчитано
numbers.Add(10);                               // изменили источник
Console.WriteLine(query.Count());              // теперь пересчитается с учётом 10
```

Чтобы «зафиксировать» результат — материализуют:

```
var list = query.ToList();      // или ToArray(), ToDictionary()
```

Повторный перебор «ленивого» запроса обычно пересчитывает источник — осторожно с дорогими источниками.

## 3. Основные операторы

### Фильтрация

```
var evens = numbers.Where(n => n % 2 == 0);
```

### Проекция

```
var names = users.Select(u => u.Name);
var nameAndAge = users.Select(u => new { u.Name, u.Age }); // анонимный тип
```

### Статистика

```
int count = numbers.Count();
int sum = numbers.Sum();
double avg = numbers.Average();
int min = numbers.Min();
int max = numbers.Max();
```

### Первый / последний / единственный

```
var first = numbers.First();           // исключение, если пусто
var firstOr = numbers.FirstOrDefault(); // null/0, если пусто
var single = numbers.Single(n => n == 5); // исключение, если не ровно один
var singleOr = numbers.SingleOrDefault(n => n == 5);
```

`Single` — опасен: бросит исключение, если совпадений больше одного.

### Проверки

```
bool anyEven = numbers.Any(n => n % 2 == 0);
bool allPositive = numbers.All(n => n > 0);
```

### Сортировка

```
var sorted = users.OrderBy(u => u.Name).ThenBy(u => u.Age);
var desc = users.OrderByDescending(u => u.Age);
```

### Уникальность и «нарезка»

```
var distinct = numbers.Distinct();
var top3 = numbers.OrderByDescending(n => n).Take(3);
var page = numbers.Skip(20).Take(10);
```

### Агрегация

```
int product = numbers.Aggregate(1, (acc, n) => acc * n);
```

### Группировка

```
var byAge = users.GroupBy(u => u.Age);
foreach (var group in byAge)
{
    Console.WriteLine($"{group.Key}: {group.Count()}");
}
```

### Join

```
var joined = authors.Join(books,
    author => author.Id,
    book => book.AuthorId,
    (author, book) => new { Author = author.Name, Book = book.Title });
```

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Не понимать deferred execution | повторный перебор, неожиданные результаты | материализуй `ToList()` при необходимости |
| `Select` в цикле вместо `SelectMany` | вложенные коллекции не «сплющиваются» | `SelectMany` |
| `Single` там, где возможен не один элемент | исключение в рантайме | `FirstOrDefault` / проверка `Any` |
| Вложенный `OrderBy` в цикле | лишняя пересортировка | сортируй один раз |
| `User.Where(...).ToList()` без нужды | лишняя аллокация | оставляй «ленивым» при однократном переборе |

## Контрольные вопросы

1. Что значит «отложенное выполнение»?
2. Когда нужна материализация `ToList()` / `ToArray()` ?

3. Чем `FirstOrDefault` отличается от `SingleOrDefault`?
4. Что делает `SelectMany`?
5. Как сделать сортировку по двум ключам?
6. Как получить топ-N элементов?
7. Что возвращает `GroupBy`?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 10

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Фильтрация пользователей

**Сложность:** ★☆☆
**Описание:** Список записей `Person(Name, Age)`. Отбери совершеннолетних, спроецируй имена.

**Требования:**

- `Where`, `Select`, `ToList`.

## Задача 2: Группировка заказов

**Сложность:** ★★☆
**Описание:** `Order(Client, Amount)`. Сгруппируй по клиенту, для каждого посчитай сумму.

**Требования:**

- `GroupBy`, внутри группы `Sum`.
- Выведи клиент → общая сумма.

## Задача 3: Join авторов и книг

**Сложность:** ★★★
**Описание:** Два списка: `Author(Id, Name)` и `Book(AuthorId, Title)`. Соедини.

**Требования:**

- `Join` + проекция в анонимный тип.
- Выведи: автор → список его книг.

## Задача 4: Топ-N и пагинация

**Сложность:** ★★☆
**Описание:** `Score(Player, Points)`.

**Требования:**

- Топ-3 игрока ( `OrderByDescending` + `Take` ).
- Вторая «страница» (Skip/Take).

## Задача 5: Рефакторинг трёх циклов

**Сложность:** ★★★
**Описание:** Дан массив `int[]`. Тремя циклами считались: чётные, сумма, максимум.

**Требования:**

- Перепиши в один LINQ-пайплайн (или в три понятных оператора).

## Задача 6: `Batch<T>`

**Сложность:** ★★★
**Описание:** Напиши extension-метод `Batch<T>(this IEnumerable<T>, int size)`, возвращающий «страницы».

**Требования:**

- Возвращает `IEnumerable<IEnumerable<T>>`.
- Используй `yield return`.
- Массив из 10, `Batch(4)` → 4, 4, 2.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 10

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: `Select` в цикле вместо `SelectMany`

### Исходный код

```
 var allNames = new List<string>();
foreach (var team in teams)
{
    allNames.AddRange(team.Players.Select(p => p.Name));
}
```

### Что не так

- Логика «сплющивания» вложенных коллекций размазана. `SelectMany` делает это выразительнее.

### Как лучше

```
var allNames = teams.SelectMany(t => t.Players).Select(p => p.Name).ToList();
```

### Вывод

`SelectMany` — для превращения коллекции коллекций в плоский список.

## Пример 2: Повторный перебор «ленивого» IEnumerable

### Исходный код

```
 var filtered = data.Where(x => ExpensiveCheck(x));
if (filtered.Any()) { ... }   // 1-й проход
var count = filtered.Count();   // 2-й проход — ExpensiveCheck снова
```

### Что не так

- `filtered` — ленивый; каждый перебор пересчитывает `ExpensiveCheck`. Двойная работа.

### Как лучше

```
 var list = data.Where(x => ExpensiveCheck(x)).ToList();
if (list.Any()) { ... }
var count = list.Count;
```

### Вывод

Материализуй результат в `ToList()` / `ToArray()`, если перебираешь несколько раз.

## Пример 3: `FirstOrDefault` для value-типов

### Исходный код

```
 int found = numbers.FirstOrDefault(n => n > 100);
if (found != 0) Console.WriteLine(found); // неверно: 0 — легитимное значение
```

### Что не так

- `FirstOrDefault` для `int` вернёт `0`, если совпадений нет. Отличить «нет элемента» от «элемент равен 0» нельзя.

### Как лучше

```
 int? found = numbers.Cast<int?>().FirstOrDefault(n => n > 100);
if (found.HasValue) Console.WriteLine(found.Value);
```

или просто проверять `Any` перед `First`:

```
 if (numbers.Any(n => n > 100))
{
    int value = numbers.First(n => n > 100);
}
```

### Вывод

Для value-типов аккуратно используй `FirstOrDefault` — лучше явная проверка `Any`.

## Пример 4: Сортировка в цикле

### Исходный код

```
foreach (var u in users)
{
    var sorted = users.OrderBy(x => x.Age); // сортировка на каждой итерации!
}
```

### Что не так

- `OrderBy` перевызывается в цикле — дорого и бессмысленно.

### Как лучше

```
var sorted = users.OrderBy(u => u.Age).ToList();
foreach (var u in sorted) { ... }
```

### Вывод

Сортировка один раз, вне цикла.

## Пример 5: `string.Concat` в цикле

### Исходный код

```
string result = "";
foreach (var s in items)
{
    result += s; // копирование строки на каждой итерации
}
```

### Что не так

- $O(n^2)$: каждая конкатенация копирует всю накопленную строку.

### Как лучше

```
string result = string.Join("", items);
// для сложного форматирования — StringBuilder
```

### Вывод

`string.Join` решает почти все случаи. В сложных случаях — `StringBuilder`.

# Модуль 11: Асинхронность

> **Цель:** использовать `async` / `await` для неблокирующих операций и корректно отменять их.

## Зачем это нужно

Операции ввода-вывода (сеть, диск, базы данных) — в тысячи раз медленнее вычислений. Если ждать их «в лоб» — программа зависает. Асинхронность позволяет не блокировать поток, пока идёт ожидание. Это обязательно для веб-API, десктопных UI и любых сервисов.

## Теория

### 1. Проблема

```
var response = httpClient.GetStringAsync("https://example.com").Result; // плохо!
```

Ожидание результата блокирует поток. В интерфейсе это «зависшее окошко».

### 2. `async` / `await`

```
async Task DownloadAsync()
{
    var client = new HttpClient();
    var content = await client.GetStringAsync("https://example.com");
    Console.WriteLine(content.Length);
}
```

- `await` приостанавливает метод, **не блокируя поток**. Поток освобождается.
- `async Task` — стандарт. `async void` — только для обработчиков событий (иначе ошибки теряются).
- `Task<T>` возвращает значение: `var data = await GetDataAsync();`

### 3. Что возвращает `await`

- `await` «разворачивает» `Task<T>` в `T`.
- Ошибки внутри `Task` всплывают в точке `await` как исключения.

### 4. `CancellationToken`

Для отмены:

```
async Task ProcessAsync(CancellationToken token)
{
    for (int i = 0; i < 100; i++)
    {
        token.ThrowIfCancellationRequested(); // проверим отмену
        await Task.Delay(100, token);
    }
}
```

```
using var cts = new CancellationTokenSource();
cts.CancelAfter(TimeSpan.FromMilliseconds(300));

try
{
    await ProcessAsync(cts.Token);
}
catch (OperationCanceledException)
{
    Console.WriteLine("Операция отменена");
}
```

## 5. `Task.WhenAll` и `Task.WhenAny`

**WhenAll** — ждать все параллельно:

```
Task<int> t1 = GetAsync(1);
Task<int> t2 = GetAsync(2);
int[] results = await Task.WhenAll(t1, t2);
```

**WhenAny** — ждать первый завершившийся:

```
var first = await Task.WhenAny(DownloadA(), DownloadB());
```

## 6. `Task.Run` для CPU-bound

`Task.Run` запускает код в фоновом потоке. Для CPU-bound-вычислений (не IO) это может помочь не блокировать UI:

```
int result = await Task.Run(() => HeavyComputation());
```

Но для IO (сеть/диск) — `async`-методы без `Task.Run`.

### 7. `.Result` / `.Wait()` и deadlock

Никогда не вызывай `.Result` / `.Wait()` внутри async-кода — это блокирует и может дать **deadlock** (взаимоблокировку) в окружениях с контекстом синхронизации (UI, ASP.NET). Всегда `await`.

### 8. `ConfigureAwait(false)`

Библиотечный код без UI/ASP.NET-контекста может использовать `ConfigureAwait(false)`, чтобы не захватывать контекст:

```
var data = await GetAsync().ConfigureAwait(false);
```

Не обязательно для новичка, но полезно знать.

### 9. `IAsyncEnumerable<T>` и `await foreach`

Асинхронный перебор (например, чтение большого потока построчно):

```
await foreach (var chunk in ReadChunksAsync())
{
    Console.WriteLine(chunk);
}
```

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| `async void` не в обработчике | ошибки теряются + нельзя await | `async Task` |
| `.Result` / `.Wait()` | блокировка, дедлок | `await` |
| Забытый `await` в async-методе | warning CS4014/CS1998 | добавить `await` или осознанно `_ = ...` |
| Fire-and-forget | потерянные исключения | дожидаться через await |
| `async` в методе без `await` | лишний статусный автомат | убрать `async` |

## Контрольные вопросы

1. Что значит «не блокирует поток»?
2. Почему `async void` опасен?
3. Что делает `CancellationToken`?

4. Как запустить несколько задач параллельно?
5. В чём разница `WhenAll` и `WhenAny`?
6. Почему `.Result` — плохо?
7. Что делает `ConfigureAwait(false)`?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 11

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Асинхронная загрузка

**Сложность:** ★☆☆
**Описание:** Метод `Task<int> FetchAsync(int id)`, имитирующий задержку через `Task.Delay`.

**Требования:**

- `Take.Delay(500)`.
- Используй `await` в `Main`.
- Выведи до и после `await`.

## Задача 2: Параллельные загрузки

**Сложность:** ★★☆
**Описание:** Запусти три `FetchAsync` параллельно через `Task.WhenAll`.

**Требования:**

- Суммируй результаты.
- Замерь время (Stopwatch) — должно быть ~500 мс, а не 1500.

## Задача 3: Отмена по таймауту

**Сложность:** ★★★
**Описание:** Используй `CancellationTokenSource` с `CancelAfter`.

**Требования:**

- Операция дольше таймаута → `OperationCanceledException`.
- Обработай через `try-catch`.

## Задача 4: «Кто быстрее»

**Сложность:** ★★☆
**Описание:** `Task.WhenAny` — первый ответ из трёх источников.

**Требования:**

- Три задержки (300/600/900 мс).
- Получи первый завершившийся результат.

## Задача 5: Цепочка с `CancellationToken`

**Сложность:** ★★★
**Описание:** Сымитируй «UI → сервис → репозиторий» с пробросом токена.

**Требования:**

- Каждый метод принимает `CancellationToken`.
- Токен пробрасывается насквозь, проверяется `ThrowIfCancellationRequested()`.

## Задача 6: Рефакторинг синхронного кода

**Сложность:** ★★★
**Описание:** Перепиши «блокирующий» код на async-версию.

**Требования:**

- До: `GetStringAsync(...).Result`.
- После: `await`.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 11

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: `.Result` внутри async-метода

### Исходный код

```
async Task<string> GetAsync()
{
    var client = new HttpClient();
    return client.GetStringAsync("https://example.com").Result; // ПЛОХО
}
```

### Что не так

- Блокирует поток и рискует получить **deadlock** в UI/ASP.NET, т.к. контекст синхронизации удерживается.

### Как лучше

```
async Task<string> GetAsync()
{
    using var client = new HttpClient();
    return await client.GetStringAsync("https://example.com");
}
```

### Вывод

Внутри async-кода — только `await`.

## Пример 2: `async void`

### Исходный код

```
async void SaveAsync() // только для обработчиков событий!
{
    await Task.Delay(100);
}
```

### Что не так

- `async void` нельзя дождаться, а исключения теряются — их никто не поймает.

### Как лучше

```
async Task SaveAsync()
{
    await Task.Delay(100);
}
```

### Вывод

Используй `async Task`. `async void` — лишь для event-обработчиков.

## Пример 3: Fire-and-forget

### Исходный код

```
Task fire = SomeAsync();
// не дожидаемся: исключения из SomeAsync потеряются
```

### Что не так

- Если `SomeAsync` бросит исключение, оно окажется «в никуда». Программа продолжит работать с потенциально неконсистентным состоянием.

### Как лучше

```
try
{
    await SomeAsync();
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

### Вывод

Любую задачу либо дожидайся через `await`, либо осознанно обрабатывай ошибки.

## Пример 4: `async`-метод без `await`

### Исходный код

```
async Task<int> ComputeAsync() // warning CS1998
{
    return 42;
}
```

### Что не так

- Метод async, но ничего асинхронного нет. Компилятор предупредит (CS1998).

### Как лучше

```
Task<int> ComputeAsync() => Task.FromResult(42);
// или просто: int Compute() => 42;
```

### Вывод

Не добавляй `async`, если нет `await`.

## Пример 5: Блокировка UI в `foreach`

### Исходный код

```
foreach (var url in urls)
{
    var html = client.GetStringAsync(url).Result; // блокирует интерфейс
}
```

### Что не так

- Каждая итерация блокирует поток, UI «замерзает».

### Как лучше

```
var tasks = urls.Select(u => client.GetStringAsync(u));
var results = await Task.WhenAll(tasks);
```

### Вывод

Для набора операций — `Task.WhenAll`, не последовательные `.Result`.

# Модуль 12: Файлы, даты и сериализация

> **Цель:** работать с файловой системой, датами и JSON как с повседневными инструментами.

## Зачем это нужно

Любое реальное приложение читает и пишет данные: конфигурацию, логи, пользовательский контент, обмен с API. Этим правят три технологии: `System.IO` (файлы), `System.Text.Json` (сериализация), `DateTime` / `DateOnly` (время). Уверенное владение ими делает тебя продуктивным сразу.

## Теория

### 1. Работа с файлами: `System.IO`

**Пути.**

Не склеивай строки вручную — `Path.Combine`:

```
string root = Environment.CurrentDirectory;
string logPath = Path.Combine(root, "logs", "app.log");
```

**Проверка существования.**

```
Console.WriteLine(File.Exists("data.txt"));
Console.WriteLine(Directory.Exists("logs"));
```

**Чтение.**

```
string all = File.ReadAllText("data.txt");
string[] lines = File.ReadAllLines("data.txt");
```

**Запись.**

```
File.WriteAllText("out.txt", "текст");
File.AppendAllText("log.txt", "новая строка\n");
```

**Построчно (для больших файлов):**

```
using var reader = new StreamReader("big.log");
string? line;
while ((line = reader.ReadLine()) is not null)
{
    Console.WriteLine(line.Length);
}
```

**Кодировки.**

По умолчанию — UTF-8. Проблемы «кракозябр» обычно из-за неверной кодировки на чтении/записи:

```
File.WriteAllText("file.txt", "текст", System.Text.Encoding.UTF8);
```

## 2. Сериализация JSON: `System.Text.Json`

`JsonSerializer` — встроен в .NET.

```
record Product(string Name, decimal Price);
```

**Сериализация:**

```
var product = new Product("Мышь", 1499.99m);
string json = JsonSerializer.Serialize(product);
Console.WriteLine(json); // {"Name":"Мышь","Price":1499.99}
```

**Десериализация:**

```
var loaded = JsonSerializer.Deserialize<Product>(json);
Console.WriteLine(loaded!.Name);
```

**Варианты опций:**

```
var options = new JsonSerializerOptions
{
    WriteIndented = true,
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase
};
```

**Десериализация списка:**

```
var products = JsonSerializer.Deserialize<List<Product>>(json);
```

### 3. Работа с датами

**`DateTime`** — момент времени (дата + время). Удобен для меток времени.

```
DateTime now = DateTime.UtcNow;    // для логики
DateTime local = DateTime.Now;     // для отображения пользователю
```

**`DateOnly` / `TimeOnly`** — просто дата / просто время (C# 10+).

```
DateOnly today = DateOnly.FromDateTime(DateTime.Today);
DateOnly birthday = new(2000, 5, 17);
int age = today.Year - birthday.Year;
```

**`TimeSpan`** — отрезок времени.

```
TimeSpan elapsed = now - start;
Console.WriteLine(elapsed.TotalMinutes);
```

**Разбор и форматирование:**

```
DateTime date = DateTime.Parse("2024-01-15");
string text = date.ToString("dd.MM.yyyy");      // 15.01.2024
string iso = date.ToString("O");                // ISO 8601
```

> Для пользовательского ввода используй `TryParse` с `CultureInfo`, для внутреннего представления — `InvariantCulture`.

### 4. Сериализация дат

`System.Text.Json` сериализует в ISO 8601. При десериализации переопределяй формат явно.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Путь жёстко зашит | программа не работает на другой машине | `Path.Combine` + относительные пути |
| `File.ReadAllText` на огромном логе | память | `StreamReader` построчно |
| `DateTime.Now` в бизнес-логике | недетерминированно, трудно тестировать | параметр или `IClock` / `DateTime.UtcNow` |

| Ошибка | Почему плохо | Как правильно |
| --- | --- | --- |
| Игнорирование кодировок | кракозябры | `Encoding.UTF8` |

## Контрольные вопросы

1. Зачем `Path.Combine`?
2. Как прочитать файл построчно?
3. Что делает `JsonSerializer`?
4. Как включить отступы в JSON-выводе?
5. Чем `DateTime.UtcNow` отличается от `DateTime.Now`?
6. Что такое `DateOnly`?
7. Что делает `TimeSpan`?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 12

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Логгер в файл

**Сложность:** ★☆☆
**Описание:** Пиши строки в файл с меткой времени.

**Требования:**

- `File.AppendAllText`.
- Каждая запись: `[2024-01-15 12:00:00] сообщение`.

## Задача 2: Заметки с хранением в JSON

**Сложность:** ★★☆
**Описание:** Меню «добавить / показать / выйти»; заметки сохраняются в `notes.json`.

**Требования:**

- `List<string>` → `JsonSerializer.Serialize` → `File.WriteAllText`.
- При старте — загрузка через `Deserialize`.

## Задача 3: Чтение большого файла построчно

**Сложность:** ★★☆
**Описание:** Посчитай число строк и слов в файле через `StreamReader`.

**Требования:**

- `while ((line = reader.ReadLine()) is not null)`.
- Покажи разницу с `ReadAllText`.

## Задача 4: Копирование с проверкой

**Сложность:** ★★☆
**Описание:** Скопируй файл, обработай `FileNotFoundException`.

**Требования:**

- `File.Copy` с проверкой источника.
- Поймай `FileNotFoundException` и `IOException`.

## Задача 5: CSV → JSON

**Сложность:** ★★★
**Описание:** Прочитай CSV, преврати в список records, сериализуй в JSON.

**Требования:**

- Формат: `Имя;Возраст;Город`.
- `record Person(string Name, int Age, string City)`.
- Выведи JSON в консоль.

## Задача 6: Даты — возраст и дедлайн

**Сложность:** ★★☆
**Описание:** Вычисли возраст по `DateOnly` рождения; сколько дней до дедлайна.

**Требования:**

- `DateOnly` и `TimeSpan`.

## Решения

▶ Задача 1 — решение

▶ Задача 2 — решение

▶ Задача 3 — решение

▶ Задача 4 — решение

▶ Задача 5 — решение

▶ Задача 6 — решение

# Разбор кода

# Разбор кода — Модуль 12

> Формат: рабочий, но плохой код → находим проблемы → рефакторим.

## Пример 1: Захардкоженный путь

### Исходный код

```
string path = "C:\\Users\\ivan\\Documents\\save.txt";
File.WriteAllText(path, data);
```

### Что не так

- Не работает на другой машине/аккаунте, ломается при переименовании папок.

### Как лучше

```
string path = Path.Combine(Environment.CurrentDirectory, "data", "save.txt");
Directory.CreateDirectory(Path.GetDirectoryName(path)!);
```

### Вывод

Относительные пути + `Path.Combine` — переносимость.

## Пример 2: `File.ReadAllText` на огромном логе

### Исходный код

```
string entire = File.ReadAllText("server.log"); // может быть сотни МБ
```

### Что не так

- Весь файл загружается в память. Для логов/стоков — зря.

### Как лучше

```
using var reader = new StreamReader("server.log");
string? line;
```

```
while ((line = reader.ReadLine()) is not null)
{
    // обработка одной строки
}
```

### Вывод

Построчное чтение для больших файлов.

## Пример 3: `DateTime.Now` в бизнес-логике

### Исходный код

```
bool IsExpired(Order order)
{
    return DateTime.Now > order.Deadline; // зависит от времени запуска
}
```

### Что не так

- Тесты непредсказуемы: результат зависит от текущего момента. Плюс `DateTime.Now` — локальное время, не всегда уместно.

### Как лучше

```
bool IsExpired(Order order, DateTime now)
{
    return now > order.Deadline;
}
```

### Вывод

Время — параметр или абстракция ( `IClock` ). Код становится детерминированным и тестируемым.

## Пример 4: Игнорирование кодировки

### Исходный код

```
File.WriteAllText("file.txt", "Привет, мир"); // может записаться не в UTF-8
```

### Что не так

- Разные платформы могут дать «кракозябры» при чтении.

### Как лучше

```
File.WriteAllText("file.txt", "Привет, мир", System.Text.Encoding.UTF8);
```

### Вывод

Явно задавай кодировку для текстовых файлов.

## Пример 5: Копирование без проверки существования

### Исходный код

```
File.Copy("old.txt", "new.txt"); // FileNotFoundException, если old.txt нет
```

### Что не так

- Необработанное исключение обрывает программу неожиданно.

### Как лучше

```
 if (File.Exists("old.txt"))
{
    File.Copy("old.txt", "new.txt", overwrite: true);
}
else
{
    Console.WriteLine("old.txt не существует");
}
```

### Вывод

Проверяй существование и обрабатывай исключения ввода-вывода.

# Модуль 13: Юнит-тестирование

> **Цель:** писать тесты, которые реально защищают от регрессий, и мыслить в стиле TDD.

## Зачем это нужно

Код, который нельзя разбить на проверяемые куски, накапливает долги: одна правка ломает другое. Тесты — «страховка» при рефакторинге, живая документация поведения и способ проектировать код, чтобы он был тестируемым. Средний разработчик пишет тесты, а не считает их роскошью.

## Теория

### 1. Зачем тесты

- Отлавливают регрессии (поломку старого при изменении нового).
- Документируют ожидаемое поведение.
- Упрощают рефакторинг — есть уверенность.

Базовый принцип — **пирамида тестов**: много быстрых юнит-тестов, меньше интеграционных, ещё меньше E2E.

### 2. TDD (кратко)

Красный → зелёный → рефакторинг:

1. Сначала пишем падающий тест (красный).
2. Пишем код, чтобы тест прошёл (зелёный).
3. Улучшаем код (рефакторинг), не ломая зелёный.

### 3. xUnit

Установка проекта тестов:

```
 dotnet new xunit -n MyApp.Tests
dotnet add MyApp.Tests reference ../src/MyApp/MyApp.csproj
```

Базовые атрибуты:

- `[Fact]` — одиночный тест.

- `[Theory]` — набор сценариев с `[InlineData]`.

## 4. Утверждения `Assert`

| Метод | Что проверяет |
|---|---|
| `Assert.Equal(expected, actual)` | равенство |
| `Assert.NotEqual` | неравенство |
| `Assert.True` / `Assert.False` | истинность |
| `Assert.Null` / `Assert.NotNull` | null |
| `Assert.Throws<T>(() => ...)` | ожидаемое исключение |
| `Assert.ThrowsAny<T>` | любое исключение типа T |

## 5. AAA-паттерн

```
[Fact]
public void Deposit_IncreasesBalance_ReturnsNewBalance()
{
    // Arrange — подготовка
    var account = new BankAccount();
    // Act — действие
    decimal result = account.Deposit(100);
    // Assert — проверка ожидания
    Assert.Equal(100m, result);
}
```

## 6. Именование

`Method_Scenario_Expected` — например, `Withdraw_InsufficientFunds_Throws`.

## 7. Моки

Моки подменяют внешние зависимости (БД, сеть, время). Чтобы мок был возможен — зависимость должна быть **интерфейсом** и вводиться через конструктор.

```
public class OrderService
{
    private readonly IClock _clock;
    public OrderService(IClock clock) => _clock = clock;
    public bool IsExpired(Order order) => _clock.Now > order.Deadline;
}
```

С мок-библиотекой (Moq / NSubstitute) подменяем `IClock`. Можно и вручную — написать фейк-класс.

### 8. Что тестировать

- Бизнес-логику (расчёты, валидацию, переходы состояний).
- Граничные случаи.

Не нужно тестировать сам фреймворк (LINQ, Console, сериализацию).

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Тест без утверждений | «делает вид» | `Assert.*` |
| Тест на `DateTime.Now` | хрупкий, недетерминированный | параметр/`IClock` |
| Один тест со множеством актов | трудно локализовать поломку | разбивать |
| Тест-дубликат | замедляет прогон | поддерживать уникальность |
| Покрытие ради процентов | формализм | покрывать важную логику |

## Контрольные вопросы

1. Что такое AAA?
2. Чем `[Fact]` отличается от `[Theory]`?
3. Как проверить, что метод бросает исключение?
4. Зачем моки?
5. Что такое пирамида тестов?
6. Зачем тесты при рефакторинге?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг
- Рабочий пример с тестами: `xunit-demo`

# Практика

# Практика — Модуль 13

> Работаем в проекте `xunit-demo`. Сначала попробуй решить сам. Решения — в конце файла.

## Задача 1: Тесты на валидацию

**Сложность:** ★☆☆
**Описание:** В `ArithmeticTests` добавь тест, что `Divide` бросает `ArgumentException` при... нет, ты уже знаешь: добавь тест на `Divide` с отрицательным делимым.

**Требования:**

- `Assert.Equal` с точным ожиданием.
- Запусти `dotnet test`.

## Задача 2: Граничные случаи

**Сложность:** ★★☆
**Описание:** Добавь тесты на `Multiply` с `decimal.MaxValue` и `decimal.MinValue`.

**Требования:**

- Обрати внимание на переполнение.
- `Assert.Throws<OverflowException>` при умножении, уходящем за диапазон.

## Задача 3: Theory-тест пароля

**Сложность:** ★★☆
**Описание:** Добавь `[Theory]` с `[InlineData]` для граничных случаев длины пароля.

**Требования:**

- 7 символов → `false`; 8 символов, если правила выполнены → `true`.

## Задача 4: Мок времени

**Сложность:** ★★★
**Описание:** Проверь `Order.IsExpired` с зависимостью от времени — подмени её интерфейсом `IClock`.

**Требования:**

- Реализуй фейк-класс `FakeClock` (вручную, без библиотек).
- Протестируй «до», «в» и «после» дедлайна.

**Подсказка:** В проекте нет мок-библиотеки, поэтому свой `FakeClock : IClock` — отличная демонстрация.

## Задача 5: Тест исключений

**Сложность:** ★★☆
**Описание:** Проверь, что `PasswordValidator.IsValid(null!)` бросает `ArgumentException`.

**Требования:**

- Используй `Assert.Throws<ArgumentException>`.

## Решения

<details>
<summary>Задача 1 — решение</summary>
</details>

<details>
<summary>Задача 2 — решение</summary>
</details>

<details>
<summary>Задача 3 — решение</summary>
</details>

<details>
<summary>Задача 4 — решение</summary>
</details>

<details>
<summary>Задача 5 — решение</summary>
</details>

# Разбор кода

# Разбор кода — Модуль 13

> Формат: плохой тест → находим проблемы → улучшаем.

## Пример 1: Тест-«пустышка»

### Исходный код

```
[Fact]
public void TestSomething()
{
    var x = 1 + 1;
    // нет Assert — тест «ничего не проверяет»
}
```

### Что не так

- Без утверждений тест бесполезен: он всегда «проходит», даже если код сломан.

### Как лучше

```
[Fact]
public void Add_TwoNumbers_ReturnsSum()
{
    Assert.Equal(2, 1 + 1);
}
```

### Вывод

Каждый тест обязан заканчиваться осмысленным `Assert.*`.

## Пример 2: Хрупкий тест на время

### Исходный код

```
[Fact]
public void IsExpired_BeforeNow_ReturnsTrue()
{
    var service = new OrderService();
```

```
    var order = new Order(DateTime.Now.AddSeconds(30)); // зависит от текущего момента
    Assert.False(service.IsExpired(order)); // может успеть истеть, если тест долгий
}
```

### Что не так

- Поведение зависит от реальных часов. Прогон «на стыке» секунд даёт плавающий результат.

### Как лучше

```
 var service = new OrderService(new FakeClock(new DateTime(2024, 1, 1)));
var order = new Order(new DateTime(2024, 1, 2));
Assert.False(service.IsExpired(order)); // всегда false
```

### Вывод

Зависимости от времени — через `IClock`/параметры.

## Пример 3: Один тест с множеством актов

### Исходный код

```
 [Fact]
public void BankAccount_FullFlow()
{
    var acc = new BankAccount();
    acc.Deposit(100);
    Assert.Equal(100m, acc.Balance);
    acc.Withdraw(30);
    Assert.Equal(70m, acc.Balance);
    acc.Withdraw(80); // исключение
}
```

### Что не так

- Если падает «Withdraw(80)», непонятно, какой именно участок сломан. Один тест проверяет слишком много.

### Как лучше

```
 [Fact]
public void Deposit_AddsToBalance()
{
```

```
    var acc = new BankAccount();
    acc.Deposit(100);
    Assert.Equal(100m, acc.Balance);
}

[Fact]
public void Withdraw_ReducesBalance()
{
    var acc = new BankAccount { /* Начальный баланс */ };
    acc.Withdraw(30);
    Assert.Equal(70m, acc.Balance);
}
```

### Вывод

Дроби на отдельные сценарии — так проще локализовать поломку.

## Пример 4: Тест-дубликат

### Исходный код

```
[Fact] public void Add_1And2_Is3() => Assert.Equal(3, Arithmetic.Add(1, 2));
[Fact] public void Add_1And2_Is3Again() => Assert.Equal(3, Arithmetic.Add(1, 2));
```

### Что не так

- Дублирующийся тест замедляет прогон и не даёт нового покрытия.

### Как лучше

```
[Theory]
[InlineData(1, 2, 3)]
[InlineData(-1, 1, 0)]
[InlineData(0, 0, 0)]
public void Add_TwoNumbers_ReturnsSum(int a, int b, int expected)
{
    Assert.Equal(expected, Arithmetic.Add(a, b));
}
```

### Вывод

`[Theory]` с `InlineData` покрывает несколько случаев одним тестом.

## Пример 5: Тест на фреймворк

### Исходный код

```
 [Fact]
public void Linq_Where_Filters()
{
    var result = new[] { 1, 2, 3 }.Where(n => n > 1).ToList();
    Assert.Equal(new[] { 2, 3 }, result);
}
```

### Что не так

- Проверяется поведение LINQ — библиотеки платформы, а не наш код. Такой тест ничего не защищает.

### Как лучше

Тестировать собственную бизнес-логику:

```
 [Fact]
public void FilterActiveUsers_ReturnsOnlyActive()
{
    var users = new[] { new User("Иван", true), new User("Мария", false) };
    var result = UserService.FilterActive(users);
    Assert.Single(result);
}
```

### Вывод

Тестируй свой код, а не инструменты.

# Модуль 14: SOLID и паттерны проектирования

> **Цель:** проектировать код, который легко расширять и поддерживать; знать основные паттерны.

## Зачем это нужно

Код пишется один раз, а читается — много раз. SOLID — пять принципов, которые делают код **устойчивым к изменениям**: ты добавляешь новую функциональность, не ломая существующую. Паттерны — проверенные решения повторяющихся задач. Это — уровень middle-разработчика.

## Теория

### 1. SOLID

#### S — Single Responsibility (единая ответственность)

Класс должен иметь только одну причину для изменения.

```
// ПЛОХО: один класс делает всё
class OrderProcessor
{
    public void Process(Order order) { /* расчёт, БД, email */ }
}

// ХОРОШО: раздельные обязанности
class OrderCalculator { public decimal Calculate(Order order) => ...; }
class OrderRepository { public void Save(Order order) => ...; }
class EmailNotifier { public void Send(Order order) => ...; }
```

#### O — Open/Closed (открытость для расширения)

Классы открыты для расширения, но закрыты для модификации.

```
// ПЛОХО: switch по типу — каждый новый тип требует правки
double Area(Shape shape) => shape switch
{
    Circle c => Math.PI * c.Radius * c.Radius,
    Rectangle r => r.Width * r.Height,
    // новый тип → правка здесь
};
```

```
// ХОРОШО: расширение через полиморфизм
abstract class Shape { public abstract double Area(); }
class Circle : Shape { public override double Area() => ...; }
```

**L — Liskov (принцип Лисков)**

Наследники не должны ломать поведение базового класса. Если переопределённый метод бросает исключение там, где базовый «всегда работает» — иерархия плохая.

**I — Interface Segregation (разделение интерфейсов)**

Клиенты не должны зависеть от методов, которые не используют. Большие интерфейсы — дробить.

```
 // ПЛОХО: «жирный» интерфейс
interface IWorker { void Work(); void Eat(); void Sleep(); }

// ХОРОШО
interface IWorker { void Work(); }
interface IReportable { void PrintReport(); }
```

**D — Dependency Inversion (инверсия зависимостей)**

Высокоуровневые модули не должны зависеть от низкоуровневых — и те, и другие зависят от абстракций.

```
 // ПЛОХО: жёсткая зависимость
class OrderService
{
    private readonly SqlOrderRepository _repo = new(); // new внутри
}

// ХОРОШО: конструкторная инъекция абстракции
class OrderService
{
    private readonly IOrderRepository _repo;
    public OrderService(IOrderRepository repo) => _repo = repo;
}
```

## 2. Паттерны

**Factory** — централизованное создание объектов:

```
interface IPaymentProcessor { void Pay(decimal amount); }

class PaymentProcessorFactory
{
    public static IPaymentProcessor Create(PaymentMethod method) => method switch
    {
        PaymentMethod.CreditCard => new CreditCardProcessor(),
        PaymentMethod.PayPal => new PayPalProcessor(),
        _ => throw new ArgumentOutOfRangeException(nameof(method))
    };
}
```

**Strategy** — взаимозаменяемые алгоритмы за интерфейсом (как `IPaymentProcessor` выше).

**Observer** — через события (модуль 09): издатель публикует, подписчики реагируют.

**Singleton** — один экземпляр:

```
sealed class Logger
{
    public static Logger Instance { get; } = new();
    private Logger() { }
    public void Log(string msg) => Console.WriteLine(msg);
}
```

Осторожно: синглтон скрывает зависимости и мешает тестам.

**Dependency Injection** — внедрение зависимостей (вручную):

```
var repo = new JsonOrderRepository();
var service = new OrderService(repo);
var ui = new ConsoleUI(service);
```

### 3. Антипаттерны

- **God Class** — класс, которому «всё можно».
- **Magic Numbers** — числа без смысла.
- **Spaghetti Code** — запутанные переходы.
- **Golden Hammer** — «молоток» для всех проблем.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Всё в `Program` | не тестируется, не понятно | слои/классы |

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Статический «глобальный» сервис | скрытая зависимость | DI/конструктор |
| switch по enum в 5 местах | нарушает OCP | полиморфизм/стратегия |
| Интерфейс-«мусорка» | сложные реализации | ISP |

## Контрольные вопросы

1. Что такое единая ответственность?
2. В чём суть принципа Open/Closed?
3. Что такое инверсия зависимостей?
4. Что даёт паттерн Factory?
5. Когда нужен Strategy?
6. Почему синглтон контроверсен?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг

# Практика

# Практика — Модуль 14

> Сначала попробуй решить сам. Подсказка — если застрял. Решение — в конце файла, в свёрнутом блоке.

## Задача 1: Рефакторинг God-класса

**Сложность:** ★★★
**Описание:** Разбей монолитный `OrderProcessor` на классы по SRP.

**Требования:**

- `OrderValidator`, `OrderCalculator`, `OrderRepository`, `EmailNotifier`.
- Каждый — одна ответственность.

## Задача 2: Стратегия оплаты

**Сложность:** ★★☆
**Описание:** `IPaymentProcessor` + три реализации + выбор через Factory.

**Требования:**

- `CreditCardProcessor`, `PayPalProcessor`, `BankTransferProcessor`.
- `PaymentProcessorFactory.Create(PaymentMethod)`.

## Задача 3: Observer-уведомления

**Сложность:** ★★☆
**Описание:** Система уведомлений о заказе на событиях.

**Требования:**

- Событие `OrderPlaced`.
- Логгер и почтовый подписчик.

## Задача 4: Ручной DI-контейнер

**Сложность:** ★★★
**Описание:** Мини-контейнер на `Dictionary<Type, object>`.

**Требования:**

- `Register<TInterface, TImpl>()`, `Resolve<TInterface>()`.
- Объясни, почему это прототип, а не фреймворк.

## Задача 5: Применение DIP

**Сложность:** ★★☆
**Описание:** `OrderService` с `new` внутри → конструкторная инъекция.

**Требования:**

- Введи `IOrderRepository` через конструктор.
- Напиши тест с фейк-репозиторием (модуль 13).

## Задача 6: OCP-рефакторинг

**Сложность:** ★★★
**Описание:** switch по 6 типам в 3 местах → виртуальные методы.

**Требования:**

- Базовый `abstract` класс с виртуальным методом.
- Новый тип — просто новый класс.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 14

> Формат: плохой код → находим проблемы → рефакторим.

## Пример 1: God-класс Program.cs

### Исходный код

```
// Program.cs — 500 строк: меню, расчёты, ввод, сохранение, валидация
Console.WriteLine("Меню: 1 — добавить, 2 — показать");
// ... вся логика прям во входной точке
```

### Что не так

- Всё в одном месте: менять сложно, тестировать невозможно, конфликты неизбежны.

### Как лучше

```
// ConsoleUI — ввод/вывод
// OrderService — логика
// OrderRepository — хранение
// Program.cs — только связка
```

### Вывод

Входная точка — тонкая; логика — в классах с ответственностью.

## Пример 2: Статический «глобальный» сервис

### Исходный код

```
static class UserService
{
    public static void Save(User user) { /* new UserRepository() внутри */ }
}
```

### Что не так

- Скрытая зависимость от `UserRepository`: её нельзя подменить в тестах, код жёстко связан.

### Как лучше

```
 class UserService
{
    private readonly IUserRepository _repo;
    public UserService(IUserRepository repo) => _repo = repo;
    public void Save(User user) => _repo.Save(user);
}
```

### Вывод

Зависимости — через конструктор (DIP).

## Пример 3: switch по enum в 5 местах

### Исходный код

```
 // в пяти методах повторяется:
switch (order.Status)
{
    case OrderStatus.New: ...
    case OrderStatus.Paid: ...
}
```

### Что не так

- Новый статус — правки во всех пяти местах. Нарушен OCP.

### Как лучше

```
 abstract class OrderState
{
    public abstract string Describe();
}

class NewState : OrderState
{
    public override string Describe() => "Новый";
}
```

### Вывод

Распределяй поведение по полиморфным классам.

## Пример 4: Интерфейс-«мусорка»

### Исходный код

```
 interface IWorker
{
    void Work();
    void Eat();
    void Sleep();
    void PrintReport();
}
```

### Что не так

- Каждый клиент обязан реализовать всё, даже ненужное. Меняется один метод — страдают все.

### Как лучше

```
 interface IWorker { void Work(); }
interface IReportable { void PrintReport(); }
```

### Вывод

Тонкие интерфейсы по ролям (ISP).

## Пример 5: `new` внутри сервиса

### Исходный код

```
 class OrderService
{
    private readonly SqlOrderRepository _repo = new(); // жёсткая связь
}
```

### Что не так

- Нельзя подменить репозиторий (фейк для тестов), класс не переиспользуется.

### Как лучше

```
 class OrderService
{
    private readonly IOrderRepository _repo;
    public OrderService(IOrderRepository repo) => _repo = repo;
}
```

### Вывод

Конструкторная инъекция — основа тестируемости.

# Модуль 15: Итоговый проект — Task Tracker

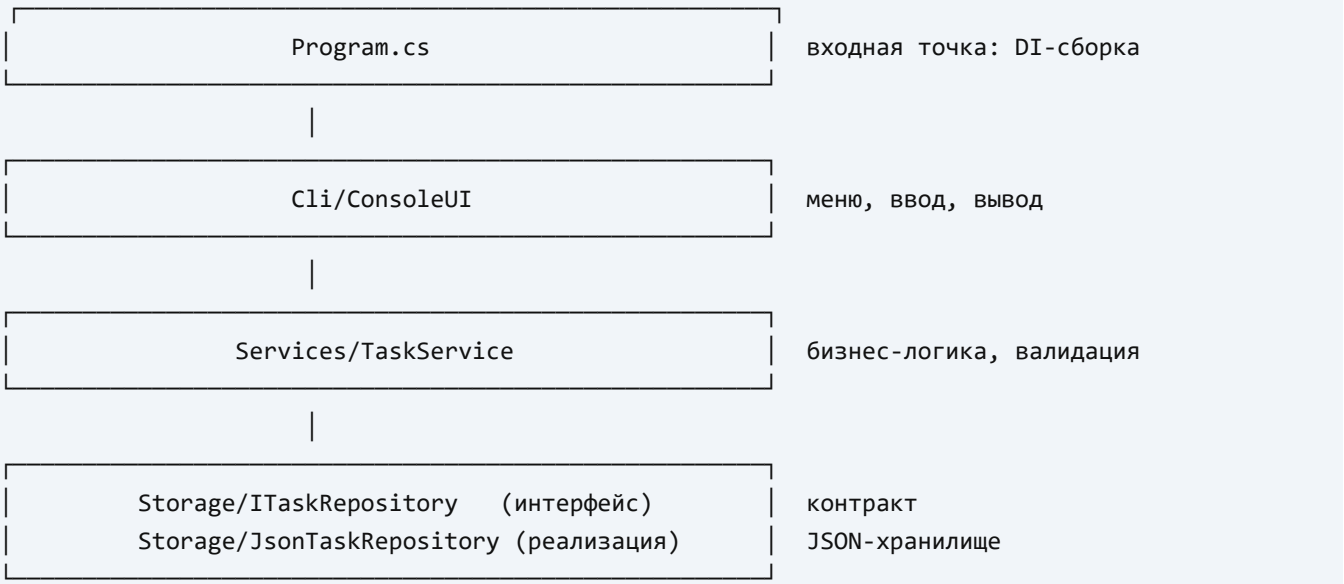
> **Цель:** собрать все знания курса в законченное приложение с архитектурой, слоями и тестами.

## О проекте

**Task Tracker** — консольный трекер задач. Поддерживает:

- добавление задач с приоритетом и дедлайном;
- просмотр, фильтрация по статусу;
- завершение, изменение приоритета, удаление;
- сохранение в JSON-файл при каждом изменении.

## Архитектура




Слои зависят вниз: UI → сервис → хранилище. Бизнес-логика **не знает** про JSON — она зависит от интерфейса `ITaskRepository`.

## Запуск и тесты

Из папки `task-tracker`:

```
# Запуск приложения
dotnet run --project src/TaskTracker
```

```
# Тесты
dotnet test
```

Данные сохраняются в `data/tasks.json` относительно текущей папки.

## Ключевые решения

| Решение | Почему |
|---|---|
| Слои (Models / Services / Storage / Cli) | единая ответственность (SRP), проект легко расширять |
| `ITaskRepository` + `JsonTaskRepository` | инверсия зависимостей (DIP): сервис можно тестировать без файлов |
| Своё исключение `InvalidTaskOperationException` | понятная диагностика ошибок домена |
| `record TaskItem` | неизменяемые данные, value-равенство |
| `required` + `init` | безопасное создание модели без «наполовину построенных» объектов |
| Асинхронность до конца (async/await) | не блокируем поток при работе с файлом |
| `CancellationToken` пробрасывается | отменяемость операций |
| Диагностика битого JSON (без падения) | приложение остаётся живым в худшем случае |

## Запущенный код

Каждый уровень архива — проверенный, собранный с `TreatWarningsAsErrors`, покрытый тестами. Запускать можно сразу.

## Дальше

- Практика — задания на расширение
- Разбор кода — решение «как надо»

# Практика

# Практика — Модуль 15

> Итоговые задания на расширение Task Tracker. Работаем в папке `task-tracker`.

## Задача 1: Поиск по названию

**Сложность:** ★★☆
**Описание:** Добавь в меню пункт «Поиск по названию».

**Требования:**

- Метод `TaskService.FindByTitleAsync(string query)` — поиск по подстроке (без учёта регистра).
- В `ConsoleUI` — пункт меню и ввод запроса.

## Задача 2: Категории

**Сложность:** ★★★
**Описание:** Добавь задачи поле `Category`.

**Требования:**

- `string Category { get; init; } = "Общее";`.
- Фильтр по категории в `GetAsync`.

## Задача 3: Статистика за неделю

**Сложность:** ★★★
**Описание:** Добавь «Сколько задач завершено за неделю».

**Требования:**

- Метод `TaskService.GetDoneCountSinceAsync(DateTime since)`.
- В UI — пункт меню.

## Задача 4: Экспорт в CSV

**Сложность:** ★★★
**Описание:** Экспортируй задачи в `export.csv`.

**Требования:**

- Заголовок: `Id;Title;Status;Priority;Deadline`.
- Вывод пути к файлу.

## Задача 5: Рефакторинг времени

**Сложность:** ★★★
**Описание:** Замени прямой `DateTime.UtcNow` в `TaskService` на абстракцию `IClock`.

**Требования:**

- `interface IClock { DateTime UtcNow { get; } }`.
- Конструкторная инъекция.
- Обнови тесты на `FakeClock` (как в модуле 13).

## Задача 6: Своё расширение

**Сложность:** ★★★
**Описание:** Придумай и реализуй одну функцию «под себя» (теги, повторяющиеся задачи, заметки).

**Требования:**

- Новый пункт меню.
- Не менее одного нового юнит-теста.

## Подсказки

- Модель `TaskItem` — `record`: чтобы изменить поле, используй `with` (как в `CompleteAsync`).
- Репозиторий тестируется через `InMemoryTaskRepository` (есть в тестах).
- Не забывай `TreatWarningsAsErrors` — код обязан собираться без предупреждений.

## Проверка

```
dotnet build && dotnet test
```

# Разбор кода

# Разбор кода — Модуль 15

> Формат: частые ошибки в подобных проектах → как избежать.

## Пример 1: Вся логика в `Program.cs`

### Плохо

```
// Program.cs на 400 строк: и меню, и расчёты, и JsonSerializer, и Console
```

### Почему

- Входная точка превращается в God-класс: трудно читать, тестировать, расширять.

### Как надо

Тонкий `Program.cs` с DI-сборкой + отдельные слои. Так и сделано в проекте.

## Пример 2: Бизнес-логика знает про JSON

### Плохо

```
 class TaskService
{
    public void Save() => File.WriteAllText("tasks.json", JsonSerializer.Serialize(...));
}
```

### Почему

- Сервис привязан к конкретному формату. Смена хранилища (например, на SQLite) — переписывание всего сервиса.

### Как надо

`ITaskRepository` + `JsonTaskRepository`. Сервис зависит от абстракции (DIP).

## Пример 3: `DateTime.Now` в сервисе

### Плохо

```
if (deadline <= DateTime.Now) throw ...;
```

### Почему

- Непредсказуемые тесты и поведение. Локальное время не для логики.

### Как надо

`DateTime.UtcNow` — или, лучше, `IClock` (задача 5 практики) для полной детерминированности.

## Пример 4: Битый JSON — молча

### Плохо

```
 catch (Exception)
{
    // ничего — тихо теряем данные
}
```

### Почему

- Пользователь не узнает о проблеме; приложение продолжит работу с «пустыми» данными.

### Как надо

- Ловить `JsonException` (не `Exception` всё подряд).
- Хотя бы диагностировать (`Console.Error.WriteLine`), не падая.

## Пример 5: Возврат `List<T>` наружу

### Плохо

```
public List<TaskItem> GetAll() => _items;
```

### Почему

- Потребитель может изменить хранилище напрямую, минуя бизнес-логику.

### Как надо

```
public IReadOnlyList<TaskItem> GetAll() => _items;
```

## Пример 6: Всё `public` без XML-доков

### Плохо

```
public class TaskService
{
    public Task<TaskItem> CreateAsync(...) // без <summary>
}
```

### Почему

- С `TreatWarningsAsErrors` + `GenerateDocumentationFile` проект просто не соберётся (CS1591). Документация — не роскошь, а требование.

### Как надо

Публичные члены — с XML-доками. Так и сделано в проекте.

# Модуль 16: Git и рабочий процесс (бонус)

> **Цель:** уверенно работать с Git в терминале: история, ветки, слияния, pull request.

## Зачем это нужно

C# ты выучил, но в реальной команде код живёт в Git. Это система контроля версий: она хранит историю изменений, позволяет работать нескольким людям одновременно и откатываться к рабочему состоянию. GitHub/ GitLab — это «облачные» хостинги Git, где живут репозитории, ведётся код-ревью и крутится CI. Без Git не проходит ни один рабочий день разработчика.

## Теория

### 1. Основные понятия

| Термин | Значение |
|---|---|
| **Репозиторий** ( `repo` ) | папка с проектом + история изменений |
| **Коммит** ( `commit` ) | «снимок» состояния проекта в момент времени |
| **Ветка** ( `branch` ) | параллельная линия разработки |
| **Merge** | слияние веток |
| **Remote** | удалённый репозиторий (например, на GitHub) |

### 2. Базовый цикл

```
 git init              # создать репозиторий
git status            # что изменилось
git add <file>        # проиндексировать изменения
git commit -m "сообщение"
git log --oneline     # история коммитов
```

**Рабочий процесс «изменить → закоммитить»:**

```
 # 1. Изменил код
git status            # видно, что изменилось
git add src/Program.cs
git commit -m "fix: исправил вылет в меню"
```

## 3. Gitignore

Правило: не коммитить артефакты. Для .NET это `bin/`, `obj/`, `*.user`. Всё это уже есть в `.gitignore` курса.

## 4. Ветки

```
git branch feat/new-feature   # создать ветку
git checkout feat/new-feature # переключиться
git switch -c feat/new-feature # создать и переключиться сразу
```

**Основное правило:** `main` — всегда рабочая; новые фичи — в своих ветках, затем merge через pull request.

## 5. Merge и rebase

**Merge** — создаёт коммит слияния:

```
git checkout main
git merge feat/new-feature
```

**Rebase** — перекладывает твои коммиты на актуальную вершину:

```
git pull --rebase origin main
```

**Правило:** никогда не rebase запушенные чужие коммиты — история уже публична.

## 6. Откаты

```
git revert <commit>   # отменить опубликованный коммит (безопасно)
git reset --soft HEAD~1  # отменить локальный коммит, сохранив изменения
git reset --hard HEAD~1  # отменить локальный коммит полностью
```

## 7. Conventional Commits

Типы коммитов: `feat`, `fix`, `refactor`, `test`, `docs`, `perf`, `build`, `ci`, `chore`.

```
fix(api): return 404 when user missing
feat(ui): add dark theme
docs: add architecture diagram
```

## 8. GitHub: pull request

1. Создал ветку, закоммитил, запушил: `git push -u origin feat/...`.
2. На GitHub — «Compare & pull request».
3. В описании: что меняешь и зачем, как проверено.
4. Код-ревью → merge.

### 9. CI/CD

GitHub Actions выполняет проверки на каждый push/PR: сборка, линт, тесты. Если проверки не прошли — PR не мёржится. В этом курсе CI настроен в `.github/workflows/build.yml`.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Коммит `bin/`, `obj/` | артефакты в истории, конфликты | `.gitignore` |
| Коммит секретов | пароли навсегда в истории | никогда не коммитить `.env` |
| Огромные коммиты | сложно ревьюить и откатывать | маленькие логичные коммиты |
| `git reset --hard` на запушенном | теряется чужая работа | `git revert` |
| Сообщение «asdf» | через месяц непонятно | conventional commits |

## Контрольные вопросы

1. Что такое коммит?
2. Как посмотреть историю?
3. Зачем ветки?
4. Чем merge отличается от rebase?
5. Как безопасно отменить опубликованный коммит?
6. Что такое pull request?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки
- Загляни в `.github/workflows/build.yml` — реальный пример CI из этого курса

# Практика

# Практика — Модуль 16

> Выполняй в отдельной учебной папке (НЕ в этом репозитории).

## Задача 1: Первый репозиторий

**Сложность:** ★☆☆
**Описание:** Создай проект, проинициализируй git, сделай первый коммит.

**Требования:**

- `git init`.
- Создай файл `README.md`, добавь его (`git add`), закоммить.
- Посмотри `git log --oneline`.

## Задача 2: Ветки

**Сложность:** ★★☆
**Описание:** Создай ветку, внеси изменения, смержи в main.

**Требования:**

- `git switch -c feature/hello`.
- Добавь код/текст, закоммить.
- Вернись в main (`git switch main`) и сделай `git merge feature/hello`.

## Задача 3: Конфликт

**Сложность:** ★★★
**Описание:** Провоцируй конфликт слияния и разреши его.

**Требования:**

- В main и в ветке измени одну и ту же строку файла.
- При merge получишь конфликт: разбери маркеры `<<<<<<<` / `=======` / `>>>>>>>`, выбери правильный вариант, закоммить.

## Задача 4: Откат

**Сложность:** ★★★
**Описание:** Освой `git revert` и `git reset`.

**Требования:**

- Сделай плохой коммит.
- Отмени его через `git revert <sha>`.
- Пойми, почему revert безопаснее reset в публичной истории.

## Задача 5: Conventional Commits

**Сложность:** ★☆☆
**Описание:** Перепиши свои будущие коммиты в формате conventional.

**Требования:**

- Используй `feat:`, `fix:`, `docs:` и т.д.

## Задача 6: Исследуй CI курса

**Сложность:** ★★☆
**Описание:** Прочитай `.github/workflows/build.yml` курса.

**Требования:**

- Пойми, что делает каждый шаг.
- Ответь: зачем `--configuration Release` и `--no-build`?

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 16

> Формат: плохие практики Git → почему → как правильно.

## Пример 1: Коммит с `bin/` и `obj/`

### Плохо

```
git add -A
git commit -m "feat: done"
# в коммит попали bin/ и obj/
```

### Почему

- Артефакты сборки засоряют историю, создают конфликты и увеличивают размер репозитория.

### Как надо

```
# в .gitignore:
bin/
obj/
*.user
```

## Пример 2: Секреты в истории

### Плохо

```
git add .env
git commit -m "chore: add env"
```

### Почему

- Даже если потом удалить файл, секрет останется в истории навсегда. Ротация ключа не отменяет утечки.

### Как надо

- Никогда не коммитить `.env`, ключи, токены.
- Если уже закоммитил — немедленно отзови ключ и очисти историю.

## Пример 3: `git reset --hard` на запушенном

### Плохо

```
git reset --hard HEAD~3
git push --force
```

### Почему

- Переписывает публичную историю и удаляет работу других разработчиков.

### Как надо

```
git revert <sha>   # безопасный обратный коммит
```

## Пример 4: Сообщение коммита «asdf»

### Плохо

```
git commit -m "asdf"
```

### Почему

- Через месяц никто не поймёт, что менялось. `git bisect` и changelog бесполезны.

### Как надо

```
git commit -m "fix(auth): handle null token"
```

## Пример 5: Огромный коммит

### Плохо

```
git commit -m "big update"
# переделал весь проект одним коммитом
```

### Почему

- Сложно ревьюить, откатывать и связывать с задачами.

### Как надо

- Маленькие коммиты по одному логическому изменению.

## Пример 6: Пуш без проверок

### Плохо

```
git push  # не прогнал тесты и линт
```

### Почему

- Ломаешь CI, и коллеги получают сломанный код.

### Как надо

```
dotnet build && dotnet test
git push
```

## Вывод

Git — это дисциплина. Хорошая история коммитов и правильная работа с ветками делают проект понятным для команды и безопасным для откатов.

# Модуль 17: Введение в ASP.NET Core

> **Цель:** построить первое веб-API на ASP.NET Core и понять, как серверы обрабатывают HTTP-запросы.

## Зачем это нужно

Ты прошёл C# и Git. Теперь самое востребованное применение языка — **веб-разработка**: REST-API, бэкенд мобильных приложений, сервисы, микросервисы. ASP.NET Core — современный кроссплатформенный веб-фреймворк от Microsoft. Умение писать API — прямой путь к работе backend-разработчиком.

## Теория

### 1. Как работает веб

Клиент (браузер, мобильное приложение) отправляет **HTTP-запрос** серверу, сервер отвечает **HTTP-ответом**.

Ключевые вещи запроса:

| Понятие | Пример |
|---|---|
| Метод | `GET`, `POST`, `PUT`, `DELETE` |
| Путь (URL) | `/api/tasks` |
| Заголовки | `Content-Type: application/json` |
| Тело (body) | JSON-данные |

Семинатика методов: `GET` — получить, `POST` — создать, `PUT` — обновить целиком, `DELETE` — удалить.

Коды ответов: `200 OK`, `201 Created`, `400 Bad Request`, `404 Not Found`, `500 Server Error`.

### 2. Что такое ASP.NET Core

- **Кроссплатформенный** — работает на Windows, Linux, macOS.
- **Встроенный веб-сервер** — Kestrel.
- **Minimal API** и **Controllers** — два стиля написания API.
- Мощный встроенный контейнер **DI**.

## 3. Minimal API

Самый простой способ:

```
var app = WebApplication.CreateBuilder(args).Build();

app.MapGet("/hello", () => "Привет, мир!");
app.MapPost("/sum", (SumRequest r) => r.A + r.B);

app.Run();
```

Каждый `app.Map...` — обработчик маршрута. Параметры приходят из query, пути или тела запроса.

## 4. Структура типичного проекта

```
src/TaskApi/
├── Program.cs          # конфигурация, DI, маршруты
├── Models/             # DTO и модели
├── Services/           # бизнес-логика
├── Storage/            # доступ к данным (репозиторий)
└── appsettings.json     # конфигурация
```

## 5. Dependency Injection

В ASP.NET Core DI встроен:

```
builder.Services.AddSingleton<ITaskRepository, InMemoryTaskRepository>();
builder.Services.AddScoped<TaskService>();
```

Контейнер сам создаёт зависимости и внедряет их в обработчики (параметры методов) и конструкторы. Это тот самый DIP из модуля 14 — только теперь автоматически.

## 6. Middleware и конвейер

Запрос проходит через цепочку middleware. По умолчанию в minimal API уже есть маршрутизация.

```
app.Use(async (context, next) =>
{
    Console.WriteLine($"Запрос: {context.Request.Method} {context.Request.Path}");
    await next();
});
```

### 7. Сериализация JSON

ASP.NET Core по умолчанию работает с System.Text.Json: объекты сами превращаются в JSON и обратно. Имена свойств — `camelCase`.

### 8. Запуск и тестирование

```
 dotnet run --project src/TaskApi
# GET http://localhost:5000/api/tasks
```

Для тестов API есть `WebApplicationFactory<Program>`, который поднимает сервер в памяти (см. `minimal-api-demo/tests`).

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Вся логика в `Program.cs` | God-метод, нетестируемо | вынести в сервисы |
| `new` сервисов внутри обработчиков | нельзя подменить для тестов | DI |
| Возврат всего `OK` | не информативно для клиента | корректные коды (`201`, `400`) |
| Синхронные вызовы в async | блокирует поток | `async` / `await` всюду |
| Не использовать `CancellationToken` | отмену нельзя пробросить | принимать и передавать дальше |

## Контрольные вопросы

1. Чем `GET` отличается от `POST`?
2. Что такое Minimal API?
3. Зачем DI в ASP.NET Core?
4. Что такое middleware?
5. Как протестировать API без реального запуска сервера?
6. Что означает статус-код 201?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки и рефакторинг
- Рабочий пример: `minimal-api-demo`

# Практика

# Практика — Модуль 17

> Работаем в проекте `minimal-api-demo`. Сначала попробуй решить сам. Решения — в конце файла.

## Задача 1: Добавить `GET /api/tasks/{id}`

**Сложность:** ★★☆
**Описание:** Верни задачу по id или 404, если её нет.

**Требования:**

- `TaskService.GetByIdAsync(Guid id)`.
- `Results.Ok` / `Results.NotFound`.

## Задача 2: `DELETE /api/tasks/{id}`

**Сложность:** ★★☆
**Описание:** Удали задачу; 204 при успехе, 404 если нет.

**Требования:**

- `Results.NoContent()` — это 204.
- Добавь `RemoveAsync` в `ITaskRepository`.

## Задача 3: Логирование через middleware

**Сложность:** ★★☆
**Описание:** Добавь middleware, логирующий метод и путь каждого запроса.

**Требования:**

- `app.Use(...)`, выводи через `app.Logger.LogInformation`.

## Задача 4: Валидация через guard

**Сложность:** ★★☆
**Описание:** В `TaskService` добавь проверку: id не пустой Guid → `ArgumentException`.

**Требования:**

- Покрой тестом: `GET /api/tasks/00000000-0000-0000-0000-000000000000` → `400`.

## Задача 5: Тест на 404

**Сложность:** ★★★
**Описание:** Напиши интеграционный тест: запрос к несуществующему id возвращает 404.

**Требования:**

- Используй `WebApplicationFactory<Program>` и случайный Guid.

## Задача 6: Средний балл задач

**Сложность:** ★★★
**Описание:** Добавь `GET /api/tasks/stats` — число задач и сколько завершено.

**Требования:**

- Верни объект `TaskStats { Total, Done }`.

## Решения

▶ Задача 1 — решение

▶ Задача 2 — решение

▶ Задача 3 — решение

▶ Задача 4 — решение

▶ Задача 5 — решение

▶ Задача 6 — решение

# Разбор кода

# Разбор кода — Модуль 17

> Формат: плохой веб-код → находим проблемы → рефакторим.

## Пример 1: Вся логика в `Program.cs`

### Исходный код

```
 app.MapPost("/api/orders", async (OrderRequest r) =>
{
    if (string.IsNullOrWhiteSpace(r.Title)) return Results.BadRequest();
    var order = new Order { Id = Guid.NewGuid(), Title = r.Title };
    // ... валидация, БД, email, логирование прямо здесь
    return Results.Created($"/api/orders/{order.Id}", order);
});
```

### Что не так

- Обработчик превращается в God-метод: и валидация, и бизнес-логика, и доступ к данным. Не тестируется, не переиспользуется.

### Как лучше

```
 app.MapPost("/api/orders", async (OrderRequest r, OrderService service, CancellationToken
token) =>
{
    try
    {
        var order = await service.CreateAsync(r.Title, token);
        return Results.Created($"/api/orders/{order.Id}", order);
    }
    catch (ArgumentException ex)
    {
        return Results.BadRequest(ex.Message);
    }
});
```

### Вывод

Обработчик — тонкий; логика — в сервисе.

## Пример 2: `new` сервиса внутри обработчика

### Исходный код

```
app.MapGet("/api/tasks", async () =>
{
    var service = new TaskService(new InMemoryTaskRepository());
    return Results.Ok(await service.GetAllAsync());
});
```

### Что не так

- Жёсткая привязка: нельзя подменить репозиторий для тестов. DI не используется.

### Как лучше

```
app.MapGet("/api/tasks", async (TaskService service, CancellationToken token) =>
    Results.Ok(await service.GetAllAsync(token)));
```

### Вывод

Параметры-зависимости DI входят в сигнатуру обработчика.

## Пример 3: Всегда `OK` вместо корректных кодов

### Исходный код

```
app.MapGet("/api/tasks/{id}", async (Guid id, TaskService service) =>
{
    var task = await service.GetByIdAsync(id);
    return Results.Ok(task); // даже если task == null
});
```

### Что не так

- Клиент не может понять, существует ли задача: получит 200 с null в теле.

### Как лучше

```
return task is null ? Results.NotFound() : Results.Ok(task);
```

### Вывод

Используй семантически верные коды: 404, 400, 201.

## Пример 4: Синхронный вызов в async-цепочке

### Исходный код

```
app.MapGet("/api/report", () =>
{
    var data = _repo.GetAll().Result; // блокирует поток
    return Results.Ok(data);
});
```

### Что не так

- Блокировка потока; под нагрузкой — исчерпание пула потоков и дедлоки.

### Как лучше

```
app.MapGet("/api/report", async (ITaskRepository repo, CancellationToken token) =>
    Results.Ok(await repo.GetAllAsync(token)));
```

### Вывод

`async` / `await` до конца цепочки.

## Пример 5: Проглатывание исключений

### Исходный код

```
try { return await service.GetAllAsync(); }
catch (Exception) { return Results.Ok(Array.Empty<TaskItem>()); }
```

### Что не так

- Баг маскируется пустым списком. Клиент не видит, что произошла ошибка.

### Как лучше

Пусть глобальный обработчик исключений вернёт 500, а в коде ловим только ожидаемые:

```
try { ... }
catch (ArgumentException ex) { return Results.BadRequest(ex.Message); }
```

### Вывод

Лови ожидаемое; непредвиденное — 500 с логом.

# Модуль 18: Базы данных и EF Core

> **Цель:** работать с реляционными БД через ORM и понимать миграции.

## Зачем это нужно

Ни одно серьёзное приложение не обходится без хранения данных. Веб-API из модуля 17 использовало in-memory хранилище — в реальности данные живут в базе. **Entity Framework Core (EF Core)** — ORM от Microsoft: ты работаешь с C#-объектами, а EF Core сам превращает их в SQL-запросы. Это — стандарт индустрии для .NET.

## Теория

### 1. Что такое ORM и EF Core

**ORM** (Object-Relational Mapping) — прослойка между объектами C# и таблицами БД.

**EF Core** даёт:

- `DbContext` — модель базы как C#-класс;
- `DbSet<T>` — представление таблицы;
- **миграции** — версионирование схемы БД;
- **LINQ-запросы** — транслируются в SQL.

### 2. Провайдеры

EF Core требует провайдер под конкретную БД:

| Провайдер | БД |
|---|---|
| `Microsoft.EntityFrameworkCore.SqlServer` | SQL Server |
| `Microsoft.EntityFrameworkCore.Sqlite` | SQLite (файловая) |
| `Microsoft.EntityFrameworkCore.InMemory` | in-memory (тесты, демо) |
| `Npgsql.EntityFrameworkCore.PostgreSQL` | PostgreSQL |

Устанавливается пакетом: `dotnet add package Microsoft.EntityFrameworkCore.Sqlite`.

### 3. Контекст и сущности

```
public class AppDbContext : DbContext
{
    public DbSet<TaskItem> Tasks => Set<TaskItem>();
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
}
```

Сущность:

```
public class TaskItem
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Title { get; set; } = "";
    public TaskStatus Status { get; set; }
}
```

## 4. Запросы через LINQ

```
var openTasks = await db.Tasks
    .Where(t => t.Status == TaskStatus.Todo)
    .OrderBy(t => t.Title)
    .ToListAsync();
```

EF Core переводит LINQ в SQL. **Важно:** `IQueryable<T>` — отложенный запрос; `ToListAsync` / `FirstOrDefaultAsync` материализуют.

## 5. `AsNoTracking`

Для чтения без отслеживания изменений:

```
var snapshot = await db.Tasks.AsNoTracking().ToListAsync();
```

Ускоряет чистое чтение.

## 6. Миграции

Миграция — «слепок» схемы БД:

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Команды генерируют классы-миграции, из которых EF Core строит таблицы.

### 7. Сохранение

```
db.Tasks.Add(new TaskItem { Title = "Новая" });
await db.SaveChangesAsync();   // выполняет INSERT
```

Все изменения в контексте применяются одним `SaveChangesAsync`.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| `ToList()` в цикле | выполнение запроса на каждой итерации | материализация один раз |
| Не `async`-методы EF | блокировка потока | `ToListAsync`, `SaveChangesAsync` |
| Хранение `decimal`/даты как строк | потери точности/сортировки | правильные типы столбцов |
| `AsNoTracking` забыт для чтения | лишние накладные на отслеживание | добавлять для чисто чтения |

## Контрольные вопросы

1. Что такое ORM?
2. Что делает `DbContext`?
3. Чем `IQueryable<T>` отличается от материализованного списка?
4. Что такое миграции?
5. Когда нужен `AsNoTracking`?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки
- Рабочий пример: `efcore-demo`

# Практика

# Практика — Модуль 18

> Работаем в проекте `efcore-demo`. Сначала попробуй решить сам. Решения — в конце файла.

## Задача 1: Фильтрация по статусу

**Сложность:** ★☆☆
**Описание:** Выведи только задачи со статусом `InProgress`.

**Требования:**

- `Where(t => t.Status == TaskStatus.InProgress)`.
- `ToListAsync`.

## Задача 2: Обновление задачи

**Сложность:** ★★☆
**Описание:** Найди задачу по названию и смени ей статус.

**Требования:**

- `FirstOrDefaultAsync`, измени свойство, `SaveChangesAsync`.

## Задача 3: Удаление

**Сложность:** ★★☆
**Описание:** Удали задачу с конкретным названием.

**Требования:**

- `Remove(entity)` + `SaveChangesAsync`.

## Задача 4: Счётчик

**Сложность:** ★☆☆
**Описание:** Посчитай общее число задач.

**Требования:**

- `CountAsync`.

## Задача 5: Пагинация

**Сложность:** ★★★
**Описание:** Выведи вторую «страницу» по 2 задачи.

**Требования:**

- `OrderBy` → `Skip(2)` → `Take(2)`.

## Задача 6: Тест на обновление

**Сложность:** ★★★
**Описание:** Напиши тест: добавить → сменить статус → проверить.

**Требования:**

- In-memory БД с уникальным именем.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 18

> Формат: плохой код с EF Core → находим проблемы → рефакторим.

## Пример 1: `ToList()` в цикле

### Исходный код

```
foreach (var title in titles)
{
    var tasks = await db.Tasks.Where(t => t.Title == title).ToListAsync();
    // обработка...
}
```

### Что не так

- Запрос к БД на каждой итерации — N запросов вместо одного (N+1).

### Как лучше

```
var tasks = await db.Tasks.Where(t => titles.Contains(t.Title)).ToListAsync();
```

### Вывод

Один запрос `WHERE IN (...)` вместо N отдельных.

## Пример 2: Синхронные методы EF

### Исходный код

```
var tasks = db.Tasks.ToList(); // блокирует поток
```

### Что не так

- Блокировка потока под нагрузкой. Все методы EF имеют async-версии.

### Как лучше

```
var tasks = await db.Tasks.ToListAsync();
```

### Вывод

`async` / `await` для всех операций с БД.

## Пример 3: Сравнение enum со строкой

### Исходный код

```
db.Tasks.Where(t => t.Status.ToString() == "Done")
```

### Что не так

- `ToString()` не транслируется в SQL корректно; зависит от культуры/регистра.

### Как лучше

```
db.Tasks.Where(t => t.Status == TaskStatus.Done)
```

### Вывод

Используй типизированные enum-сравнения.

## Пример 4: Загрузка всех сущностей для одной

### Исходный код

```
 var all = await db.Tasks.ToListAsync();
var one = all.FirstOrDefault(t => t.Id == id);
```

### Что не так

- Тянем всю таблицу в память ради одной строки.

### Как лучше

```
var one = await db.Tasks.FirstOrDefaultAsync(t => t.Id == id);
```

### Вывод

Пусть запрос сужает выборку на стороне БД.

## Пример 5: Забытый `SaveChangesAsync`

### Исходный код

```
db.Tasks.Add(new TaskItem { Title = "Без сохранения" });
// нет SaveChangesAsync() — добавление потеряется
```

### Что не так

- Изменения не применяются к БД, пока не вызван `SaveChangesAsync`.

### Как лучше

```
db.Tasks.Add(new TaskItem { Title = "С сохранением" });
await db.SaveChangesAsync();
```

### Вывод

`SaveChangesAsync` — обязательный шаг после изменений.

# Модуль 19: Продвинутая архитектура

> **Цель:** проектировать приложения, которые легко расширять, тестировать и поддерживать на уровне больших проектов.

## Зачем это нужно

Когда проект растёт, «просто классы» перестают спасать. Нужна **архитектура** — правила, как раскладывать код по слоям, чтобы доменная логика не зависела от фреймворков и БД. Это и есть уровень senior: уметь видеть большую систему и принимать архитектурные решения.

## Теория

### 1. Слои и направления зависимостей

Классическая трёхслойная модель:

```
Presentation (UI/API) → Application (сервисы) → Infrastructure (БД, внешние сервисы)
```

Зависимости направлены **внутрь**, к домену. Домен — ядро, ни от чего не зависящее.

### 2. Clean Architecture

Популярная реализация идеи слоёв (по Роберту Мартину):

```
┌─────────────────────────────────┐
│   Presentation (API/UI)         │  внешний слой
├─────────────────────────────────┤
│   Application (сервисы,         │  use cases, команды
│   CQRS, медиаторы)              │
├─────────────────────────────────┤
│   Domain (сущности,             │  бизнес-правила
│   value objects, интерфейсы)    │
├─────────────────────────────────┤
│   Infrastructure (БД, HTTP,     │  детали: EF Core, HttpClient
│   почта)                        │
└─────────────────────────────────┘
```

**Ключевое правило:** зависимости направлены к центру. `Domain` не знает про `Infrastructure`. Интерфейсы объявляются в `Application` / `Domain`, а реализуются в `Infrastructure` (DIP).

### 3. Паттерн Repository

Одна из реализаций DIP для данных:

```
 // объявлен в Application
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(Guid id);
}

// реализован в Infrastructure
public sealed class EfOrderRepository : IOrderRepository
{
    private readonly AppDbContext _db;
    public async Task<Order?> GetByIdAsync(Guid id) =>
        await _db.Orders.FirstOrDefaultAsync(o => o.Id == id);
}
```

### 4. CQRS (кратко)

**CQRS** — разделение чтения и записи: `Command` (изменения) и `Query` (чтение) — разные модели. Упрощает масштабирование и оптимизацию чтения.

### 5. MediatR (упоминание)

Паттерн Mediator — команды распространяются через медиатор, а не через прямые ссылки. Популярен с CQRS.

### 6. Подход: сначала контракты

Проектируй API и интерфейсы до реализации. Контракты — «скелет» системы.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Домен знает про EF Core | смена БД ломает домен | DIP, интерфейсы в домене |
| Всё в `Infrastructure` | логика теряется | Application для use cases |
| Гигантский слой сервисов | отсутствие границ | разбивать по функционалу |
| Нет интерфейсов на границах | нельзя подменить для тестов | контракты |

## Контрольные вопросы

1. Куда направлены зависимости в Clean Architecture?
2. Какой слой не зависит ни от чего?
3. Что решает паттерн Repository?
4. Что такое CQRS?
5. Зачем интерфейсы на границах слоёв?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки

# Практика

# Практика — Модуль 19

> Сначала попробуй решить сам. Решения — в конце файла.

## Задача 1: Карта проекта

**Сложность:** ★☆☆
**Описание:** Спроектируй структуру папок для Web API «Магазин».

**Требования:**

- Слои: Domain, Application, Infrastructure, Presentation.
- Опиши, что лежит в каждом.

## Задача 2: Интерфейс репозитория

**Сложность:** ★★☆
**Описание:** Объяви `IProductRepository` в Application.

**Требования:**

- `GetByIdAsync`, `GetAllAsync`, `AddAsync`.
- Без зависимостей от EF.

## Задача 3: Реализация репозитория

**Сложность:** ★★☆
**Описание:** Реализуй `EfProductRepository` на EF Core в Infrastructure.

**Требования:**

- Наследует `IProductRepository`.
- Использует `AppDbContext`.

## Задача 4: Сервис-применение

**Сложность:** ★★☆
**Описание:** `ProductService` (Application) использует `IProductRepository`.

**Требования:**

- Конструкторная инъекция.
- Бизнес-логика (например, проверка цены).

## Задача 5: Регистрация в DI

**Сложность:** ★★☆
**Описание:** Зарегистрируй интерфейс и реализацию.

**Требования:**

- `AddScoped<IProductRepository, EfProductRepository>()`.

## Задача 6: Тест с фейком

**Сложность:** ★★★
**Описание:** Протестируй `ProductService`, подменив `IProductRepository` фейком.

**Требования:**

- Фейк-класс в тестах.
- Проверь бизнес-логику.

## Решения

▶ **Задача 1 — решение**

▶ **Задача 2 — решение**

▶ **Задача 3 — решение**

▶ **Задача 4 — решение**

▶ **Задача 5 — решение**

▶ **Задача 6 — решение**

# Разбор кода

# Разбор кода — Модуль 19

> Формат: плохая архитектура → находим проблемы → рефакторим.

## Пример 1: Домен знает про EF Core

### Исходный код

```
// Domain/Entities/Order.cs
public class Order
{
    public void Save() => new AppDbContext().SaveChanges(); // зависимость от EF!
}
```

### Что не так

- Домен связан с инфраструктурой. Смена БД ломает домен; тесты невозможны без EF.

### Как лучше

Домен держит только бизнес-правила. Сохранение — через репозиторий в Infrastructure.

### Вывод

Домен не должен знать про БД.

## Пример 2: Сервис-«каша» без границ

### Исходный код

```
public class OrderService
{
    // и создание, и оплата, и отчёт, и email — всё здесь
}
```

### Что не так

- Класс разрастается, нарушения SRP, сложно поддерживать.

### Как лучше

Разбить на `OrderCreationService`, `OrderPaymentService`, `OrderReportService`, `NotificationService`.

### Вывод

Use cases — отдельными классами.

## Пример 3: Нет интерфейсов на границах

### Исходный код

```
public class OrderService
{
    private readonly EfOrderRepository _repo = new();
}
```

### Что не так

- Жёсткая связь с конкретной реализацией. Тесты требуют настоящую БД.

### Как лучше

```
public class OrderService
{
    private readonly IOrderRepository _repo;
    public OrderService(IOrderRepository repo) => _repo = repo;
}
```

### Вывод

Контракты на границах слоёв обязательны.

## Пример 4: Вся логика в инфраструктуре

### Исходный код

```
public class AppDbContext : DbContext
{
    // сюда добавили и расчёты скидок, и валидацию
}
```

### Что не так

- Контекст — это не место для бизнес-логики.

### Как лучше

Валидация и расчёты — в Application/Domain; контекст отвечает за доступ к данным.

### Вывод

Инфраструктура хранит данные, а не правила.

## Пример 5: Прямые ссылки между сервисами

### Исходный код

```
 public class CheckoutService
{
    private readonly PaymentService _payment = new();
    private readonly InventoryService _inventory = new();
}
```

### Что не так

- Сервисы жёстко связаны; переиспользование и тестирование затруднены.

### Как лучше

Инъекция через конструктор + интерфейсы. При большом числе зависимостей — MediatR/Mediator.

### Вывод

Зависимости — через DI и контракты.

# Модуль 20: Наблюдаемость — логирование, метрики, трейсинг

> **Цель:** проектировать сервисы так, чтобы их было видно и можно было диагностировать в проде.

## Зачем это нужно

В разработке код «работает», а в проде — «лежит». Наблюдаемость (observability) — способность понять, что происходит в системе: **логи** (что случилось), **метрики** (как быстро/много), **трейсы** (где потрачено время). Это отличает senior: он пишет код, который можно расследовать.

## Теория

### 1. Три столпа наблюдаемости

| Столп | Вопрос | Инструмент в .NET |
|---|---|---|
| **Логи** | что произошло? | `ILogger<T>` |
| **Метрики** | насколько загружена система? | `Meter`, OpenTelemetry |
| **Трейсы** | где время в запросе? | `Activity`, OpenTelemetry (Tracing) |

### 2. Логирование: `ILogger<T>`

В ASP.NET Core логирование встроено через DI:

```
 public class OrderService
{
    private readonly ILogger<OrderService> _logger;
    public OrderService(ILogger<OrderService> logger) => _logger = logger;

    public async Task CreateAsync(Order order)
    {
        _logger.LogInformation("Создание заказа {OrderId}", order.Id);
        // ...
        _logger.LogInformation("Заказ создан {OrderId} за {ElapsedMs} мс", order.Id, 12);
    }
}
```

**Структурированное логирование** — сообщение с параметрами, а не конкатенация:

```
// ПЛОХО:
_logger.LogInformation($"Создан заказ {order.Id}");
// ХОРОШО (параметры отдельно):
_logger.LogInformation("Создан заказ {OrderId}", order.Id);
```

Уровни: `Trace`, `Debug`, `Information`, `Warning`, `Error`, `Critical`.

### 3. Метрики

`System.Diagnostics.Metrics.Meter`:

```
var meter = new Meter("Shop.Orders", "1.0");
var ordersCreated = meter.CreateCounter<int>("orders.created");
// ...
ordersCreated.Add(1);
```

Метрики собираются и экспортируются (Prometheus и т.п.).

### 4. Трейсинг: `Activity` и OpenTelemetry

`Activity` — единица работы (например, HTTP-запрос). OpenTelemetry связывает логи, метрики и трейсы, отправляя их в коллектор.

```
using var activity = new ActivitySource("Shop.Orders").StartActivity("CreateOrder");
// внутри activity можно вешать теги/span-ы
```

### 5. Структурированные логи — зачем

Поиск и фильтрация по полям, агрегация, трассировка запроса по `TraceId`.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Конкатенация в логах | неструктурированно, не фильтруется | параметры `{Name}` |
| `LogInformation` для всего | шум, засоряет | выбирать уровень |
| Логировать секреты | утечка | никогда не логировать пароли/токены |
| Не логировать ошибки в catch | тишина при сбое | `LogError(ex, ...)` |

## Контрольные вопросы

1. Что такое три столпа наблюдаемости?
2. Зачем параметры в сообщениях лога?
3. Что такое метрики?
4. Что такое трейс?
5. Какие бывают уровни логирования?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки

# Практика

# Практика — Модуль 20

> Сначала попробуй решить сам. Решения — в конце файла.

## Задача 1: Сервис с логированием

**Сложность:** ★★☆
**Описание:** Добавь `ILogger` в `OrderService` и логируй создание заказа.

**Требования:**

- Параметры `{OrderId}` в сообщении.
- Уровень `Information`.

## Задача 2: Обработка ошибок с логированием

**Сложность:** ★★☆
**Описание:** В `catch` логируй через `LogError(ex, ...)`.

**Требования:**

- Не проглатывать исключение.
- Лог содержит исключение.

## Задача 3: Метрика

**Сложность:** ★★★
**Описание:** Создай `Meter` и счётчик созданных заказов.

**Требования:**

- `CreateCounter<int>("orders.created")`.
- Наращивай при создании.

## Задача 4: Уровни

**Сложность:** ★☆☆
**Описание:** Разведи уровни: `Information` для обычного, `Warning` для подозрительного, `Error` для сбоев.

**Требования:**

- Примеры для каждого уровня.

## Задача 5: Трейс `Activity`

**Сложность:** ★★★
**Описание:** Оберни создание заказа в `Activity`.

**Требования:**

- `ActivitySource(name).StartActivity("CreateOrder")`.

## Задача 6: Структурированный лог списка

**Сложность:** ★★★
**Описание:** Залогируй количество заказов с параметром.

**Требования:**

- `LogInformation("Заказов: {Count}", orders.Count)`.

## Решения

▶ Задача 1 — решение

▶ Задача 2 — решение

▶ Задача 3 — решение

▶ Задача 4 — решение

▶ Задача 5 — решение

▶ Задача 6 — решение

# Разбор кода

# Разбор кода — Модуль 20

> Формат: плохой код наблюдаемости → находим проблемы → рефакторим.

## Пример 1: Конкатенация в логах

### Исходный код

```
_logger.LogInformation($"Заказ {order.Id} создан");
```

### Что не так

- Строка собирается до вызова: параметр не структурирован, фильтрация и агрегация невозможны.

### Как лучше

```
_logger.LogInformation("Заказ {OrderId} создан", order.Id);
```

### Вывод

Параметры `{...}` — основа структурированного логирования.

## Пример 2: Пустой catch

### Исходный код

```
 catch (Exception)
{
    // тишина
}
```

### Что не так

- Ошибка теряется. Невозможно расследовать инцидент.

### Как лучше

```
 catch (Exception ex)
{
    _logger.LogError(ex, "Не удалось обработать заказ {OrderId}", order.Id);
    throw;
}
```

### Вывод

Логируй ошибки с исключением и перебрасывай.

## Пример 3: Логирование секретов

### Исходный код

```
_logger.LogInformation("Авторизация пользователя {Token}", token);
```

### Что не так

- Секрет попадает в логи. Это утечка.

### Как лучше

Никогда не логировать пароли, токены, ключи. Логировать id пользователя, но не секрет.

### Вывод

Секреты — не для логов.

## Пример 4: Info вместо Warning для ошибок

### Исходный код

```
_logger.LogInformation("Оплата не прошла: {Message}", ex.Message);
```

### Что не так

- Сбой записан как обычное событие — алерты не сработают.

### Как лучше

```
_logger.LogWarning("Оплата отклонена {OrderId}: {Reason}", order.Id, reason);
```

### Вывод

Уровень должен отражать серьёзность.

## Пример 5: «Лог-водопад»

### Исходный код

```
 _logger.LogInformation("Шаг 1");
// ...
_logger.LogInformation("Шаг 2");
// ... десятки шагов
```

### Что не так

- Шум. Трудно найти главное.

### Как лучше

Логировать значимые события и ошибки, а не каждый шаг. Для детального разбора — трейсы.

### Вывод

Качество важнее количества логов.

# Модуль 21: Безопасность веб-приложений

> **Цель:** защищать API: аутентификация, авторизация, секреты и типичные уязвимости.

## Зачем это нужно

Выложить API в интернет без защиты — значит дать злоумышленникам доступ к данным. Безопасность — навык senior: понимать, как аутентифицировать пользователей, как разграничивать права и как не подставиться под типичные атаки (OWASP Top 10).

## Теория

### 1. Аутентификация vs авторизация

- **Аутентификация** — «кто ты?» (проверка личности).
- **Авторизация** — «что тебе можно?» (права).

### 2. JWT (JSON Web Token)

Популярный способ аутентификации API:

- При логине сервер выдаёт подписанный токен.
- Клиент шлёт его в заголовке `Authorization: Bearer <token>`.
- Сервер проверяет подпись, не обращаясь к БД на каждый запрос.

```
 // регистрация схемы JWT
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidIssuer = issuer,
            ValidAudience = audience,
            IssuerSigningKey = new SymmetricSecurityKey(keyBytes)
        };
    });
```

**Секрет ключа** — в конфигурации/env-переменных, никогда в коде.

### 3. Авторизация на основе ролей

```
app.MapGet("/admin", () => "Админ")
   .RequireAuthorization("Admin");
```

Политики и роли — через `AddAuthorization`.

### 4. Секреты

- Хранить в переменных окружения, user-secrets или в безопасном хранилище (Vault).
- **Никогда**: ключи в репозитории, в `appsettings.json` в репо.

### 5. OWASP Top 10 (главное)

| Уязвимость | Защита |
|---|---|
| SQL-инъекции | параметризованные запросы / EF Core (не сырые строки) |
| XSS | не выводить пользовательские данные как HTML без экранирования |
| Подделка запроса (CSRF) | анти-в-RF-токены, проверка Origin |
| Слабые пароли | хэширование (bcrypt/Argon2), политика сложности |
| Утечка данных | ограничить доступ, шифрование, логировать аккуратно |
| Небезопасная десериализация | не принимать произвольные типы |

### 6. Проверка входных данных

Валидация на границе API (модели запросов), а не только на клиенте.

## Типичные ошибки

| Ошибка | Почему плохо | Как правильно |
|---|---|---|
| Секреты в коде/репо | утечка | env-переменные, Vault |
| Отсутствие `RequireAuthorization` | открытые эндпоинты | защищать по необходимости |
| Хранение паролей в открытом виде | кража паролей | хэширование |
| Доверие входным данным | инъекции | валидация и параметризация |

## Контрольные вопросы

1. Чем аутентификация отличается от авторизации?
2. Что такое JWT?
3. Где хранить секреты?
4. Что такое SQL-инъекция и как защититься?
5. Что такое XSS?

## Дальше

- Практика — задачи на закрепление
- Разбор кода — типичные ошибки

# Практика

# Практика — Модуль 21

> Сначала попробуй решить сам. Решения — в конце файла.

## Задача 1: Хэширование пароля

**Сложность:** ★★☆
**Описание:** Используй `PasswordHasher<T>` (или самописный хэш) для хранения пароля.

**Требования:**

- Не хранить пароль в открытом виде.
- Сравнивать только хэши.

## Задача 2: JWT-настройка

**Сложность:** ★★★
**Описание:** Подключи аутентификацию JWT в ASP.NET Core.

**Требования:**

- `AddAuthentication(...) + AddJwtBearer(...)`.
- `TokenValidationParameters` с issuer/audience/ключом.

## Задача 3: Ролевая авторизация

**Сложность:** ★★★
**Описание:** Защити эндпоинт только для роли `Admin`.

**Требования:**

- `RequireAuthorization("Admin")`.

## Задача 4: Секреты через env

**Сложность:** ★★☆
**Описание:** Вынеси строку подключения и ключ в переменные окружения.

**Требования:**

- `Environment.GetEnvironmentVariable("DB_CONNECTION")`.

## Задача 5: Валидация входных данных

**Сложность:** ★★☆
**Описание:** Добавь валидацию модели запроса.

**Требования:**

- Проверка длины/формата.
- Не доверять клиенту.

## Задача 6: Запрет сырых SQL

**Сложность:** ★★★
**Описание:** Перепиши сырой SQL на EF Core/параметризованные запросы.

**Требования:**

- Никакой конкатенации в запрос.

## Решения

▶ Задача 1 — решение

▶ Задача 2 — решение

▶ Задача 3 — решение

▶ Задача 4 — решение

▶ Задача 5 — решение

▶ Задача 6 — решение

# Разбор кода

# Разбор кода — Модуль 21

> Формат: небезопасный код → находим проблемы → рефакторим.

## Пример 1: SQL-инъекция

### Исходный код

```
var sql = $"SELECT * FROM Users WHERE Name = '{name}'";
```

### Что не так

- Если `name` пришёл от пользователя, внедрение `' OR '1'='1` опустошит таблицу.

### Как лучше

```
var users = await db.Users.Where(u => u.Name == name).ToListAsync();
```

### Вывод

Параметризация — обязательна.

## Пример 2: Пароль в открытом виде

### Исходный код

```
db.Users.Add(new User { Password = rawPassword });
```

### Что не так

- При утечке БД крадутся пароли в открытом виде.

### Как лучше

```
var hasher = new PasswordHasher<object>();
db.Users.Add(new User { PasswordHash = hasher.HashPassword(null!, rawPassword) });
```

### Вывод

Храни хэш, не пароль.

## Пример 3: Открытый эндпоинт

### Исходный код

```
app.MapGet("/private/data", handler); // без RequireAuthorization
```

### Что не так

- Доступ без проверки личности.

### Как лучше

```
app.MapGet("/private/data", handler).RequireAuthorization();
```

### Вывод

Защищай приватные маршруты.

## Пример 4: Секреты в коде

### Исходный код

```
var secret = "my-super-secret-123"; // в коде!
```

### Что не так

- Секрет попадёт в репозиторий.

### Как лучше

```
var secret = Environment.GetEnvironmentVariable("JWT_SECRET") ??
    throw new InvalidOperationException("JWT_SECRET is not set");
```

### Вывод

Секреты — только из окружения/хранилища.

## Пример 5: Доверие входным данным

### Исходный код

```
app.MapPost("/upload", (string input) => Process(input));
// input идёт прямо в обработку без проверки
```

### Что не так

- Неконтролируемый ввод — путь к XSS, инъекциям и переполнению.

### Как лучше

Валидируй формат, длину, тип на границе API.

### Вывод

Любой ввод — потенциально опасен.